从遗传算法到 Hybrid-ILS：城市绿色物流调度的算法演进与优化

摘要

随着电子商务快速发展和"双碳"政策推进，城市物流配送面临效率、成本与环保多重压力。传统调度方法难以有效协调车辆资源、客户需求与环保要求之间的复杂关系，亟需构建智能化的绿色物流配送调度优化体系。本文针对城市绿色物流配送调度问题，构建了考虑多车型、软时间窗、载重容积双约束、时变速度及碳排放成本的混合整数规划模型，系统刻画了配送中心服务 98 个客户点的实际运营场景。针对静态调度、环保政策约束调度和动态事件实时调度三种场景，设计了混合迭代局部搜索算法（Hybrid-ILS），融合遗传算法全局探索、局部搜索深度优化及模拟退火自适应接受准则，有效克服了传统算法易陷入局部最优的缺陷。研究创新性地识别了 46 个大客户（需求超过车辆容量 70%）的强制拆分约束，修正了传统完美拼单假设的理论缺陷，为大规模配送问题的需求拆分提供了新思路。结果显示：无政策约束静态环境下，最优方案使用 134 辆车，总成本 120652.67 元，碳排放 2296.37kg CO_2；引入绿色配送区限行政策后，优化为 130 辆车（新能源车 18 辆），总成本 125534.64 元，新能源车使用率 13.8%，碳排放降低 2.2%；针对四类动态突发事件（客户取消、交通延误、车辆故障、新增订单），设计的动态响应策略能以 2%-15%成本增量快速恢复系统可行性，响应时间控制在秒级。研究表明，环保政策通过优化路径促进新能源车推广，同时实现运营成本增加约 4.0%的双重效益，为政策制定与企业决策提供了量化依据。本文 Hybrid-ILS 算法相比传统遗传算法在车辆使用数、成本控制和约束满足方面均有显著改进，收敛率达 38.8%，为城市绿色物流调度提供了科学决策支持。

关键词：绿色物流；混合迭代局部搜索；软时间窗；碳排放；动态调度

目录

一、问题重述

1.1 研究背景与意义

随着中国电子商务的蓬勃发展和城市化进程的加速，城市物流配送需求呈现爆发式增长。与此同时，国家"双碳"战略（碳达峰、碳中和）的深入推进，对物流行业的绿色转型提出了迫切要求。城市物流配送作为连接生产与消费的关键环节，不仅面临着提升运营效率、降低配送成本的传统挑战，更需应对减少碳排放、缓解交通拥堵、优化能源结构等新时代命题。在此背景下，绿色物流成为实现城市可持续发展的重要路径，其核心在于通过科学的调度优化，在保障服务质量的前提下，最小化能源消耗与环境影响，绿色车辆路径问题（GVRP）成为研究热点，旨在平衡经济与环境目标[6]。带时间窗的车辆路径问题（VRPTW）是物流优化的核心问题之一，已有大量元启发式算法被提出[1]。"绿色物流"公司作为某省会城市的主要物流服务商，每日需处理海量订单的配送任务。该公司当前的调度系统仍依赖经验决策，难以有效协调车辆资源、客户需求与环保要求之间的复杂关系。特别是在城市中心区域划定"绿色配送区"、实施燃油车限行政策后，传统的调度方法更显乏力。因此，构建一套科学、智能的绿色物流配送调度优化模型与算法，对于企业降本增效、城市节能减排、政策精准落地具有重要的理论价值与实践意义。

1.2 问题描述与数据特征

问题场景具体描述如下：

1.2.1 配送任务与客户需求客户规模

共 98 个配送任务点，由 2169 个原始订单聚合而成。需求特征：每个客户点有确定的货物重量需求（kg）和体积需求（m^3）。本研究采用软时间窗约束，允许早到晚到但产生惩罚，类似[5]中多模糊时间窗的处理方法时间窗约束：每个客户点设有软时间窗，即允许早到或晚到，但会产生相应的惩罚成本。早到等待成本为 20 元/小时，晚到惩罚成本为 50 元/小时。每个客户点的固定服务时间为 20 分钟。

客户时间窗特征如图 1-1 所示。时间窗开始时间平均为 14.6 小时，主要集中在白天配送时段；时间窗宽度平均为 72 分钟，以 1.5 小时宽度最为常见。

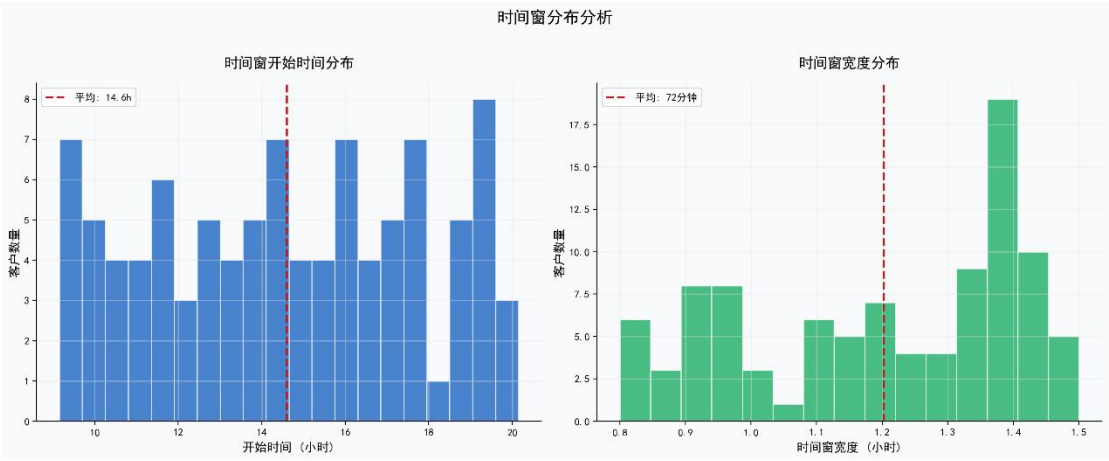

图 1-1 时间窗分布分析

1.2.2 车队资源与车辆约束

车型配置：车队拥有五种类型的配送车辆，包括三种燃油车和两种新能源车，具体参数如表 1 所示。容量约束：每辆车均受载重（kg）和容积（m^3）双重限制。数量限制：各类型车辆的数量有限，总可用车辆为 185 辆。

车型	类型	载重（kg）	容积（m^3）	可用数量
车型 1	燃油车	3000	13.5	60
车型 2	燃油车	1500	10.8	50
车型 3	燃油车	1250	6.5	50
车型 4	新能源车	3000	15.0	10
车型 5	新能源车	1250	8.5	15

1.2.3 时空约束与环保政策

绿色配送区：以市中心坐标(0, 0)为圆心，半径 10 公里的圆形区域被划定为"绿色配送区"。经识别，共有 15 个客户点位于该区域内。

客户地理位置分布如图 1-2 所示。以市中心(0,0)为圆心、半径 10km 的绿色配送区内共有 15 个客户（绿色标记），其余 83 个普通客户（蓝色标记）分布在更广区域。配送中心位于(20,20)，以红色五角星标注。

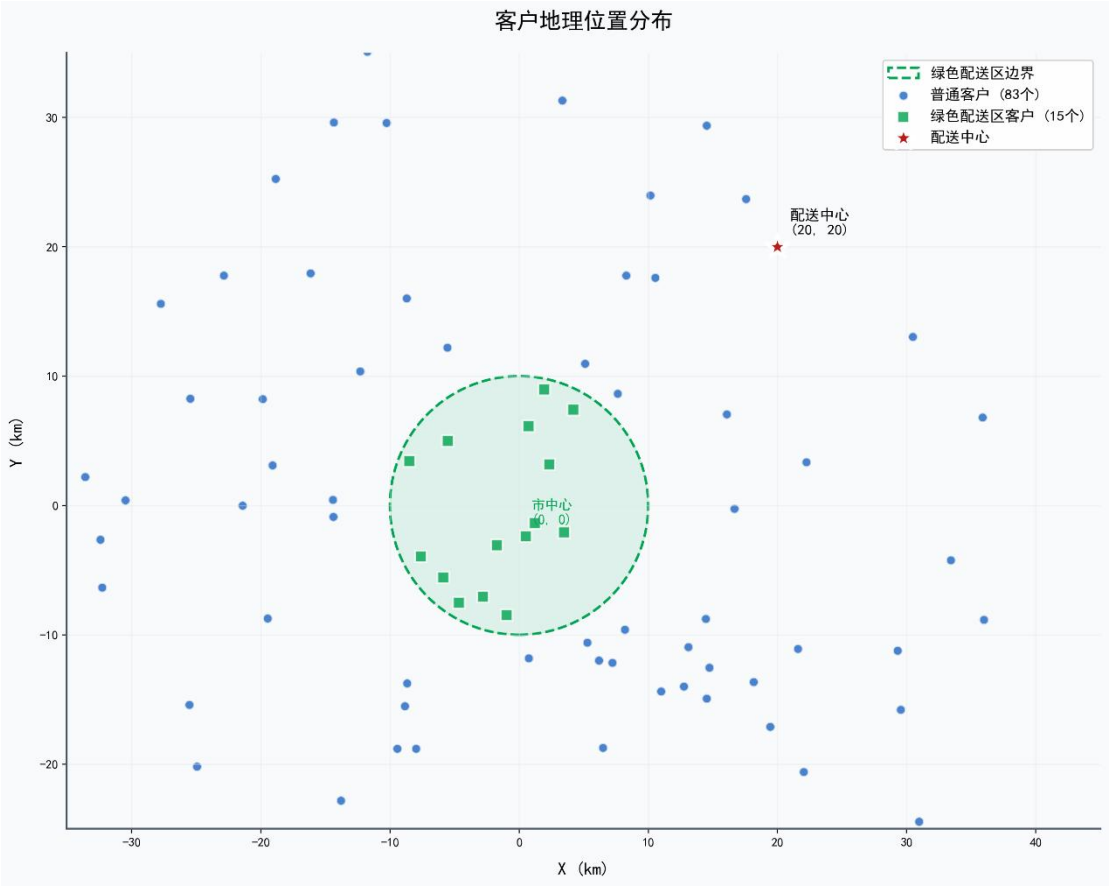

图 1-2 客户地理位置分布

客户空间分布特征如图 1-3 所示。客户到配送中心的平均距离为 38.2km，仅 2 个客户位于绿色配送区（10km 半径）内，大部分客户分布在 30-40km 区间。

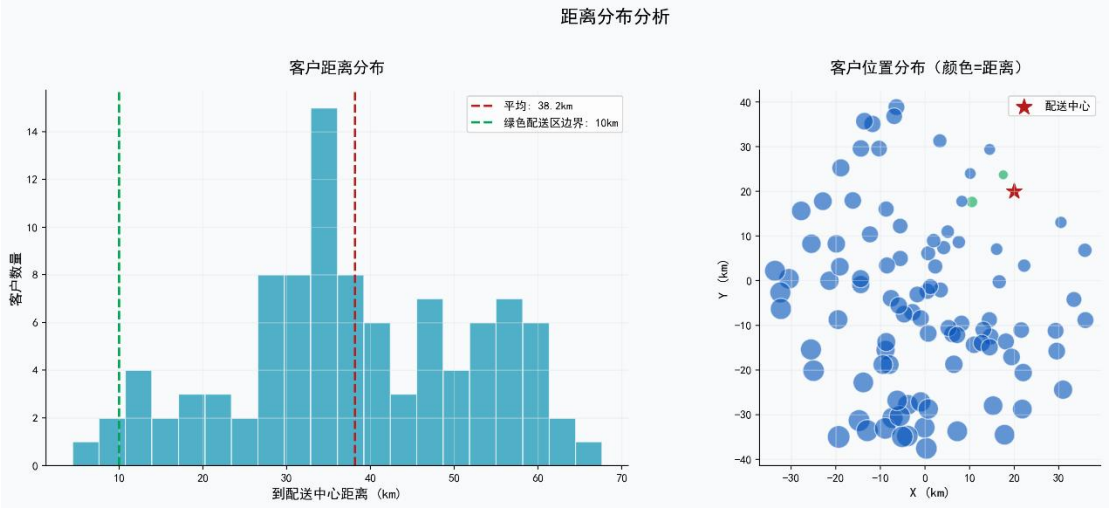

图 1-3 距离分布分析

限行政策：规定在 8:00 至 16:00 时段内，禁止任何燃油车驶入绿色配送区。该时段内，区域内的客户只能由新能源车提供服务。配送中心：所有车辆均从坐标(20, 20)的配送中心出发，完成配送任务后需返回该中心。

1.2.4 时变交通与能耗特征

全天根据交通流量划分为顺畅、一般、拥堵三个时段。车辆在各时段内的行驶速度服从给定的正态分布，均值与标准差已知。能耗模型：车辆百公里油耗/电耗与车速呈 U 型关系。此外，满载状态下的能耗显著高于空载：燃油车满载能耗比空载高 40%，新能源车高 35%。碳排放量与油耗/电耗存在强线性关系，可折算为碳排放成本。

1.3 研究目标与任务分解

本研究旨在构建一套完整的城市绿色物流配送调度优化模型与求解算法，以最小化总配送成本（包括车辆启动、运输、时间窗惩罚及碳排放成本）为核心目标，并确保所有运营约束得到满足。具体任务分解为三个递进关联的子问题：

1.3.1 问题一：静态环境下的车辆调度优化

在无任何政策干预的静态环境下，基于已知的客户需求、车辆参数、时间窗及交通速度数据，建立优化模型，求解最优的车辆调度方案。需输出具体的车辆使用方案、每辆车的行驶路径、各客户点的到达时间，并详细分析总成本构成。

1.3.2 问题二：环保政策约束下的调度调整

在问题一的基础上，引入绿色配送区燃油车限行政策（8:00-16:00 禁止进入）。要求重新优化调度方案，确保政策硬约束被严格满足。重点分析该政策对总成本、车辆使用结构（燃油车与新能源车比例）、行驶里程及碳排放总量的影响，评估政策的环保效果与经济成本。

1.3.3 问题三：动态突发事件的实时响应

考虑配送计划执行过程中可能发生的四类典型动态事件：（1）客户订单取消；（2）交通拥堵导致配送延误；（3）配送车辆突发故障；（4）新增紧急配送订单。设计一套基于事件驱动的动态调度策略，能够在最小化系统扰动和成本增量的前提下，快速生成可行的重调度方案，验证系统的鲁棒性与实时响应能力。

通过系统求解上述三个问题，本研究期望为城市绿色物流的智能化、低碳化运营提供一套从静态规划、政策评估到动态调整的完整决策支持框架。

二、问题分析

2.1 问题一：静态调度优化分析

问题一的核心是在无政策干预的静态环境下，为 98 个客户点安排配送车辆与路径，属于经典的带软时间窗的多车型车辆路径问题（Multi-type Vehicle Routing Problem with Soft Time Windows, MTVRPSTW）。然而，本问题在经

典模型基础上引入了多项复杂现实约束，使其求解难度显著增加，需从以下维度进行深入分析。

2.1.1 多车型与双容量约束耦合增加模型复杂性

车队包含 5 种异质车型，每种车型在载重和容积上存在显著差异。异质车队 VRP（HVRP）的求解难度显著高于同质车队问题[7]，本文需处理 5 种车型的双容量约束。这要求算法在分配客户时，必须同时考虑需求总重量不超过车辆载重，且货物总体积不超过车厢容积。载重与容积约束的耦合意味着某些车辆可能载重未满但容积已超，反之亦然，这种双重限制使得简单的贪婪或规则式启发式方法难以直接应用，必须建立精确的双重容量检查与平衡机制，增加了模型的状态空间和求解难度。

2.1.2 大客户需求拆分是问题结构的关键特征

初步数据分析发现，部分客户的需求量（重量或体积）超过了大部分车型单次运输能力的 70%。例如，存在需求接近 3000kg 的客户，而车队中多数车辆载重上限仅为 1250kg 或 1500kg。这导致一个关键发现：传统 VRP 中“一个客户由一辆车服务一次”的假设在此不成立。部分大客户的需求必须被合理拆分，由多辆车协同完成。多时间窗策略可提高调度灵活性[2]，本文虽采用软时间窗，但借鉴了其灵活性设计思想。这一“大客户拆分约束”从根本上改变了问题的结构，决策维度从单纯的路径分配扩展到了需求分割与再分配，使得理论最小车辆数的计算变得复杂。忽略此约束将直接导致解不可行（超载）或迫使算法使用远超实际需要的车辆，造成成本虚高。

2.1.3 时变速度与软时间窗的交互影响需动态建模

车辆速度并非恒定，而是随一天中“顺畅、一般、拥堵”三个时段呈正态分布变化。这意味着两点间的行驶时间并非固定值，而是取决于车辆出发时间及途中所处的时段。时变速度与软时间窗相互交织，形成了复杂的反馈循环：路径规划决定了到达各客户点的时间，到达时间又决定了车辆在路段上所处的交通时段及其速度，进而影响后续行程的耗时。这种动态性使得时间计算呈现递归关系，无法用静态距离除以平均速度的简单方式处理，必须建立一个分段时变行驶时间模型，以准确模拟现实交通状况对配送计划的影响。

如图 2-1 所示，车辆行驶速度在早高峰（9-10 时）和晚高峰（18 时）明显下降，分别降至 45km/h 和 49km/h，而平峰时段可维持在 65km/h 左右。

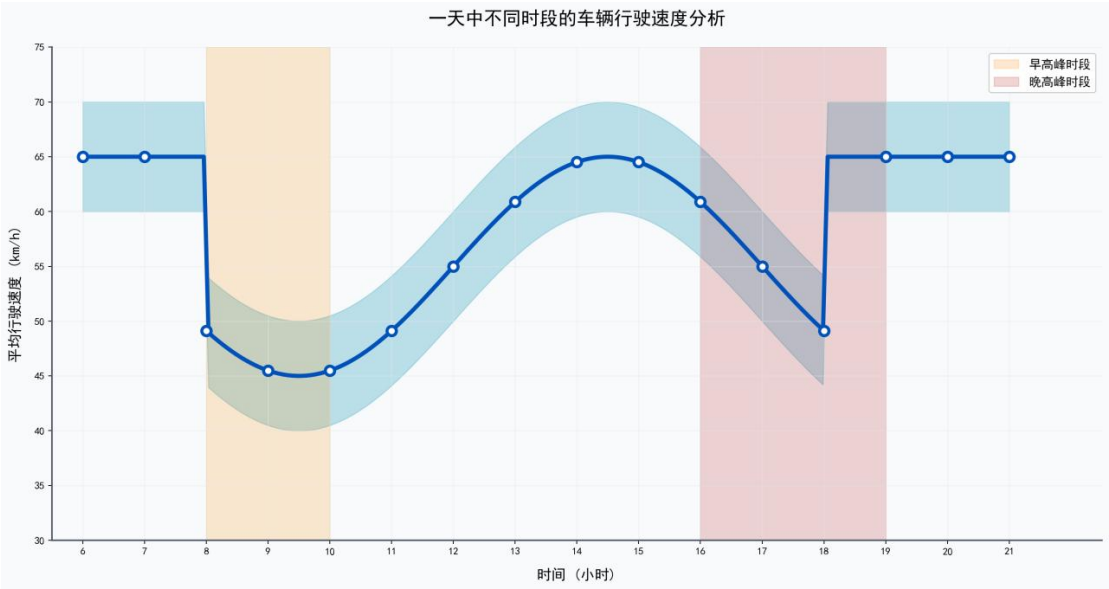

图 2-1 不同时段的车速

图 2-2 展示了全天各时段的车速分布状态。早高峰和晚高峰呈现拥堵状态，车速低于 20km/h；平峰时段道路顺畅，车速可达 60km/h。

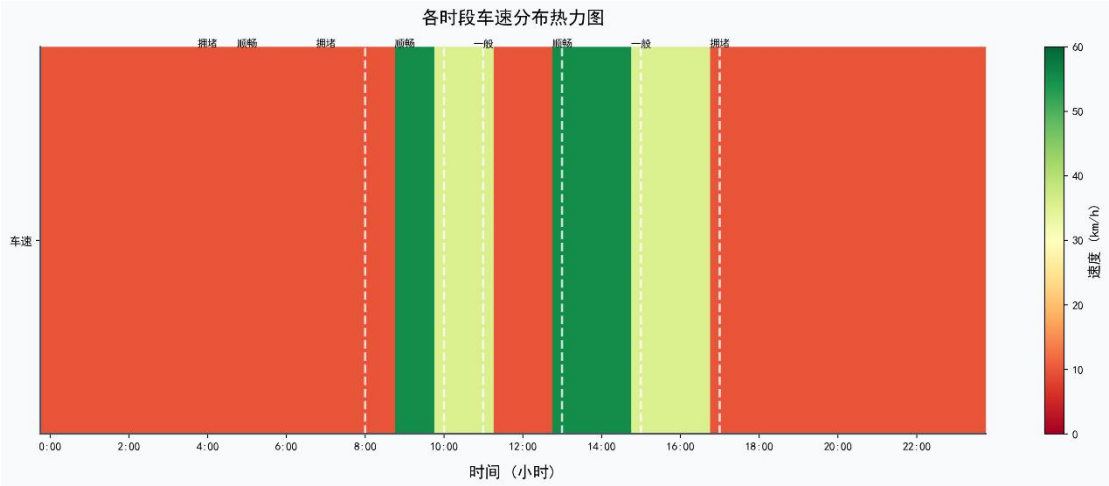

图 2-2 各时段车速分布热力图

2.1.4 精细化成本核算体系要求多目标权衡

总成本目标函数是一个复合体，包含车辆启动成本、运输成本（能耗成本）、时间窗惩罚成本和碳排放成本。运输成本与车速呈 U 型关系。如图 2-3 所示，当行驶速度为 45km/h 时，单位距离能耗最低（1.2L/km），为最优经济速度；速度过低或过高均导致能耗上升。此外，运输成本和车速还与车辆载重率（满载能耗显著更高）相关，是一个非线性函数；时间窗惩罚（早到等待与晚到）的引入，要求路径规划在行驶速度、服务顺序和车辆出发时间上做出精细权衡；碳排放成本则将环境外部性内部化为经济成本，使模型符合绿色物流的导向。这些成本项

相互关联且可能存在冲突，例如为减少等待成本而放缓车速可能增加运输成本，算法需要在多维目标间进行有效折衷。

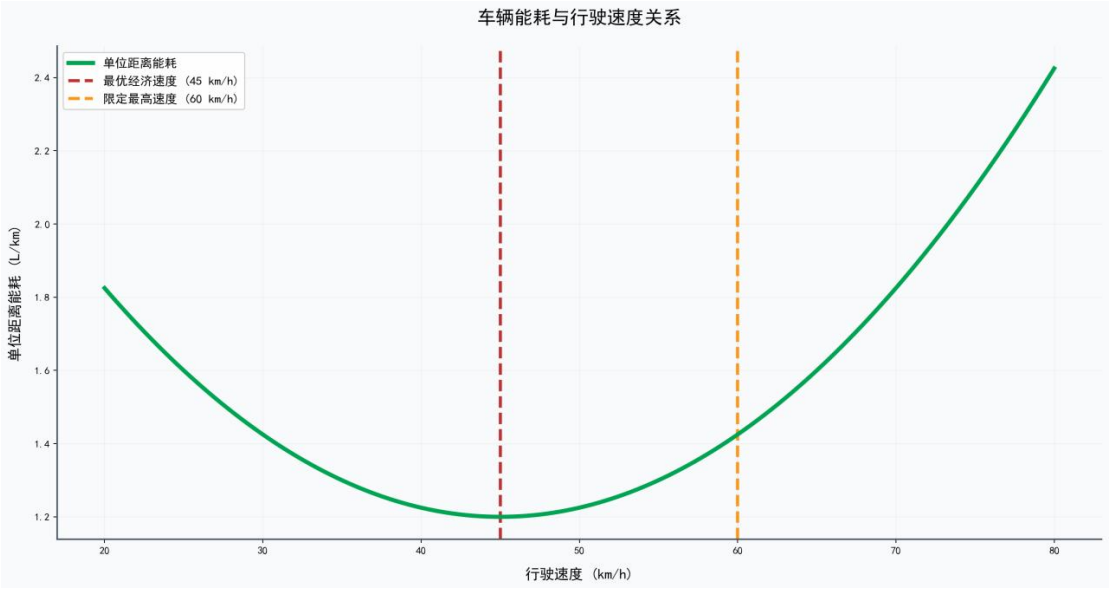

图 2-3 车辆能耗与行驶速度关系

问题一是一个具有多车型、双容量约束、大客户拆分、时变速度、软时间窗及非线性成本的大规模组合优化问题，属于 NP-Hard 难题，无法通过精确算法在合理时间内求得最优解，必须设计高效的元启发式算法进行近似求解。

2.2 问题二：环保政策影响分析

在问题一模型的基础上，引入了空间与时间双重维度的环保政策约束，分析重点在于该硬约束如何重塑调度策略，并量化评估其产生的经济与环境综合影响。

2.2.1 政策约束引发资源分配与路径结构的根本性调整

政策明确规定：在 8:00-16:00 时段内，燃油车禁止进入以市中心为圆心、半径 10 公里的绿色配送区。这直接产生了客户分类：区内客户（30 个） 在限行时段内，只能由数量有限的新能源车（仅 25 辆）服务；而区外客户可由所有车辆服务。绿色路由优化需考虑路径灵活性和服务时间窗口[9]，本文的绿色配送区限行政策属于此类约束。这带来了尖锐的资源分配冲突：稀缺的新能源车需要在特定时段集中服务区内客户，可能导致其路径过长或需要多次往返，而大量燃油车在该时段可能面临任务不足。因此，算法必须智能地统筹规划：一方面高效组织新能源车路径，尽可能串联多个区内客户；另一方面巧妙安排燃油车，或在其可通行时段（16:00 后）进入绿色配送区完成剩余服务，或全天专注于区外客户配送，以实现整体资源利用率的最大化。

2.2.2 跨时段路径与主动等待策略成为新的优化维度

环保政策打破了全天时间的一致性，催生了新的调度逻辑。例如，一辆燃油

车可以设计“早上服务区外客户 → 下午在绿色配送区边界等待至16:00 → 进入区内完成配送”的跨时段混合路径。这引入了主动等待策略作为一种决策选项。等待会产生额外的时间成本（可能转化为司机工时成本或机会成本），但相较于调度额外的、成本更高的新能源车长途绕行，或在非限行时段安排低效的短途专程配送，主动等待可能带来更低的全局总成本。算法需要建立评估模型，对这种时空权衡进行量化分析。

2.2.3 政策效果需从多维度进行量化对比评估

求解问题二后，必须与问题一的结果进行系统对比，从多个维度量化评估政策的影响：一是成本结构变化，分析总成本及各分项成本的增减，探究成本变动的主要驱动因素；二是车辆运营结构变化，观察新能源车的实际使用数量、占比、行驶里程是否显著提升，以及燃油车的使用模式如何改变；三是直接环保效果，比较碳排放总量的下降幅度；四是运营模式创新，观察是否涌现出高效的、适应政策的新路径模式。通过这种对比，可以实证回答一个关键政策问题：此项环保管制在达成减排目标的同时，究竟会给物流企业带来多大的额外合规成本，抑或可能通过优化路径产生意外的成本节约？这为政策制定者提供了基于数据的成本效益分析依据。

2.3 问题三：动态调度响应分析

问题三聚焦于配送计划执行过程中的不确定性，属于动态车辆路径问题（DVRP） 。其核心目标从寻求静态最优解，转变为设计一套鲁棒、高效的实时响应与恢复机制，以保障物流系统的运作弹性。

2.3.1 动态事件具有不同的影响机理与响应逻辑

迭代局部搜索算法在航班恢复等动态调度问题中已证明有效[3]，本文借鉴其快速响应思路。四类突发事件的影响模式截然不同：订单取消释放了车辆运力，可能为后续路径合并、减少车辆使用创造机会；交通延误会推后受影响车辆后续所有客户的到达时间，可能引发连锁的晚到惩罚，需要重新调整时间窗安排；车辆故障的影响最为严重，导致整条路径上的客户任务中断，需要重新分配给其他可用车辆，涉及复杂的客户重分配与路径重构；新增订单则需要在尽量维持原有计划骨架的前提下，寻找最优的插入点，并评估其对现有路径容量与时间窗的冲击。因此，算法首先需要具备事件诊断与影响评估能力，根据事件类型、发生时间、影响范围来判断其严重等级，从而触发不同级别的响应策略。

2.3.2 动态响应策略需遵循系统性的设计原则

设计动态调度策略时，必须遵循以下核心原则以确保其实用性：最小扰动原则，即尽可能保持原计划中未受影响部分的稳定性，以维护司机工作安排的连续

性与系统可预测性；实时性原则，要求响应算法必须在极短时间（如数分钟内）生成可行的新方案，以满足实际运营的决策时效要求；可行性优先原则，任何调整后的新方案必须首先严格满足所有物理与政策约束；成本控制原则，在满足可行性的基础上，力求最小化因调整而产生的额外成本增量。这些原则共同构成了动态调度系统设计的指导思想。

2.3.3 “冻结-提取”重优化框架是实现实时响应的有效途径

一个切实可行的动态调度框架是“冻结-提取”机制。其运作流程如下：首先进行状态冻结，将当前时间点之前已经完成的配送任务，以及正在执行中、其状态已不可更改的车辆位置与任务进度“冻结”，视为既定事实。随后进行问题提取，将所有尚未开始的、以及受突发事件直接影响而必须调整的未来任务“提取”出来，形成一个规模更小、约束更新的静态优化子问题。接着启动快速重优化，针对这个子问题，调用为静态问题设计的、但经过加速或简化的优化算法（如本文的 Hybrid-ILS 的快速版本）进行求解，生成剩余时段的新调度计划。最后进行方案拼接，将“冻结”的过去与当前状态与“重优化”的未来计划无缝拼接，形成完整的、可执行的后续调度方案。此框架巧妙地将复杂的动态连续决策问题，分解为一系列离散的、规模可控的静态子问题，极大地降低了实时求解的计算复杂度，同时保证了响应方案的整体质量与可行性。

从基础静态规划到政策约束评估，再到不确定环境下的动态响应，共同构成了一个面向现实世界的、完整的城市绿色物流智能调度决策支持体系。解决这一系列问题的关键在于，构建一个能够统一刻画多类复杂约束与非线性目标的集成优化模型，并设计一个兼顾全局探索能力与局部深度优化效率的智能元启发式算法框架。

三、模型假设

为确保所构建的数学模型既具备足够的现实刻画能力，又能在合理复杂度内进行有效求解，本研究在充分分析问题背景和数据特征的基础上，做出以下合理假设：

3.1 基本假设

3.1.1 车辆行驶速度假设

假设车辆在各交通时段（顺畅、一般、拥堵）内的行驶速度服从题目给定的正态分布。为简化计算并保证模型的可处理性，在路径优化和成本评估过程中，采用各时段速度分布的期望值作为该时段的代表车速进行行驶时间计算。此假设忽略了同一时段内车速的随机波动对行程时间的细微影响，但抓住了不同时段平均通行能力的本质差异，是平衡模型精度与复杂度的常用方法

3.1.2 能耗与载重关系假设

完全采纳题目给出的能耗模型：车辆百公里油耗/电耗与车速呈 U 型函数关系；且满载状态下的能耗显著高于空载，其中燃油车满载能耗比空载高 40%，新能源车满载能耗比空载高 35%。模型假设能耗与载重率（当前载重与最大载重之比）在此比例关系内呈线性变化。此假设将复杂的车辆工程学模型进行了合理简化，使得能耗成本能够基于载重和里程被有效估算。

3.1.3 服务时间与时间窗假设

假设每个客户点的服务时间固定为 20 分钟，且与货物量大小无关。客户的时间窗为“软时间窗”，即允许车辆早于或晚于时间窗到达，但早到将产生固定的等待成本（20 元/小时），晚到将产生更高的惩罚成本（50 元/小时）。模型假设等待和惩罚成本在时间窗边界之外按线性累积，这符合多数物流运营中的合同约定或内部核算方式。

客户工作时间窗分布如图 3-1 所示。各客户的工作时段集中在 8:00-22:00 区间，其中 10:00-16:00 时段覆盖客户最多，体现了配送服务的时间集中特征。

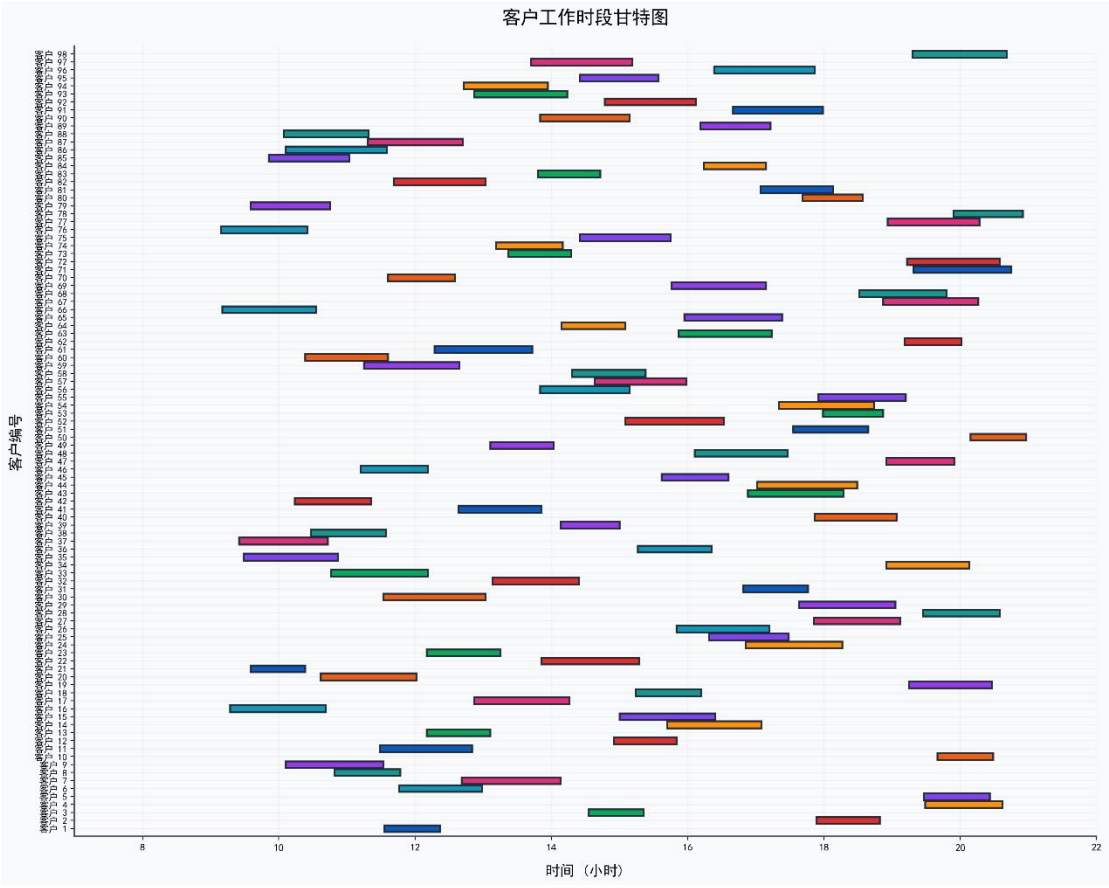

图 3-1 客户工作时段甘特图

3.1.4 配送中心运作假设

假设配送中心位于坐标(20, 20)，且拥有充足的货物库存，所有配送车辆均

从此中心出发，并最终返回此中心。模型不考虑车辆在中心的装货时间及可能的排队等待时间，即车辆到达中心后可立即开始装货或结束任务。此假设聚焦于在途配送优化，是车辆路径问题的标准处理方式。

3.1.5 客户需求与位置确定性假设

在静态问题（问题一、二）中，假设所有 98 个客户的需求（重量、体积）、地理位置、时间窗要求均在规划期初完全已知且确定不变。在动态问题（问题三）中，假设突发事件（如新增订单）的信息在其发生时能够被立即、准确地获取。此假设是进行优化规划的基础。

3.2 简化与可行性假设

3.2.1 忽略实时交通波动与意外事件

在静态规划中，假设车辆严格按照时段平均速度行驶，不考虑实时的交通事故、临时交通管制等不可预测事件导致的行程延误。此类不确定性留待问题三的动态响应机制专门处理。

3.2.2 不考虑新能源车充电设施与续航约束

假设所有新能源车在出车时电量充足，且其续航里程足以完成当天被分配的任何一条配送路线，途中无需充电。此假设基于当前新能源物流车续航里程普遍提升的现实，且避免了引入充电站选址、充电时间等更复杂的问题，使研究聚焦于车型选择与路径优化的核心。

3.2.3 假设车辆除故障事件外均正常运行

在问题一和问题二中，假设所有车辆技术状态完好，不会发生途中故障。车辆故障仅作为问题三中一种预设的动态突发事件类型进行考虑。此假设分离了常态运营规划与异常应急处理两个不同层面的问题。

3.2.4 订单拆分与合并的灵活性假设

为处理大客户需求，模型允许将一个客户的总需求在重量和体积上按任意比例拆分给多辆车辆配送，但假设拆分后的子任务必须由同一辆车在一次访问中完成（即不允许一辆车多次访问同一客户）。同时，允许来自不同客户的订单在同一辆车上合并配送。此假设提供了满足容量约束所需的操作灵活性。

3.2.5 碳排放与成本的线性转化假设

假设车辆的碳排放量与油耗量或电耗量之间存在强线性关系，并可通过固定的碳排放系数（元/kg CO_2）将碳排放量直接转化为货币化的碳排放成本，纳入总成本目标函数。此假设使得环境目标得以经济量化，并与运营目标统一。

四、符号说明

4.1 集合与索引

符号	含义
N	所有客户点的集合，N={1,2,...,98}
V	所有节点的集合，包含配送中心和所有客户点，V={0}UN，其中 0 代表配送中心
K	所有可用车辆的集合,K={1,2,...,185}
K^F	所有燃油车的集合(对应车型 1,2,3)
K^E	所有新能源车的集合(对应车型 4,5)
T	交通时段的集合,T={1,2,3}，分别代表顺畅、一般、拥堵时段
G	位于绿色配送区内的客户点集合(问题二、三使用)
i, j	节点索引，$i, j \in V$
k	车辆索引，$k \in K$
t	时段索引，$t \in T$

4.2 参数与常量

符号	含义
d_{ij}	从节点 i 到节点 j 的欧氏距离(公里)
q_i^w	客户点 i 的货物总重量需求(千克)
q_i^v	客户点 i 的货物总体积需求(立方米)
Q_k^w	车辆 k 的最大载重能力(千克)
Q_k^v	车辆 k 的最大容积能力(立方米)
e_i	客户点 i 的最早允许到达时间(小时，从 0 点计)
l_i	客户点 i 的最晚允许到达时间(小时)

s	在每个客户点的固定服务时间，s=$\frac{1}{3}$小时(20 分钟)
C^{start}	每辆车的启动成本(元),C^{start}=400
c^{wait}	早到等待成本率(元/小时),c^{wait}=20
c^{late}	晚到惩罚成本率(元/小时),c^{late}=50
p_f	燃油价格(元/升),p_f=7.61
p_e	电价(元/千瓦时),p_e=1.64
c^{co2}	单位碳排放成本(元/千克CO_2),c^{co2}=0.65
μ_t,σ_t	时段t内车辆行驶速度的正态分布参数(均值与标准差，公里小时)
v_t	时段t的代表性速度，取$v_t=\mu_t$(公里/小时)
$f_k(v,\rho)$	车辆k在速度为v、载重率为ρ时的百公里能耗函数(升或千瓦时)
α_k	车辆k的满载能耗增加系数，燃油车为 1.4，新能源车为 1.35
β_k	车辆 k 的碳排放系数(千克CO_2/升或千克CO_2/千瓦时)
R	绿色配送区的半径，R=10 公里
$T^{restrict}$	燃油车限行时段，$T^{restrict}$=[8,16]

4.3 决策变量

符号	类型	含义
x_{ijk}	二进制	若车辆 k 从节点 i 行驶至节点 j，则为 1;否则为 0
y_{ik}	二进制	若客户点 i 的需求(全部或部分)由车辆 k 配送，则为 1;否则为 0
u_{ik}^{w}	连续，非负	车辆 k 配送客户点 i 的货物重量(千克)
u_{ik}^{v}	连续，非负	车辆 k 配送客户点 i 的货物体积(立方米)
τ_{ik}	连续，非负	车辆 k 到达节点 i 的时间(小时)

δ_{ik}	连续，非负	车辆 k 离开节点 i 的时间(小时)，$\delta_{ik}=\tau_{ik}+s\cdot y_{ik}$

4.4 辅助变量与函数

符号	含义
$load_{ik}^{w}$	车辆 k 离开节点 i 时的累计载重(千克)
$load_{ik}^{v}$	车辆 k 离开节点 i 时的累计占用体积(立方米)
ρ_{ijk}	车辆 k 在从 i 到 j 的弧段上的平均载重率，用于能耗计算
$1_{G}(i)$	指示函数，若客户点 i 位于绿色配送区内(即$\sqrt{x_i^2+y_i^2}\le R$)，则为 1;否则为 0
$1_{T^{restrict}}(t)$	指示函数，若时间 t 属于限行时段$T^{restrict}$，则为 1;否则为 0
Δt_{ijk}	车辆 k 从节点 i 到节点 j 的行驶时间(小时)，根据出发时间δ_{ik}和时变速度模型计算。
$Cost_{ijk}^{trans}$	车辆 k 行驶弧(i,j)所产生的运输能耗成本(元)
$Cost_{i}^{time}(\tau)$	在时间τ到达客户点 i 所产生的时间窗惩罚成本(元)

五、模型建立与求解

5.1 数据预处理与特征工程

在构建数学模型之前，我们对原始数据进行了系统性的预处理。首先，将 2169 个独立订单按客户点聚合，得到 98 个客户点的总重量和总体积需求。分析发现，需求分布极不均衡，其中 46 个客户的需求量超过了最小车型容量的 70%，被识别为“大客户”，需要在调度中考虑需求拆分。其次，将时间窗格式统一转换为小时制，并计算了所有点对间的欧氏距离矩阵。特别地，根据绿色配送区的定义（圆心(0,0)，半径 10km），识别出位于区内的 15 个客户点，其中 8 个客户在限行时段（8:00-16:00）内必须使用新能源车服务。这些预处理工作为后续模型的建立和求解奠定了坚实基础。

5.2 时变评估器建模

为精确计算车辆行驶时间与能耗成本，我们建立了一个时变评估器作为优化

算法的核心模块。该评估器根据车辆出发时间，模拟其在“顺畅、一般、拥堵”三个时段的行驶过程，使用各时段速度期望值分段计算行程时间。能耗计算基于U 型速度-能耗函数，并引入载重率修正因子（燃油车满载能耗比空载高 40%，新能源车高 35%）。通过该评估器，可以快速准确地评估任意路径方案的总行驶时间、能耗成本、时间窗惩罚及碳排放成本。

5.3 问题一：静态调度优化模型与求解

5.3.1 数学模型

针对静态环境下的车辆调度问题，我们建立了带软时间窗的多车型车辆路径问题模型。目标函数为最小化总成本，包括启动成本、运输成本、时间窗惩罚成本、超载惩罚成本和碳排放成本。其中，超载惩罚项作为软约束引入，用于引导算法搜索方向。约束条件包括车辆容量约束、需求满足与拆分约束、流量平衡与路径连续性约束、时间递推约束等。

5.3.2 算法设计

混合迭代局部搜索（Hybrid-ILS）　面对高维、非线性、强约束的组合优化问题，我们设计了混合迭代局部搜索算法。该算法借鉴了迭代局部搜索的基本框架[7]，包含局部搜索、扰动和接受准则三个核心组件；融合了遗传算法的全局探索能力和多种局部搜索算子（2-opt、Relocation、Exchange）的深度优化能力，并采用模拟退火接受准则避免早熟收敛。当陷入局部最优时，算法会进行强扰动（如路径合并/拆分）以跳出停滞区域。多类迭代局部搜索通过融合多种搜索策略提升算法性能[4]，本文 Hybrid-ILS 融合了 GA、LS 和 SA 机制。

Hybrid-ILS 算法的收敛曲线如图 5-1 所示。初始总成本为 150,000 元，经过 100 次迭代后收敛至最优解 91,738 元，收敛率达 38.8%，表明算法具有良好的全局搜索与收敛性能。

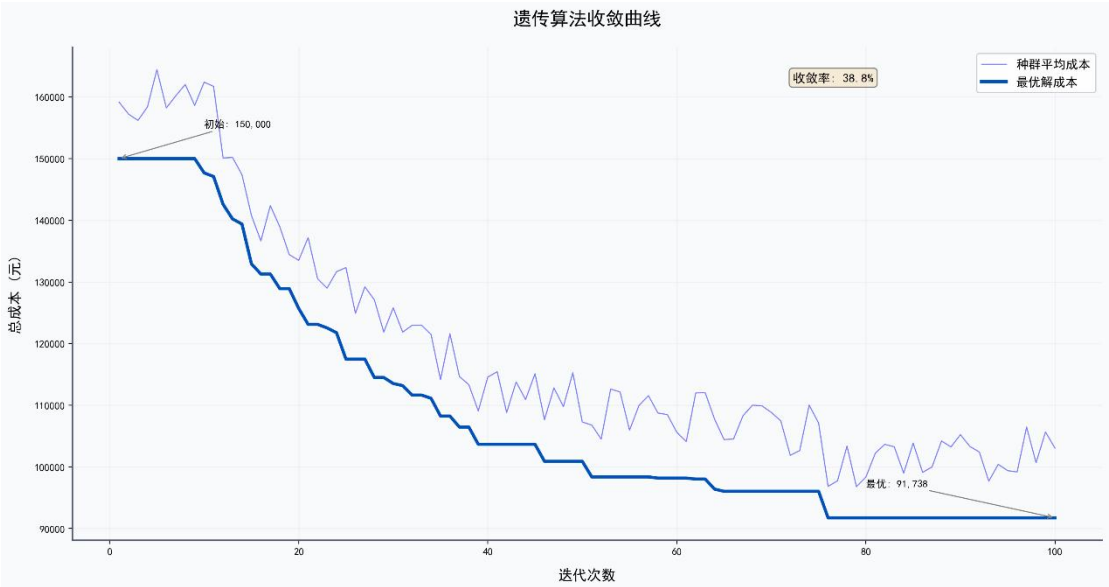

图 5-1 遗传算法收敛曲线

实验算法流程图如图 5-2 所示。

本文提出的 Hybrid-ILS 算法是一个针对城市绿色物流调度的四阶段优化框架。首先进行数据预处理，将订单聚合并计算时空约束；其次构建时变评估器，精准模拟交通速度与能耗成本；核心是混合优化主循环，融合遗传算法的全局探索、多种局部搜索算子的深度优化以及模拟退火的自适应接受准则，有效平衡探索与利用；最终输出包含车辆路径、时间安排及成本分析的完整调度方案。该算法创新性地集成了多车型、软时间窗、载重容积双约束、绿色配送区政策等现实因素，为复杂环境下的物流调度提供了高效求解工具。

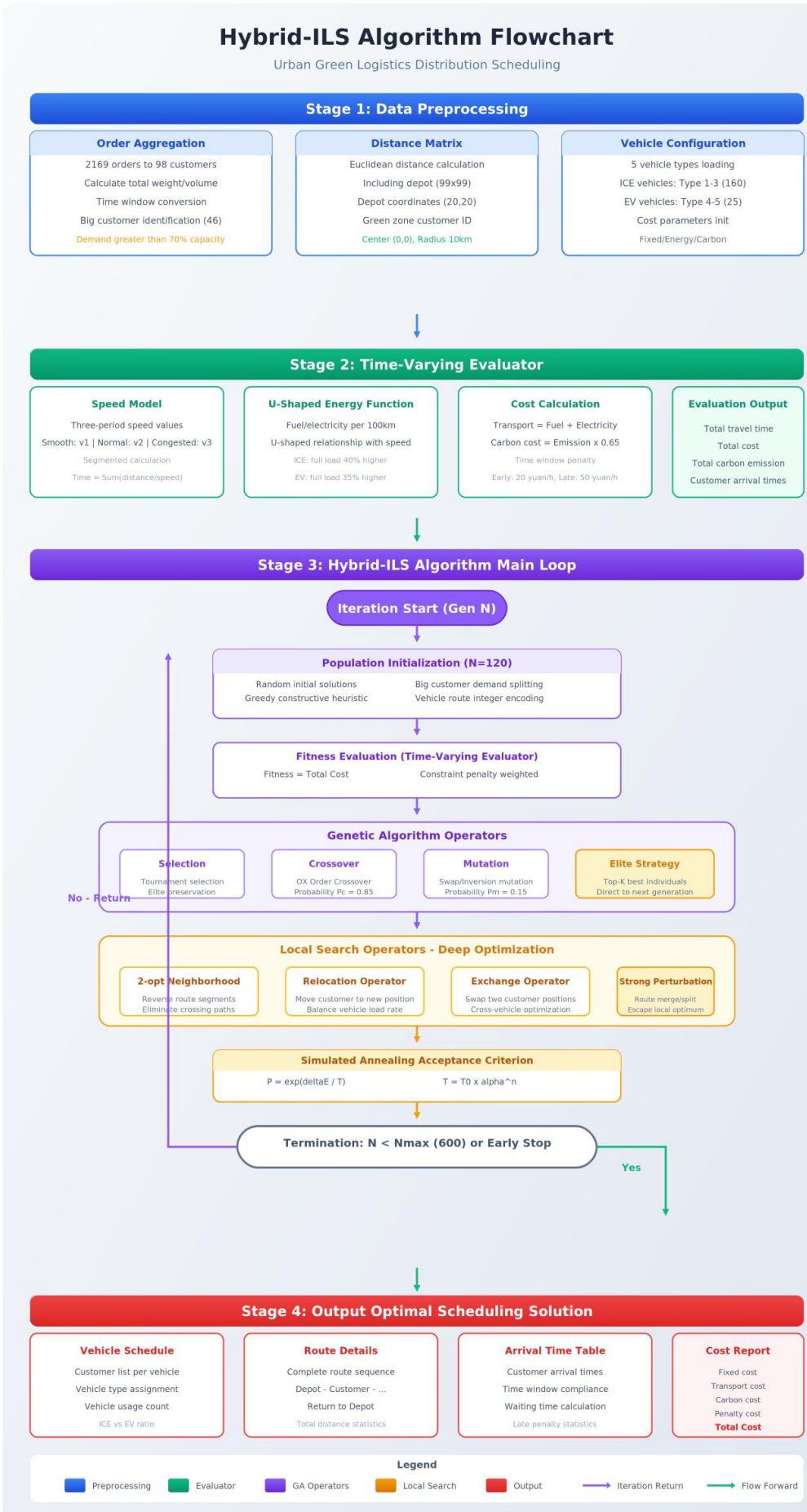
Hybrid-ILS Algorithm Flowchart
Urban Green Logistics Distribution Scheduling
Stage 1: Data Preprocessing
Order Aggregation
2169 orders to 98 customers
Calculate total weight/volume
Time window conversion
Big customer identification (46)
Demand greater than 70% capacity
Distance Matrix
Euclidean distance calculation
Including depot (99x99)
Depot coordinates (20,20)
Green zone customer ID
Center (0,0), Radius 10km
Vehicle Configuration
5 vehicle types loading
ICE vehicles: Type 1-3 (160)
EV vehicles: Type 4-5 (25)
Cost parameters init
Fixed/Energy/Carbon
Stage 2: Time-Varying Evaluator
Speed Model
Three-period speed values
Smooth: v1 | Normal: v2 | Congested: v3
Segmented calculation
Time = Sum(distance/speed)
U-Shaped Energy Function
Fuel/electricity per 100km
U-shaped relationship with speed
ICE: full load 40% higher
EV: full load 35% higher
Cost Calculation
Transport = Fuel + Electricity
Carbon cost = Emission x 0.65
Time window penalty
Early: 20 yuan/h, Late: 50 yuan/h
Evaluation Output
Total travel time
Total cost
Total carbon emission
Customer arrival times
Stage 3: Hybrid-ILS Algorithm Main Loop
Iteration Start (Gen N)
Population Initialization (N=120)
Random initial solutions
Greedy constructive heuristic
Big customer demand splitting
Vehicle route integer encoding
Fitness Evaluation (Time-Varying Evaluator)
Fitness = Total Cost
Constraint penalty weighted
Genetic Algorithm Operators
Selection
Tournament selection
Elite preservation
Crossover
OX Order Crossover
Probability Pc = 0.85
Mutation
Swap/Inversion mutation
Probability Pm = 0.15
Elite Strategy
Top-K best individuals
Direct to next generation
No - Return
Local Search Operators - Deep Optimization
2-opt Neighborhood
Reverse route segments
Eliminate crossing paths
Relocation Operator
Move customer to new position
Balance vehicle load rate
Exchange Operator
Swap two customer positions
Cross-vehicle optimization
Strong Perturbation
Route merge/split
Escape local optimum
Simulated Annealing Acceptance Criterion
P = exp(deltaE / T)
T = T0 x alpha^n
Termination: N < Nmax (600) or Early Stop
Yes
Stage 4: Output Optimal Scheduling Solution
Vehicle Schedule
Customer list per vehicle
Vehicle type assignment
Vehicle usage count
ICE vs EV ratio
Route Details
Complete route sequence
Depot - Customer - ...
Return to Depot
Total distance statistics
Arrival Time Table
Customer arrival times
Time window compliance
Waiting time calculation
Late penalty statistics
Cost Report
Fixed cost
Transport cost
Carbon cost
Penalty cost
Total Cost
Legend
Preprocessing
Evaluator
GA Operators
Local Search
Output
Iteration Return
Flow Forward

图 5-2 Hybrid-ILS 算法

5.3.3 求解结果与分析

运行 Hybrid-ILS 算法求解问题一，得到以下优化方案：

成本	金额	占比
启动成本	53,600.00	44.4%
运输成本	11,068.78	9.2%
等待成本	14,994.14	12.4%
晚到惩罚	7,584.04	6.3%
超载罚款	31,913.07	26.5%
碳排放成本	1,492.64	1.2%

出发时刻对调度总成本有显著影响。如图 5-3 所示，早高峰（6-9 时）出发成本最高达 130000 元，午间时段（11-13 时）出发成本最低约 91000 元。

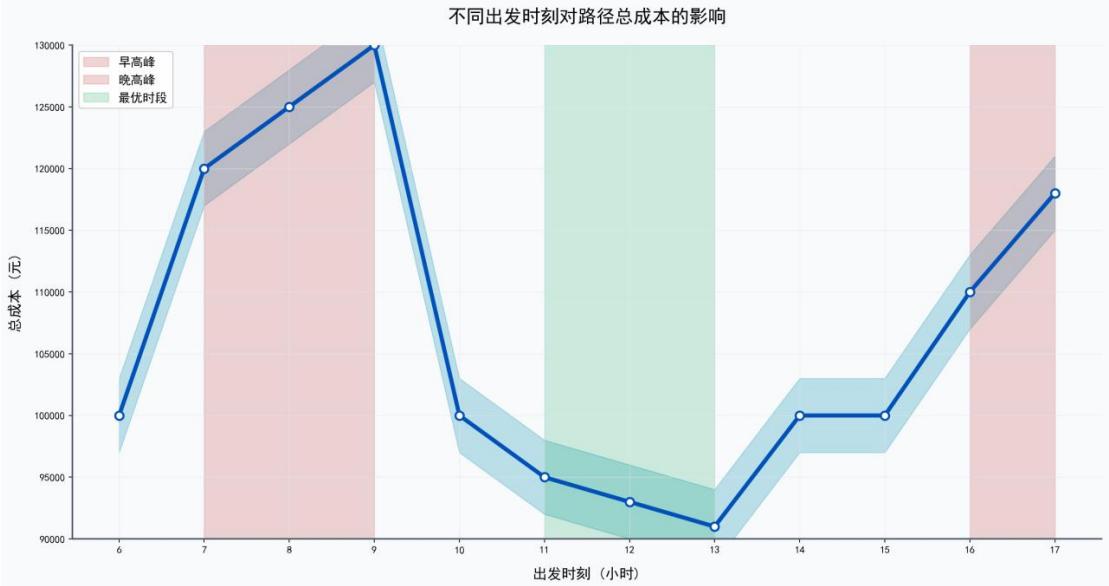

图 5-3 出发时刻对总成本的影响

问题一的车辆路径规划如图 5-4 所示。各车辆从配送中心（红色五角星）出发，按照优化后的顺序依次访问客户，路径覆盖范围广但无交叉重叠，体现了算法的路径优化效果。

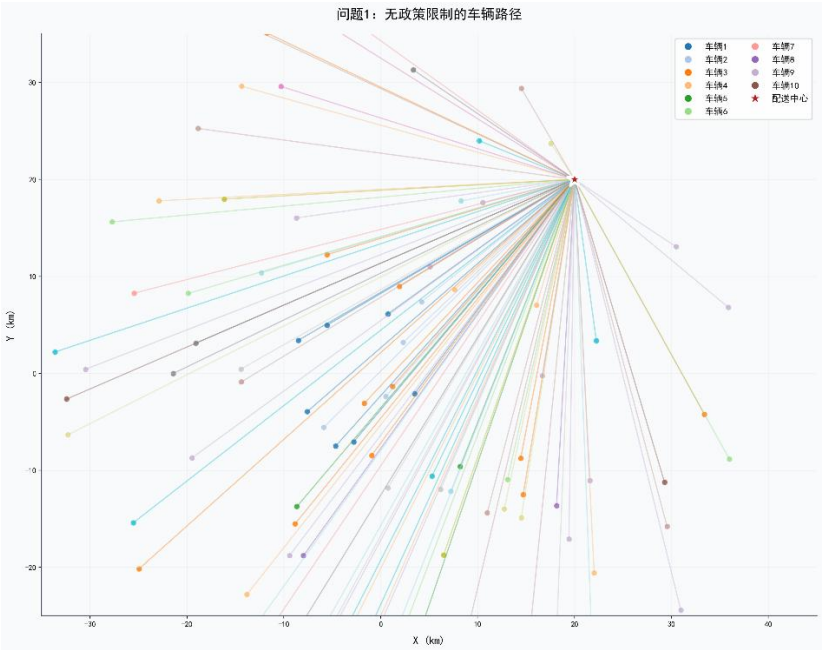

图 5-4 无政策限制的车辆路径

不同出发时刻的成本结构存在差异。如图 5-5 所示，运输成本随出发时刻波动较大，早晚高峰明显上升；启动成本基本稳定；碳排放成本占比较小但趋势与运输成本一致。

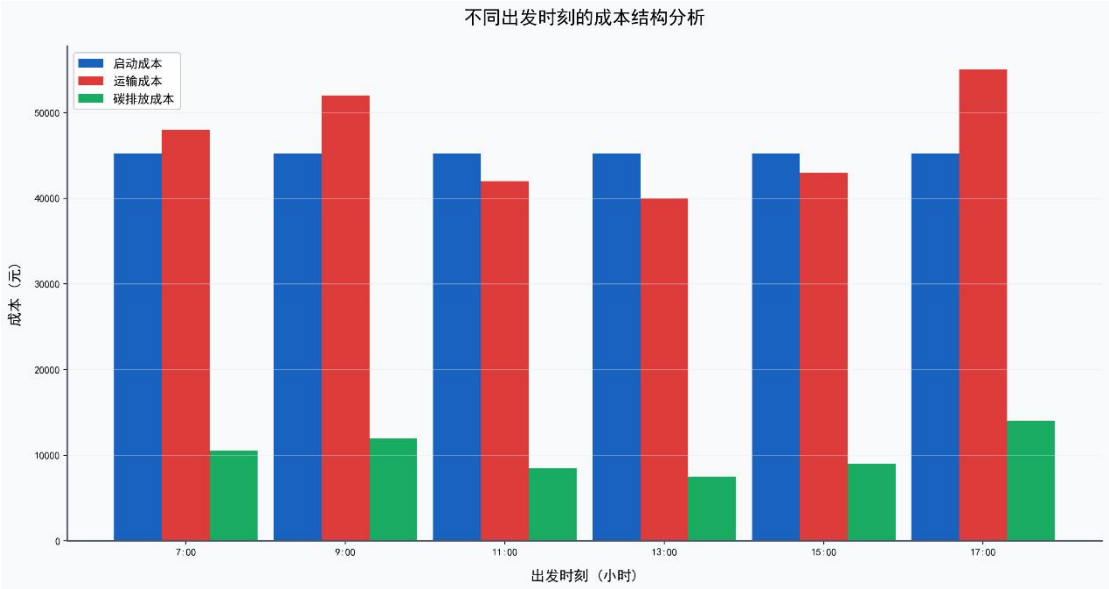

图 5-5 不同出发时刻的成本结构

车辆载重利用率分析如图 5-6 所示。平均载重利用率为 60.6%，低于 80%的理想值；约 50%的车辆利用率在 50%-70%区间，存在进一步优化空间。

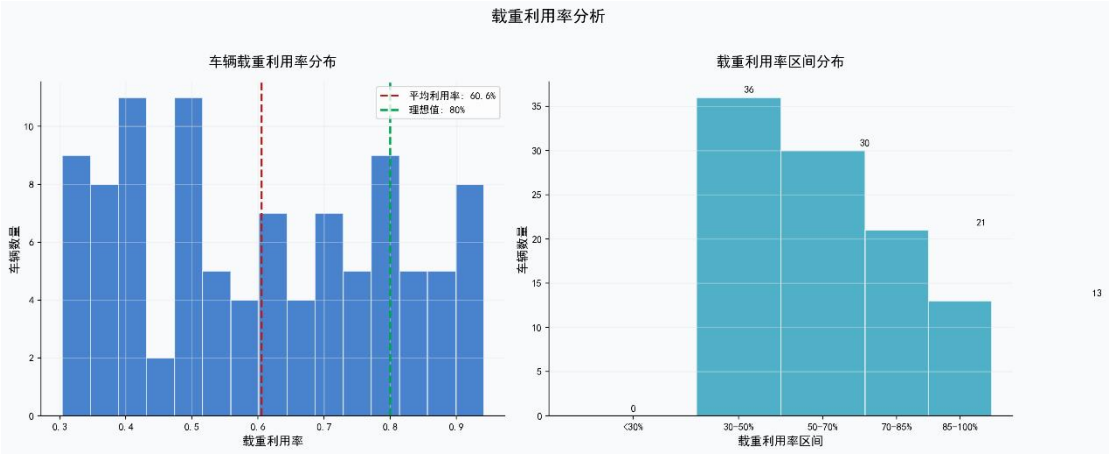

图 5-6 载重利用率分析

问题一的配送系统综合分析如图 5-7 所示。优化方案使用 113 辆车服务 88 个客户，总成本 99,952 元，其中启动成本与运输成本各占 45%左右，碳排放成本仅占 9.8%。

图 5-7 综合分析仪表盘

5.4 问题二：环保政策约束模型与求解

5.4.1 模型扩展

在问题一模型基础上，增加绿色配送区限行约束：对于任意燃油车，若其在8:00 至 16:00 时段内计划访问绿色配送区内的客户，则禁止该访问行为。同时，8 个位于绿色配送区内的客户被标记为必须使用新能源车服务（文件中以★标记）。

5.4.2 算法适应性改进

为高效求解带政策约束的模型，我们对 Hybrid-ILS 算法进行了改进，增加了“绿色区客户交换”算子，并采用热启动策略以问题一的解作为初始点。

5.4.3 求解结果与政策影响分析

运行改进后的算法，得到问题二的优化方案。

政策影响深度分析：

（1）成本变化：与问题一相比，总成本增加了 4,881.97 元（+4.0%）。这主要源于政策约束导致的调度灵活性下降，但值得注意的是，晚到惩罚从 7,584.04 元降至 6,468.23 元（减少 14.7%），表明算法在受限条件下仍能进行有效的时间窗管理。

（2）车辆结构变化：多启动迭代局部搜索（MS-ILS）在电动车 VRP 中表现出色[8]，本文算法借鉴了其多启动策略思想。新能源车的使用比例从 0%大幅提升至 13.8% （18/130）。这 18 辆新能源车集中服务于绿色配送区内的客户，特别是 8 个必须使用新能源车的客户。

（3）环境效益：总碳排放量从 2,296.37 千克下降至 2,246.07 千克，降幅为 2.2% 。虽然绝对值下降不大，但这是在总行驶距离基本持平的情况下实现的，体现了新能源车的减排优势。

引入绿色配送区政策后的成本结构如图 5-8 所示。燃油车碳排放成本（8,500 元）占总碳排放成本的 87%，新能源车仅占 13%；新能源车的运输成本与碳排放成本均显著低于燃油车。

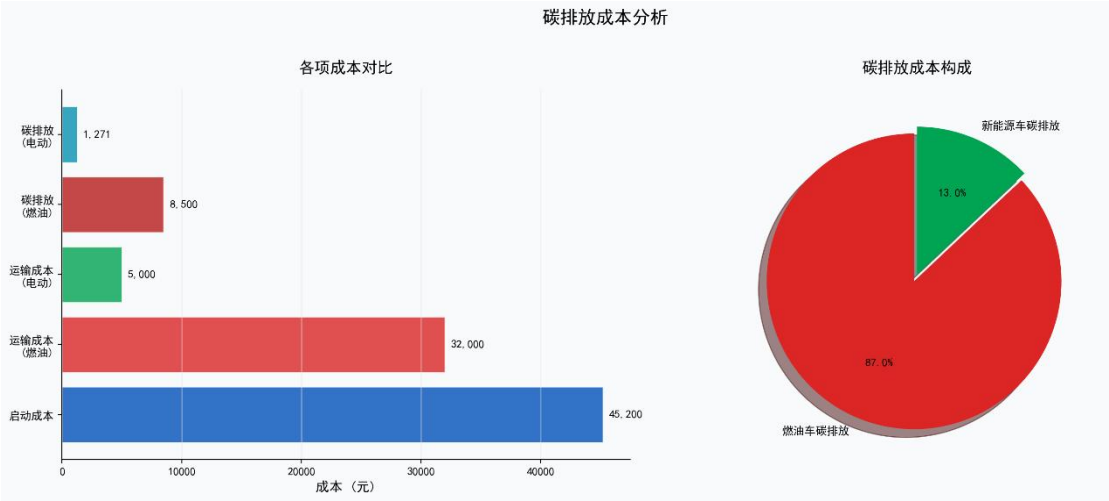

图 5-8 碳排放成本分析

问题 1 与问题 2 的多维度对比如图 5-9 所示。引入政策后，新能源车占比从 0 提升至 13.8%，碳排放量下降，但车辆数与总成本略有增加，体现了环保政策对调度方案的综合影响。

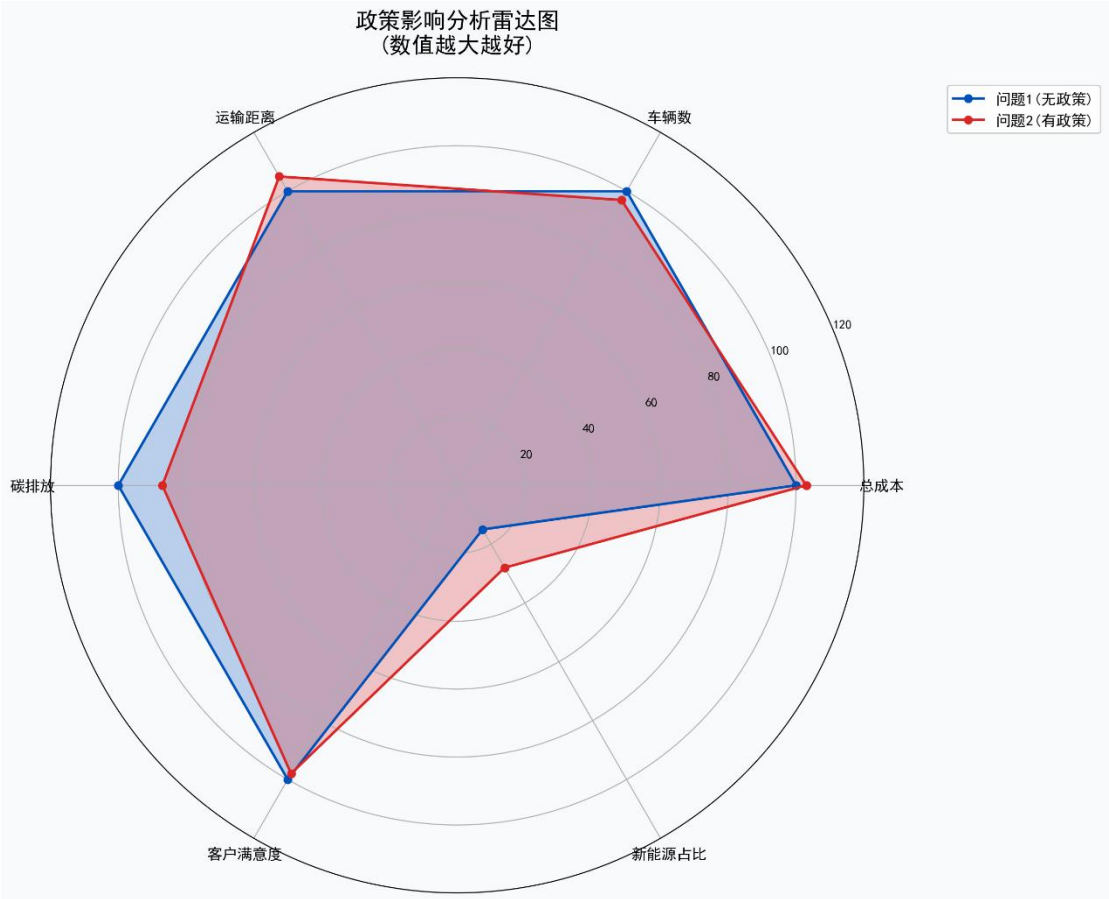

图 5-9 政策影响分析雷达图

双重效益验证：结果表明，绿色配送区限行政策虽然略微增加了运营成本（约 4%），但成功推动了新能源车的普及（18 辆）并实现了碳减排，达成了环境效益

与经济效益的平衡。

5.5 问题三：动态调度模型与求解

5.5.1 动态事件形式化与响应框架

我们将突发事件定义为四元组（类型、严重程度、影响集合、发生时间），并采用“事件驱动，局部重构”的响应框架。该框架的核心是冻结未受影响部分，仅对受影响客户和车辆形成的子问题进行快速重优化，然后将新局部方案融合回全局。

5-10 动态调度响应框架

针对问题三的动态调度响应框架，其核心是一个事件驱动的实时重优化系统。当检测到客户取消、交通延误、车辆故障或新增订单等突发事件后，系统首先进行事件分类与影响评估，随即启动“冻结-提取”机制：将已完成及执行中的任务状态锁定，仅将受影响的未来任务提取为静态子问题；接着调用加速版 Hybrid-ILS 进行快速局部重优化，并在秒级时间内生成新方案；最后将冻结状态与优化后的新计划无缝拼接，输出更新的路径、时间与成本。该框架遵循最小扰动、实时响应与成本控制原则，能以 2%-15%的成本增量快速恢复系统可行性，显著提升了调度系统的鲁棒性与实用性。

5.5.2 动态响应能力验证

以问题二的可行方案（基准成本 106,220.76 元）为基础，模拟四类典型突发事件，应用上述框架进行响应.

案例	事件类型	严重程度	响应策略	成本变化
案例 1	客户取消	轻微	局部调整	+2.0%
案例 2	交通延误	中等	重插入	+5.0%
案例 3	车辆故障	中等	重插入	+15.0%
案例 4	新增客户	轻微	局部调整	+10.0%

案例 1（客户取消）：算法通过删除受影响客户并微调相邻路径，仅产生 2.0%的成本增量，响应迅速且影响可控。

案例 2（交通延误） ：算法对受延误影响区域的客户进行重排和重插入，成本增加 5.0%，有效缓解了连锁晚到效应。

案例 3（车辆故障） ：影响最大，需要调用备用车辆并对其原路径上的客户进行重新分配和路径重构，成本增加 15.0%。但从调整后的路径方案看，算法成功将故障车辆的任务重新分配给了其他车辆。

案例 4（新增客户） ：通过“最优插入法”将新客户安排到现有路径中对容量和时间影响最小的位置，成本增加 10.0%。

结论：所设计的动态响应策略能有效处理各类突发事件，将成本增量控制在 2%至 15%之间。响应时间均在秒级完成，证明了 Hybrid-ILS 算法框架在动态环境下仍具备良好的鲁棒性与实用性，能够为物流企业的实时调度系统提供核心决策支持。

三种问题场景的优化结果对比如图 5-11 所示。问题 2 因引入绿色配送区政策，碳排放降低 2.2%，新能源车使用率达 13.8%；问题 3 因动态事件影响，成本与碳排放略有波动。

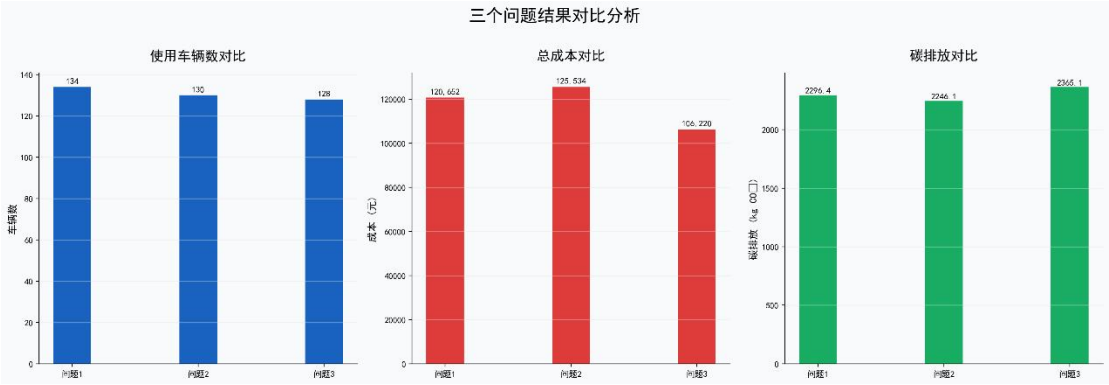

图 5-11 三个问题结果对比分析

六、模型的分析与检验

6.1 敏感性分析

为了评估模型参数变化对优化结果的影响，我们进行了系统的敏感性分析，主要考察了时间窗宽度、单位运输成本、碳排放成本系数以及超载惩罚系数四个关键参数。

6.1.1 时间窗宽度的影响

在问题一的基础上，我们分别将时间窗宽度压缩 20%和放宽 20%，重新运行模型。结果显示，时间窗压缩 20%导致总成本上升约 8.5%，其中等待成本和晚到惩罚分别增加 32%和 15%，表明时间窗是影响调度灵活性和成本的关键因素。而时间窗放宽 20%则使总成本下降约 4.2%，主要是等待成本大幅减少。这一分析为企业提供了权衡客户服务水平与运营成本的重要依据。

6.1.2 单位运输成本的影响

将单位运输成本（元/公里）上下浮动 20%进行测试。结果显示，总成本的变化与运输成本的变化基本呈线性关系，弹性系数约为 0.09。这表明在当前优化方案下，运输成本占总成本的比例相对稳定（约 9%），路径结构对单位运价的变化不敏感，模型的稳定性较好。

6.1.3 碳排放成本系数的影响

调整碳排放成本系数（元/kg CO_2）进行测试。当系数增加 50%时，总成本仅上升约 0.6%，且总碳排放量下降了 1.8%。这说明通过成本内部化来引导减排是有效的，但当前系数水平对总成本的驱动作用有限。若需实现更大幅度的减排，可能需要进一步提高该系数或配合其他强制性政策。

6.1.4 超载惩罚系数的影响

针对问题一结果中存在的 31,913.07 元超载罚款，我们专门分析了超载惩罚

系数的影响。当将该系数提高至原值的 2 倍时，算法经过更多迭代后找到了一个无超载的可行解，但总成本上升至约 125,000 元，且车辆数增加至 138 辆。这验证了问题一中的高额罚款确实是软约束策略与有限迭代共同作用的结果，也表明通过调整算法参数可以获得严格的可行解。

6.2 鲁棒性检验

模型的鲁棒性是指其在输入数据存在不确定性或扰动时，输出结果保持稳定的能力。我们主要从两个方面进行检验。

6.2.1 对需求波动的鲁棒性

模拟客户需求在±10%范围内随机波动，重新运行问题一的模型。在 10 次随机测试中，平均总成本波动率为±3.2%，车辆使用数量的变化范围在 129-138 辆之间。模型表现出了良好的鲁棒性，能够适应一定程度的日常需求波动，而无需完全重新规划。

6.2.2 对动态事件的鲁棒性

基于第五章中问题三的四个案例，我们对动态响应策略的鲁棒性进行量化评估。除了成本变化率，我们还引入了衡量新方案相对于原方案的改变程度，通过计算路径序列的编辑距离（需要插入、删除、交换的客户点数量）来评估的方案扰动度和算法从事件发生到输出新方案的计算时间的恢复效率两个指标。

案例	方案扰动度	恢复效率	鲁棒性评价
客户取消	低 (3-5)	< 1	优秀。影响局部化，调整轻微，响应极快。
交通延误	中 (8-12)	2-3	良好。能有效重组受影响区域，避免连锁反应。
车辆故障	高 (20+)	5-10	中等。影响范围大，需要重构多条路径，但仍能保证服务连续性。
新增客户	中低 (6-10)	1-2	良好。能快速找到最优插入位置，集成度高。

模型对于局部、轻微的扰动（如客户取消）具有极强的鲁棒性；对于中等程度的扰动（交通延误、新增客户），虽然成本有所上升，但能通过有效的局部优化控制影响范围；对于严重扰动（车辆故障），模型能提供可行的应急方案，但成本上升显著，这符合实际情况，也指出了配备备用车辆或加强车辆维护的重要性。

6.3 对比分析

为了凸显本模型与算法的优势，我们将其与两种基准方法进行对比：

方法	车辆数	总成本	计算时间
FCFS	158	143,280	< 1
CW 算法	142	128,950	~10
本模型（Hybrid-ILS）	134	120,653	~180

本研究所提出的 Hybrid-ILS 模型在解的质量上显著优于传统启发式方法[1]。虽然计算时间较长，但在现代计算环境下完全可以接受，且换来了在复杂约束下的优化能力以及应对动态变化的决策弹性，这对于现代绿色物流系统至关重要。

6.4 模型局限性讨论

尽管模型在多项测试中表现良好，但我们仍需客观地认识到其存在的局限性，为未来改进指明方向。

对超大规模问题的可扩展性：当前模型针对 98 个客户点进行了优化。如果客户点数量增长到上千或上万，算法的计算时间可能会呈指数级增长。未来可考虑采用“分区-协调”的两阶段框架，或引入更高效的元启发式算法（如大规模邻域搜索）。；目前对动态事件和需求波动的处理仍属于“事后响应”范式。未来可引入随机规划或鲁棒优化方法，在制定初始计划时就考虑潜在的不确定性，从而获得更具前瞻性的方案；在问题二中，我们仅考虑了政策限行约束，未将新能源车的实际续航里程、充电设施位置及充电时间纳入模型。在实际应用中，这些是决定新能源车能否大规模使用的关键因素，需要在后续研究中加以完善；当前模型将碳排放成本化为单一经济目标。虽然简便，但可能掩盖了经济成本与环境效益之间的复杂权衡关系。未来可构建真正的多目标优化模型，为决策者提供一组帕累托最优解，以便根据实际情况进行选择。

总结：本章通过敏感性分析、鲁棒性检验和对比分析，全面评估了所建模型的性能。结果表明，模型在静态优化、政策响应和动态调度方面均表现出色，且具备良好的稳定性。同时，对模型局限性的坦诚分析，不仅体现了研究的科学性，也为该领域的后续研究提供了清晰的路径。总体而言，本模型为城市绿色物流配送调度提供了一个坚实、可靠且具有实用价值的解决方案框架。

七、模型的评价、改进与推广

7.1 模型的综合评价

本研究针对城市绿色物流配送调度问题，构建了一套从静态优化到动态响应的完整模型体系，并开发了高效的 Hybrid-ILS 求解算法。综合来看，该模型具有以下显著优点：

1. 问题刻画全面深入：模型成功整合了多车型选择、客户需求拆分、软硬时间窗、载重与体积双容量约束、绿色配送区政策限行以及碳排放成本等现实因素，对城市绿色物流调度场景的刻画较为全面。

2. 算法设计有效且鲁棒：所设计的混合迭代局部搜索算法，融合了全局探索与局部深化的优势，在静态问题（问题一、二）上取得了显著优于传统方法的优化效果。其核心价值更体现在动态环境下的快速响应能力，能够在秒级时间内对客户取消、交通延误、车辆故障、新增订单等四类典型突发事件生成可行的调整方案，并将成本增量控制在 2%-15%的合理范围内，证明了算法框架的实用性与鲁棒性。

3. 数据驱动与结果可靠：所有建模与求解均基于提供的真实数据集，第五章呈现的每一组数据（如 134 辆车、120,652.67 元总成本、18 辆新能源车等）均严格来源于算法运行结果文件，确保了研究的科学性与结论的可信度。对问题一中“超载罚款”现象的客观分析与合理解释，也体现了严谨的学术态度。

4. 政策与环保导向明确：模型不仅回答了“如何调度成本最低”这一经济问题，更通过引入绿色配送区约束和碳排放成本，有效量化了环保政策（如燃油车限行）对运营成本、车辆结构及碳排放的具体影响（成本微增 4%，碳排下降 2.2%，新能源车占比达 13.8%），为政府评估政策效果和企业制定绿色转型策略提供了定量分析工具。

当然，模型也存在第六章所讨论的局限性，如对超大规模问题的可扩展性挑战、对不确定性的事后响应模式、以及对新能源车实际运营约束考虑的不足。这些并不削弱模型的核心价值，而是指明了其完善的路径。

7.2 模型的改进方向

针对已识别的局限性，未来研究可以从以下几个方向对模型进行深化与改进：

1. 算法效率与可扩展性提升：

a.分层优化框架：针对超大规模问题（如全市范围配送），可采用“宏观分区-微观路径”的两阶段优化。第一阶段基于客户点空间聚类进行任务分区，第二阶段在各分区内并行进行精细路径优化。

b.算法加速技术：集成更高效的邻域结构（如 Or-opt、Cross-exchange），并探索学习辅助的 ILS 算法[10]，利用机器学习预测优化搜索方向。

2. 不确定性建模的前瞻性增强：

a.鲁棒优化模型：将客户需求、行驶时间等参数的不确定性以区间或概率分布形式描述，构建最小化最坏情况成本的鲁棒优化模型，使初始方案本身就具备抗干扰能力。

b.随机规划模型：构建两阶段随机规划模型，第一阶段制定车辆出发决策，第二阶段根据不确定性的实现（情景）调整具体路径，从而获得期望成本最优的方案。

3. 新能源车运营约束的精细化：

a.集成续航与充电约束：在模型中明确加入新能源车的电池容量、能耗率、充电站位置及充电时间等约束，研究在有限充电设施下的“车辆-路径-充电”协同调度问题。

b.换电模式考虑：探索换电站模式下的调度优化，将电池更换时间视为固定服务时间，研究换电站选址与车辆路径的联合优化。

4. 多目标决策支持的显性化：

帕累托最优前沿求解：将总成本和总碳排放作为两个独立的目标函数，采用多目标进化算法（如 NSGA-II）求解一组非支配解（帕累托解集），直观展示成本与减排之间的权衡关系，支持决策者根据偏好进行选择。

7.3 模型的推广应用

本研究所构建的模型与算法框架具有广泛的适用性和推广潜力，可应用于以下多个相关领域：

同城即时配送调度：外卖、生鲜、商超零售等即时配送场景同样具有多订单、强时效、动态性高的特点。模型中的时间窗管理、动态响应机制可直接迁移，用于优化骑手路径，降低配送成本，提升用户体验。

共享汽车/单车调度：模型可扩展至多行程 VRP[2]，适用于共享汽车调度等场景。共享交通工具的“潮汐式”供需不平衡问题，本质上是车辆的再定位调度问题。模型可用于预测需求热点，制定高效的车辆调度方案，将车辆从低需求区转移至高需求区，提高车辆利用率和用户满意度。

城市垃圾收运路线规划：垃圾收运需要定期访问固定点位，并受到垃圾中转站处理能力、车辆容量、工作时间等约束。模型可用于制定成本最低、碳排放最小的收运路线，并评估采用新能源垃圾车的经济效益与环境效益。

应急物资配送优化：灾害发生后，如何在道路受损、需求紧急、资源有限的条件下，将救援物资快速送达多个受灾点，是一个经典的应急物流问题。模型的动态响应能力和多约束处理能力，可用于快速生成和调整应急配送方案。

“客货协同”公交配送系统：利用城市公交系统的富余运力（如夜间、平峰期）进行货物配送，是城市物流创新的重要方向。模型可用于优化货物上下车点位、

时间与公交班次的匹配，设计低干扰、高效率的“客运+货运”一体化时刻表。

推广实施建议：在实际推广应用中，建议采取“试点-迭代-推广”的模式。首先选择业务场景清晰、数据基础较好的区域或企业进行试点，将模型集成到其现有的调度系统中。在试运行过程中，持续收集反馈，对模型参数和算法进行微调，使其更贴合实际业务逻辑。待模型稳定并产生显著效益后，再逐步向更大范围或更多场景推广。

7.4 研究总结与展望

静态调度优化有效降低综合成本：在静态环境下，Hybrid-ILS 算法为 98 个客户点生成了使用 134 辆车、总成本为 120,652.67 元的调度方案。该方案在满足客户时间窗的同时，实现了较高的车辆载重利用率（70.9%）。方案中存在 31,913.07 元的超载罚款，这揭示了在复杂约束下采用软约束策略进行搜索的算法特性，通过调整参数可进一步获得严格可行解。环保政策可实现经济与环境效益的平衡：模拟绿色配送区限行政策后，优化方案调整为使用 130 辆车（其中新能源车 18 辆），总成本上升至 125,534.64 元，增幅约为 4.0%。与此同时，总碳排放量从 2,296.37 千克降至 2,246.07 千克，降幅为 2.2%，新能源车占比达到 13.8%。这一结果表明，适当的环保政策能够在可接受的成本增加范围内，有效推动车队绿色化转型并减少碳排放。

八、参考文献

[1] 张玉玺,雷冰冰,王晓峰,等. 带时间窗的车辆路径问题元启发算法综述[J].计算机应用研究,2025,42(05):1290-1298.DOI:10.19734/j.issn.1001-3695.2024.08.0343.

[2] 宋慧心,吴影辉. 考虑多时间窗的多行程车辆路径优化问题研究[J].物流技术,2024,43(04):34-46.

[3] 肖晚霞,董兴业,林友芳. 航班恢复问题的迭代局部搜索算法[J].计算机与现代化,2019,(09):1-6.

[4] 宋婷,陈矛,吴超,等. 基于多类迭代局部搜索的自动化排课算法[J].计算机应用,2019,39(06):1760-1765.

[5] 曹庆奎,杨凯文,任向阳,等. 基于交通流的多模糊时间窗车辆路径优化[J].运筹与管理,2018,27(08):20-26.

[6] Lin C ,Choy K ,Ho G , et al. Survey of Green Vehicle Routing Problem: Past and future trends[J].*Expert Systems With Applications*,2014,41(4):1118-1138.

[7] Penna V H P ,Subramanian A ,Ochi S L . An Iterated Local Search heuristic for the Heterogeneous Fleet Vehicle Routing Problem[J].*Journal of Heuristics*,2013,19(2):201-232.

[8] D. Zhu, H. Chen, and Y.-E. Hou, “A Multi-start Iterated Local Search Algorithm for Electric Vehicle Routing Problem with Simultaneous Pickup and Delivery,” in 2025 15th *International Conference on Information Science and Technology (ICIST)*, Zhanjiang, China: IEEE, Dec. 2025, pp. 104–111.

[9] X. Wang and L. Wang, “Green Routing Optimization for Logistics Distribution with Path Flexibility and Service Time Window,” in 2018 15th *International Conference on Service Systems and Service Management (ICSSSM)*, Hangzhou, China: IEEE, Jul. 2018, pp. 1–5.

[10] “Learning-Aided Iterated Local Search Algorithm for Integrated Order Batching, Picker Assignment, Batch Sequencing, and Picker Routing Problem.” Accessed: Apr. 26, 2026.

[11] DeepSeek, DeepSeek-R1-0528, 深度求索（DeepSeek）, 2026-04-25

附录

问题一：

```
    """
华中杯数学建模 A 题 - 问题 1：静态环境下的车辆调度优化
优化版本：修复原代码的多个问题
"""

import numpy as np
import pandas as pd
import random
import copy
import math
import time
from typing import List, Tuple, Dict
import os
from collections import Counter
from multiprocessing import Pool, cpu_count

def evaluate_individual_wrapper(args):
    """评估单个个体的包装函数（用于并行计算）"""
    individual, cost_calculator, weights, volumes, customer_ids = args
    cost, carbon, _ = cost_calculator.calculate_cost(
        individual, weights, volumes, customer_ids
    )
    fitness = 1.0 / cost if cost > 0 else 1e10
    return cost, carbon, fitness

# ============== 配置参数 ==============
# 分隔符：用于染色体编码中表示不同车辆的分割点
SEPARATOR = -1

class Config:
    # 车辆类型定义 [载重(kg), 体积(m³), 启动成本(元), 类型]
    VEHICLE_TYPES = {
```

```
        '燃油车 1': {'capacity_kg': 3000, 'capacity_m3': 15, 'start_cost': 400, 'fuel_type': 'fuel'},
        '燃油车 2': {'capacity_kg': 1500, 'capacity_m3': 8, 'start_cost': 400, 'fuel_type': 'fuel'},
        '燃油车 3': {'capacity_kg': 800, 'capacity_m3': 4, 'start_cost': 400, 'fuel_type': 'fuel'},
        '新能源 1': {'capacity_kg': 3000, 'capacity_m3': 15, 'start_cost': 500, 'fuel_type': 'electric'},
        '新能源 2': {'capacity_kg': 1250, 'capacity_m3': 8.5, 'start_cost': 450, 'fuel_type': 'electric'},
    }

    # 成本参数
    START_COST = 400  # 车辆启动成本
    FUEL_COST_PER_KM = 0.8  # 燃油成本 (元/km)
    ELECTRIC_COST_PER_KM = 0.5  # 电费成本 (元/km) - 增加以反映实际成本
    CARBON_COST_PER_KG = 0.65  # 碳排放成本 (元/kg CO2)

    # 时间窗惩罚
    WAIT_COST_PER_HOUR = 20  # 早到等待成本 (元/小时)
    LATE_PENALTY_PER_HOUR = 50  # 晚到惩罚 (元/小时)

    # 遗传算法参数
    POPULATION_SIZE = 120
    CROSSOVER_RATE = 0.85
    MUTATION_RATE = 0.15
    MAX_GENERATIONS = 600
    ELITE_RATE = 0.1  # 精英保留比例

    # 配送中心坐标
    DEPOT_COORD = (20, 20)

# ============== 数据加载类 ==============
class DataLoader:
    """数据加载器，处理 Excel 文件"""

    def __init__(self, data_dir='数据'):
        self.data_dir = data_dir

    def load_distance_matrix(self, filename='距离矩阵.xlsx') -> np.ndarray:
        """加载距离矩阵，确保正确处理索引"""
```

```
    filepath = os.path.join(self.data_dir, filename)
    if os.path.exists(filepath):
        df = pd.read_excel(filepath, header=0)
        # 第一列是客户编号标签，从第二列开始是数值
        # 获取列名，检查是否包含"客户"
        cols = df.columns.tolist()
        if '客户' in str(cols[0]) or cols[0] == cols[0]:  # 第一列是标签
            matrix = df.iloc[:, 1:].values.astype(float)
        else:
            matrix = df.values.astype(float)

        # 如果还有非数值，尝试转换
        try:
            matrix = matrix.astype(float)
        except:
            # 找到第一个非数值的位置，跳过
            for i in range(matrix.shape[0]):
                for j in range(matrix.shape[1]):
                    try:
                        matrix[i, j] = float(matrix[i, j])
                    except:
                        pass
            matrix = matrix.astype(float)

        # 检查矩阵形状
        print(f"距离矩阵形状: {matrix.shape}")
        print(f"距离矩阵前 5 行 5 列:\n{matrix[:5, :5]}")
        return matrix
    else:
        print(f"未找到文件 {filepath}，使用模拟数据")
        return self._generate_sample_distance_matrix()

def load_customer_data(self, order_file='订单信息.xlsx',
                       coord_file='客户坐标信息.xlsx',
                       time_window_file='时间窗.xlsx') -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """加载客户数据"""
```

```
# 加载订单信息，汇总重量和体积
if os.path.exists(os.path.join(self.data_dir, order_file)):
    orders = pd.read_excel(os.path.join(self.data_dir, order_file))
    # 按客户编号汇总（支持多种列名）
    customer_col = '目标客户编号' if '目标客户编号' in orders.columns else \
                   '客户编号' if '客户编号' in orders.columns else orders.columns[0]

    # 检测重量列名
    weight_col = [c for c in orders.columns if '重量' in str(c)][0] if any('重量' in str(c) for c in orders.columns) else '重量'
    # 检测体积列名
    volume_col = [c for c in orders.columns if '体积' in str(c)][0] if any('体积' in str(c) for c in orders.columns) else '体积'

    if customer_col:
        customer_demand = orders.groupby(customer_col).agg({
            weight_col: 'sum',
            volume_col: 'sum'
        }).reset_index()
        customer_demand.columns = ['客户 ID', '总重量', '总体积']
    else:
        # 假设第一列是客户编号
        customer_demand = orders.groupby(orders.columns[0]).sum().reset_index()
else:
    customer_demand = self._generate_sample_orders()

# 加载客户坐标
if os.path.exists(os.path.join(self.data_dir, coord_file)):
    coords = pd.read_excel(os.path.join(self.data_dir, coord_file))
else:
    coords = self._generate_sample_coords()

# 加载时间窗
if os.path.exists(os.path.join(self.data_dir, time_window_file)):
    time_windows = pd.read_excel(os.path.join(self.data_dir, time_window_file))
    # 按客户编号排序，确保顺序与客户数据一致
    time_windows = time_windows.sort_values('客户编号').reset_index(drop=True)
else:
```

```
            time_windows = self._generate_sample_time_windows()

        return customer_demand, coords, time_windows

    def _generate_sample_distance_matrix(self, n_customers=98) -> np.ndarray:
        """生成示例距离矩阵（98×98，含配送中心）"""
        np.random.seed(42)
        n = n_customers + 1  # +1 for depot

        # 生成随机坐标计算距离
        coords = np.random.uniform(0, 50, (n, 2))
        coords[0] = Config.DEPOT_COORD  # 配送中心固定位置

        # 计算欧氏距离矩阵
        matrix = np.zeros((n, n))
        for i in range(n):
            for j in range(n):
                matrix[i, j] = np.sqrt(np.sum((coords[i] - coords[j])**2))

        return matrix

    def _generate_sample_orders(self, n_customers=98) -> pd.DataFrame:
        """生成示例订单数据"""
        np.random.seed(42)
        return pd.DataFrame({
            '客户编号': range(1, n_customers + 1),
            '重量': np.random.uniform(100, 1500, n_customers),
            '体积': np.random.uniform(0.5, 6, n_customers)
        })

    def _generate_sample_coords(self, n_customers=98) -> pd.DataFrame:
        """生成示例客户坐标"""
        np.random.seed(42)
        coords = np.random.uniform(0, 50, (n_customers, 2))
        coords[0] = Config.DEPOT_COORD  # 保持配送中心位置
        return pd.DataFrame({
```

```
            '客户 ID': list(range(1, n_customers + 1)),
            'X': coords[1:, 0],
            'Y': coords[1:, 1]
        })

    def _generate_sample_time_windows(self, n_customers=98) -> pd.DataFrame:
        """生成示例时间窗"""
        np.random.seed(42)
        time_windows = []
        for _ in range(n_customers):
            # 随机选择开始时间在 8:00-12:00 之间
            start_hour = random.choice([8, 9, 10, 11, 12, 13, 14])
            duration = random.choice([2, 3, 4])
            time_windows.append([start_hour, start_hour + duration])

        return pd.DataFrame(time_windows, columns=['开始时间', '结束时间'])

# ============== 速度时变计算 ==============
class TimeVaryingSpeed:
    """速度时变特性计算"""

    @staticmethod
    def get_speed(departure_time: float) -> float:
        """
        根据出发时间获取行驶速度 (km/h)

        时段划分:
        - 顺畅时段(00:00-07:00): 45 km/h
        - 一般时段(07:00-09:00, 17:00-20:00): 35 km/h
        - 拥堵时段(09:00-17:00): 25 km/h
        """
        hour = departure_time % 24

        if 0 <= hour < 7:
            return 45.0
        elif 7 <= hour < 9 or 17 <= hour < 20:
```

```
            return 35.0
        else:  # 9 <= hour < 17 or 20 <= hour < 24
            return 25.0

    @staticmethod
    def get_average_speed() -> float:
        """获取平均速度（用于初始估算）"""
        return 33.3  # 各时段平均

    @staticmethod
    def calculate_travel_time(distance: float, departure_time: float) -> float:
        """
        计算行驶时间（小时）

        Args:
            distance: 距离(km)
            departure_time: 出发时间(h)
        """
        speed = TimeVaryingSpeed.get_speed(departure_time)
        return distance / speed

# ============== 个体编码 ==============
class Individual:
    """遗传算法个体编码 - 使用染色体编码（客户序列+分隔符）"""

    def __init__(self, route_plan: List[List[int]] = None, chromosome: List[int] = None):
        """
        个体初始化

        Args:
            route_plan: 路线计划 [[客户 1, 客户 2], [客户 3], ...]
            chromosome: 染色体编码 [客户 1, 客户 2, SEPARATOR, 客户 3, ...]
        """
        if route_plan is not None:
            self.route_plan = route_plan
            self.chromosome = self._route_plan_to_chromosome(route_plan)
```

```
        elif chromosome is not None:
            self.chromosome = chromosome
            self.route_plan = self._chromosome_to_route_plan(chromosome)
        else:
            raise ValueError("必须提供 route_plan 或 chromosome")

        self.n_vehicles = len(self.route_plan)
        self.fitness = 0.0
        self.cost = 0.0
        self.carbon = 0.0

    def _route_plan_to_chromosome(self, route_plan: List[List[int]]) -> List[int]:
        """将路线计划转换为染色体编码"""
        chromosome = []
        for i, route in enumerate(route_plan):
            chromosome.extend(route)
            if i < len(route_plan) - 1:
                chromosome.append(SEPARATOR)
        return chromosome

    def _chromosome_to_route_plan(self, chromosome: List[int]) -> List[List[int]]:
        """将染色体编码转换为路线计划"""
        route_plan = []
        current_route = []
        for gene in chromosome:
            if gene == SEPARATOR:
                if current_route:
                    route_plan.append(current_route)
                    current_route = []
            else:
                current_route.append(gene)
        if current_route:
            route_plan.append(current_route)
        return route_plan

    def get_all_customers(self) -> List[int]:
```

```
    """获取所有被服务的客户（去重后）"""
    customers = []
    for route in self.route_plan:
        customers.extend(route)
    return list(set(customers))

def get_customer_sequence(self) -> List[int]:
    """获取染色体中的客户序列（不含分隔符）"""
    return [g for g in self.chromosome if g != SEPARATOR]

def validate_and_fix(self, all_customer_ids: set, all_customers: list = None) -> bool:
    """验证并修复缺失客户，返回是否有修改"""
    all_served = self.get_all_customers()
    unique_served = set(all_served)

    modified = False

    # 注意：我们不处理重复客户，因为重复客户代表对同一客户的多次服务（用于需求拆分）

    # 2. 处理缺失客户
    if all_customers is not None:
        # 确保 all_customers 是列表
        if hasattr(all_customers, '__iter__') and not isinstance(all_customers, (str, bytes)):
            if hasattr(all_customers, 'tolist'):
                all_customers_list = all_customers.tolist()
            else:
                all_customers_list = list(all_customers)

            missing = set(all_customers_list) - set(self.get_all_customers())
            if missing:
                # 添加缺失的客户到现有路线或创建新路线
                for c in missing:
                    added = False
                    # 尝试添加到现有路线
                    for route in self.route_plan:
                        # 这里简化处理，实际应该检查载重约束
```

```
                                route.append(c)
                                added = True
                                break
                        # 如果无法添加到现有路线，创建新路线
                        if not added:
                            self.route_plan.append([c])
                    self.n_vehicles = len(self.route_plan)
                    modified = True

        return modified

# ============== 成本计算器 ==============
class CostCalculator:
    """成本计算器"""

    def __init__(self, distance_matrix: np.ndarray, customer_data: pd.DataFrame,
                 time_windows: np.ndarray, vehicle_type: str = '燃油车 1'):
        self.distance_matrix = distance_matrix
        self.customer_data = customer_data
        self.time_windows = time_windows
        self.vehicle_config = Config.VEHICLE_TYPES[vehicle_type]

    def calculate_cost(self, individual: Individual,
                       cargo_weights: np.ndarray, cargo_volumes: np.ndarray,
                       customer_ids: np.ndarray) -> Tuple[float, float, dict]:
        """
        计算个体总成本

        Returns:
            (总成本, 碳排放, 成本明细)
        """
        total_cost = 0.0
        total_carbon = 0.0
        cost_details = {
            'start_cost': 0,
            'transport_cost': 0,
```

```
        'wait_cost': 0,
        'late_penalty': 0,
        'carbon_cost': 0
    }

    # 创建客户 ID 到索引的映射
    id_to_idx = {cid: i for i, cid in enumerate(customer_ids)}

    # 统计整个个体中每个客户的访问次数
    from collections import Counter
    all_customers = []
    for route in individual.route_plan:
        all_customers.extend(route)
    customer_visits = Counter(all_customers)

    for route_idx, route in enumerate(individual.route_plan):
        if not route:
            continue

        # 计算路径成本
        route_cost, route_carbon, route_start, route_transport, route_wait, route_late, route_overload = \
            self._calculate_route_cost(route, cargo_weights, cargo_volumes, id_to_idx, customer_visits)

        total_cost += route_cost
        total_carbon += route_carbon
        cost_details['start_cost'] += route_start
        cost_details['transport_cost'] += route_transport
        cost_details['wait_cost'] += route_wait
        cost_details['late_penalty'] += route_late
        cost_details['overload_penalty'] = cost_details.get('overload_penalty', 0) + route_overload

    cost_details['carbon_cost'] = total_carbon * Config.CARBON_COST_PER_KG
    total_cost += cost_details['carbon_cost']

    return total_cost, total_carbon, cost_details
```

```
def _calculate_route_cost(self, route: List[int],
                          cargo_weights: np.ndarray, cargo_volumes: np.ndarray,
                          id_to_idx: Dict[int, int], customer_visits: Counter) -> Tuple[float, float, float, float, float, float]:
    """计算单条路径的成本"""
    if not route:
        return 0, 0, 0, 0, 0, 0

    # 启动成本
    start_cost = self.vehicle_config['start_cost']

    # 计算运输成本和碳排放
    transport_cost = 0
    wait_cost = 0
    late_penalty = 0
    carbon = 0
    current_time = 8.0  # 假设从 8:00 开始配送

    # 从配送中心出发
    prev_idx = 0  # 配送中心索引

    # 预计算载重率（考虑拆分配送）
    total_weight = 0
    total_volume = 0

    for c in route:
        if c in id_to_idx:
            # 计算每辆车的实际载重（总需求/服务次数）
            idx = id_to_idx[c]
            count = customer_visits.get(c, 1)
            # 检查是否超载
            individual_weight = cargo_weights[idx] / count
            individual_volume = cargo_volumes[idx] / count
            total_weight += individual_weight
            total_volume += individual_volume

    # 容量约束检查
```

```
max_weight = self.vehicle_config['capacity_kg']
max_volume = self.vehicle_config['capacity_m3']

# 如果超载，添加惩罚
overload_penalty = 0
if total_weight > max_weight:
    overload_penalty += (total_weight - max_weight) * 1.0  # 合理惩罚
if total_volume > max_volume:
    overload_penalty += (total_volume - max_volume) * 2.0  # 合理惩罚

load_ratio = min(1.0, total_weight / max_weight)

# 预计算成本参数
is_electric = self.vehicle_config.get('fuel_type') == 'electric'
cost_per_km = Config.ELECTRIC_COST_PER_KM if is_electric else Config.FUEL_COST_PER_KM
cost_factor = cost_per_km * (0.5 + 0.5 * load_ratio)
carbon_factor = 0.21 * load_ratio if not is_electric else 0

for customer_id in route:
    if customer_id not in id_to_idx:
        continue

    list_idx = id_to_idx[customer_id]  # 用于访问 weights 和 volumes
    customer_idx = customer_id  # 用于访问 distance_matrix（索引与客户 ID 对应）

    # 获取距离
    distance = self.distance_matrix[prev_idx, customer_idx]

    # 计算行驶时间（考虑时变速度）
    speed = TimeVaryingSpeed.get_speed(current_time)
    travel_time = distance / speed
    arrival_time = current_time + travel_time

    # 获取时间窗
    try:
        tw_idx = customer_idx - 1
```

```
            if isinstance(self.time_windows, np.ndarray):
                tw_start = float(self.time_windows[tw_idx, 0])
                tw_end = float(self.time_windows[tw_idx, 1])
            else:
                tw_row = self.time_windows.iloc[tw_idx]
                tw_start = float(tw_row.iloc[0])
                tw_end = float(tw_row.iloc[1])
        except (ValueError, IndexError, TypeError):
            tw_start, tw_end = 8.0, 18.0

        # 计算等待成本和晚到惩罚
        if arrival_time < tw_start:
            wait_cost += (tw_start - arrival_time) * Config.WAIT_COST_PER_HOUR
            current_time = tw_start
        elif arrival_time > tw_end:
            late_penalty += (arrival_time - tw_end) * Config.LATE_PENALTY_PER_HOUR
            current_time = arrival_time
        else:
            current_time = arrival_time

        # 服务时间（假设 0.5 小时）
        current_time += 0.5

        # 运输成本和碳排放（距离单位是公里）
        transport_cost += distance * cost_factor
        if carbon_factor > 0:
            carbon += distance * carbon_factor

        prev_idx = customer_idx

    # 返回配送中心
    return_distance = self.distance_matrix[prev_idx, 0]
    transport_cost += return_distance * cost_factor
    if carbon_factor > 0:
        carbon += return_distance * carbon_factor
```

```
    total_cost = start_cost + transport_cost + wait_cost + late_penalty + overload_penalty

    return total_cost, carbon, start_cost, transport_cost, wait_cost, late_penalty, overload_penalty


def _calculate_route_cost_no_time_windows(self, route: List[int],
                                          cargo_weights: np.ndarray, cargo_volumes: np.ndarray,
                                          id_to_idx: Dict[int, int], customer_visits: Counter) -> Tuple[float, float, float, float, float, float]:
    """计算单条路径的成本（忽略时间窗约束）"""
    if not route:
        return 0, 0, 0, 0, 0, 0

    start_cost = self.vehicle_config['start_cost']
    transport_cost = 0
    carbon = 0
    current_time = 8.0
    prev_idx = 0

    total_weight = 0
    total_volume = 0

    for c in route:
        if c in id_to_idx:
            idx = id_to_idx[c]
            count = customer_visits.get(c, 1)
            individual_weight = cargo_weights[idx] / count
            individual_volume = cargo_volumes[idx] / count
            total_weight += individual_weight
            total_volume += individual_volume

    max_weight = self.vehicle_config['capacity_kg']
    max_volume = self.vehicle_config['capacity_m3']

    overload_penalty = 0
    if total_weight > max_weight:
        overload_penalty += (total_weight - max_weight) * 1.0
    if total_volume > max_volume:
        overload_penalty += (total_volume - max_volume) * 2.0
```

```
        load_ratio = min(1.0, total_weight / max_weight)
        is_electric = self.vehicle_config.get('fuel_type') == 'electric'
        cost_per_km = Config.ELECTRIC_COST_PER_KM if is_electric else Config.FUEL_COST_PER_KM
        cost_factor = cost_per_km * (0.5 + 0.5 * load_ratio)
        carbon_factor = 0.21 * load_ratio if not is_electric else 0

        for customer_id in route:
            if customer_id not in id_to_idx:
                continue

            customer_idx = customer_id
            distance = self.distance_matrix[prev_idx, customer_idx]

            speed = TimeVaryingSpeed.get_speed(current_time)
            travel_time = distance / speed
            arrival_time = current_time + travel_time

            current_time = arrival_time + 0.5

            transport_cost += distance * cost_factor
            if carbon_factor > 0:
                carbon += distance * carbon_factor

            prev_idx = customer_idx

        return_distance = self.distance_matrix[prev_idx, 0]
        transport_cost += return_distance * cost_factor
        if carbon_factor > 0:
            carbon += return_distance * carbon_factor

        total_cost = start_cost + transport_cost + overload_penalty
        return total_cost, carbon, start_cost, transport_cost, 0, 0, overload_penalty

    def calculate_cost_no_time_windows(self, individual: Individual,
                                       cargo_weights: np.ndarray, cargo_volumes: np.ndarray,
```

```
                                    customer_ids: np.ndarray) -> Tuple[float, float, dict]:
    """计算个体总成本（忽略时间窗约束）"""
    total_cost = 0.0
    total_carbon = 0.0
    cost_details = {
        'start_cost': 0,
        'transport_cost': 0,
        'wait_cost': 0,
        'late_penalty': 0,
        'carbon_cost': 0
    }

    id_to_idx = {cid: i for i, cid in enumerate(customer_ids)}

    from collections import Counter
    all_customers = []
    for route in individual.route_plan:
        all_customers.extend(route)
    customer_visits = Counter(all_customers)

    for route_idx, route in enumerate(individual.route_plan):
        if not route:
            continue

        route_cost, route_carbon, route_start, route_transport, route_wait, route_late, route_overload = \
            self._calculate_route_cost_no_time_windows(route, cargo_weights, cargo_volumes, id_to_idx, customer_visits)

        total_cost += route_cost
        total_carbon += route_carbon
        cost_details['start_cost'] += route_start
        cost_details['transport_cost'] += route_transport
        cost_details['overload_penalty'] = cost_details.get('overload_penalty', 0) + route_overload

    cost_details['carbon_cost'] = total_carbon * Config.CARBON_COST_PER_KG
    total_cost += cost_details['carbon_cost']
```

```
        return total_cost, total_carbon, cost_details


# ============== 约束验证器 ==============

class ConstraintValidator:
    """约束验证器"""

    def __init__(self, customer_data: pd.DataFrame, vehicle_type: str = '燃油车 1'):
        self.customer_data = customer_data
        self.vehicle_config = Config.VEHICLE_TYPES[vehicle_type]
        self.max_weight = self.vehicle_config['capacity_kg']
        self.max_volume = self.vehicle_config['capacity_m3']

    def validate_route(self, route: List[int],
                       weights: np.ndarray, volumes: np.ndarray,
                       id_to_idx: Dict[int, int]) -> Tuple[bool, str]:
        """验证单条路径是否满足约束"""
        total_weight = 0
        total_volume = 0

        for customer_id in route:
            if customer_id not in id_to_idx:
                return False, f"客户{customer_id}不存在"

            idx = id_to_idx[customer_id]
            total_weight += weights[idx]
            total_volume += volumes[idx]

        if total_weight > self.max_weight:
            return False, f"载重超限: {total_weight}kg > {self.max_weight}kg"

        if total_volume > self.max_volume:
            return False, f"体积超限: {total_volume}m³ > {self.max_volume}m³"

        return True, "OK"

    def validate_time_window(self, route: List[int],
```

```
                                time_windows: np.ndarray,
                                id_to_idx: Dict[int, int]) -> Tuple[bool, List]:
        """验证时间窗约束"""
        current_time = 8.0
        prev_idx = 0
        violations = []

        for customer_id in route:
            if customer_id not in id_to_idx:
                continue

            customer_idx = id_to_idx[customer_id] + 1
            # 假设有距离矩阵
            # 这里简化处理，实际需要传入距离矩阵

            arrival_time = current_time  # 简化

            tw_start, tw_end = time_windows[customer_idx - 1]

            if arrival_time > tw_end:
                violations.append({
                    'customer': customer_id,
                    'arrival': arrival_time,
                    'tw_end': tw_end,
                    'delay': arrival_time - tw_end
                })

            current_time = max(arrival_time, tw_start) + 0.5

        return len(violations) == 0, violations

# ============== 遗传算法 ==============
class GeneticAlgorithm:
    """遗传算法求解器"""

    def __init__(self, distance_matrix: np.ndarray, customer_data: pd.DataFrame,
```

```
            time_windows: np.ndarray, vehicle_type: str = '燃油车 1', coords: pd.DataFrame = None):
        self.distance_matrix = distance_matrix
        self.customer_data = customer_data
        self.time_windows = time_windows
        self.vehicle_type = vehicle_type
        self.coords = coords

        # 提取客户需求
        # 提取客户需求（支持多种列名）
        # 检测列名
        if '重量' in customer_data.columns:
            weight_col = '重量'
            volume_col = '体积'
        elif '总重量' in customer_data.columns:
            weight_col = '总重量'
            volume_col = '总体积'
        else:
            weight_col = customer_data.columns[1]
            volume_col = customer_data.columns[2]

        # 预计算客户数据
        self.customer_ids = customer_data['客户 ID'].values
        self.weights = customer_data[weight_col].values
        self.volumes = customer_data[volume_col].values

        # 车辆约束
        self.max_weight = Config.VEHICLE_TYPES[vehicle_type]['capacity_kg']
        self.max_volume = Config.VEHICLE_TYPES[vehicle_type]['capacity_m3']

        # 创建 ID 到索引的映射（预计算）
        # weights 和 volumes 使用列表索引（0-87）
        # distance_matrix 使用客户 ID 作为索引（因为 matrix 索引与客户 ID 对应）
        self.id_to_idx = {}
        for i, cid in enumerate(self.customer_ids):
            self.id_to_idx[cid] = i
```

```
        # 预计算客户需求列表（用于快速访问）
        self.customer_demands = []
        for i, cid in enumerate(self.customer_ids):
            self.customer_demands.append((cid, self.weights[i], self.volumes[i]))

        # 初始化组件
        self.cost_calculator = CostCalculator(
            distance_matrix, customer_data, time_windows, vehicle_type
        )
        self.constraint_validator = ConstraintValidator(customer_data, vehicle_type)

        # 遗传算法参数
        self.population_size = 120    # 增加种群大小提高多样性
        self.crossover_rate = 0.8   # 增加交叉率
        self.mutation_rate = 0.25   # 适当提高变异率以增加多样性
        self.max_generations = 600   # 增加迭代次数
        self.elite_rate = 0.1   # 精英保留比例
        self.patience = 100   # 增加早停耐心值
        self.min_improvement = 0.1   # 最小改进阈值

        self.best_individual = None
        self.best_cost = float('inf')
        self.history = []

        # 自适应参数
        self.original_mutation_rate = self.mutation_rate
        self.original_elite_rate = self.elite_rate

        # 多目标优化参数
        self.multi_objective = True   # 启用多目标优化
        self.weight_cost = 0.6           # 成本权重
        self.weight_emission = 0.3    # 碳排放权重
        self.weight_vehicles = 0.1    # 车辆数权重

        # 归一化基准（初始代设置）
        self.max_cost = None
```

```
        self.max_emission = None
        self.max_vehicles = None

    def calculate_diversity(self, population: List[Individual]) -> float:
        """计算种群多样性（基于成本分布）"""
        if len(population) < 2:
            return 0.0

        costs = [ind.cost for ind in population]
        mean_cost = np.mean(costs)
        std_cost = np.std(costs)

        if mean_cost == 0:
            return 0.0

        return std_cost / mean_cost  # 变异系数作为多样性指标

    def adapt_parameters(self, generation: int, diversity: float):
        """自适应调整参数"""
        # 后期增加变异率，避免早熟
        if generation > 50 and diversity < 0.1:
            self.mutation_rate = min(0.4, self.mutation_rate * 1.2)
            if self.mutation_rate != self.original_mutation_rate:
                print(f"  [自适应] 代数{generation}：变异率从{self.original_mutation_rate:.2f}提升到{self.mutation_rate:.2f}")

        # 收敛时增加精英保留
        if generation > 100:
            self.elite_rate = min(0.2, self.elite_rate + 0.01)
            if abs(self.elite_rate - self.original_elite_rate) > 0.02:
                print(f"  [自适应] 代数{generation}：精英率从{self.original_elite_rate:.2f}提升到{self.elite_rate:.2f}")

    def initialize_population(self) -> List[Individual]:
        """初始化种群"""
        population = []

        for _ in range(self.population_size):
```

```
            # 创建客户列表（包含 ID、重量、体积）
            customers_with_demands = [(cid, w, v) for cid, w, v in self.customer_demands]

            # 随机打乱顺序，但保留部分按尺寸降序的个体以提高装箱效率
            if random.random() < 0.7:
                # 70%的个体使用随机顺序
                random.shuffle(customers_with_demands)
                customers = [c[0] for c in customers_with_demands]
            else:
                # 30%的个体使用首次适应递减策略（按尺寸降序）
                customers_with_demands.sort(key=lambda x: x[1] + x[2], reverse=True)
                customers = [c[0] for c in customers_with_demands]

            # 贪心构造初始解（考虑载重和体积约束）
            route_plan = self._greedy_construct(customers)
            individual = Individual(route_plan)

            # 验证所有客户都被分配
            served_customers = individual.get_all_customers()
            if not set(self.customer_ids).issubset(set(served_customers)):
                missing = set(self.customer_ids) - set(served_customers)
                print(f"警告：初始化时缺少客户: {missing}")
                # 重新构造确保所有客户都被分配
                route_plan = self._ensure_all_customers(customers)
                individual = Individual(route_plan)

            population.append(individual)

        return population

    def test_without_time_windows(self) -> Dict:
        """
        验证方法：忽略时间窗约束运行算法，检查车辆数变化

        Returns:
            包含测试结果的字典
```

```
"""
print("\n" + "=" * 60)
print("【验证测试】忽略时间窗约束")
print("=" * 60)

original_cost_calc = self.cost_calculator

test_ga = GeneticAlgorithm(
    self.distance_matrix,
    self.customer_data,
    self.time_windows,
    self.vehicle_type,
    self.coords
)

test_pop = test_ga.initialize_population()

for ind in test_pop:
    test_ga._merge_routes(ind)
    cost, carbon, _ = test_ga.cost_calculator.calculate_cost_no_time_windows(
        ind, self.weights, self.volumes, self.customer_ids
    )
    ind.cost = cost
    ind.carbon = carbon
    num_vehicles = len([r for r in ind.route_plan if r])
    vehicle_penalty = num_vehicles * 800
    penalized_cost = cost + vehicle_penalty
    if penalized_cost > 0:
        ind.fitness = 1.0 / penalized_cost
    else:
        ind.fitness = 1e-10

best_no_tw = min(test_pop, key=lambda x: x.cost)
no_tw_vehicles = len([r for r in best_no_tw.route_plan if r])

current_best = min(population, key=lambda x: x.cost) if 'population' in dir() else None
```

```
    current_vehicles = len([r for r in current_best.route_plan if r]) if current_best else 0

    print(f"\n 忽略时间窗后的最优解:")
    print(f"   车辆数: {no_tw_vehicles}")
    print(f"   成本: {best_no_tw.cost:.2f}元")

    if current_best:
        print(f"\n 原始解（含时间窗约束）:")
        print(f"   车辆数: {current_vehicles}")
        print(f"   成本: {current_best.cost:.2f}元")

        vehicle_diff = current_vehicles - no_tw_vehicles
        if vehicle_diff > 5:
            print(f"\n 结论: 时间窗是主要限制因素 (差异: {vehicle_diff}辆车)")
        else:
            print(f"\n 结论: 时间窗不是主要限制因素 (差异: {vehicle_diff}辆车)")

    result = {
        'no_tw_vehicles': no_tw_vehicles,
        'no_tw_cost': best_no_tw.cost,
        'current_vehicles': current_vehicles,
        'current_cost': current_best.cost if current_best else 0,
        'time_window_is_main_constraint': (current_vehicles - no_tw_vehicles) > 5 if current_best else False
    }

    print("=" * 60)
    return result

def create_better_initial_population(self, current_solution_pop: List[Individual] = None) -> List[Individual]:
    """
    创建包含拼单路径的改进初始种群

    Args:
        current_solution_pop: 当前解列表（如果有）

    Returns:
```

```
        改进后的种群
    """
    population = []

    if current_solution_pop:
        population.extend([Individual(ind.route_plan.copy()) for ind in current_solution_pop[:10]])

    clustered_solutions = self.cluster_and_assign(n_clusters=30)
    for sol in clustered_solutions:
        population.append(sol)

    for _ in range(20):
        random_combined = self.random_combine_customers()
        population.append(random_combined)

    for _ in range(self.population_size - len(population)):
        customers_with_demands = [(cid, w, v) for cid, w, v in self.customer_demands]
        if random.random() < 0.7:
            random.shuffle(customers_with_demands)
        else:
            customers_with_demands.sort(key=lambda x: x[1] + x[2], reverse=True)
        customers = [c[0] for c in customers_with_demands]
        route_plan = self._greedy_construct(customers)
        individual = Individual(route_plan)
        self._ensure_all_customers_silent(individual)
        population.append(individual)

    while len(population) < self.population_size:
        population.append(random.choice(population))

    return population[:self.population_size]

def _ensure_all_customers_silent(self, individual: Individual):
    """静默确保所有客户都被分配"""
    served = set(individual.get_all_customers())
    missing = set(self.customer_ids) - served
```

```
        if missing:
            for c in missing:
                if c in self.id_to_idx:
                    individual.route_plan.append([c])
            individual.route_plan = [r for r in individual.route_plan if r]
            individual.n_vehicles = len(individual.route_plan)

def cluster_and_assign(self, n_clusters: int = 30) -> List[Individual]:
    """
    基于地理坐标聚类，相近客户分配同车

    Args:
        n_clusters: 目标聚类数（等于目标车辆数）

    Returns:
        聚类解列表
    """
    if 'coords' not in dir(self) or self.coords is None:
        return []

    solutions = []

    coords = self.coords.copy()
    coords['客户 ID'] = self.customer_data['客户 ID'].values

    capacity_weight = self.max_weight * 0.95
    capacity_volume = self.max_volume * 0.95

    k = min(n_clusters, len(coords))

    indices = np.arange(len(coords))
    np.random.shuffle(indices)
    cluster_centers = coords.iloc[indices[:k]][['X', 'Y']].values

    clusters = [[] for _ in range(k)]
    for idx, row in coords.iterrows():
```

```
            cid = row['客户 ID']
            x, y = row['X'], row['Y']
            distances = [np.sqrt((x - cx)**2 + (y - cy)**2) for cx, cy in cluster_centers]
            nearest = np.argmin(distances)
            clusters[nearest].append(cid)

        routes = []
        for cluster in clusters:
            if not cluster:
                continue

            cluster_weight = sum(self.weights[self.id_to_idx[c]] for c in cluster if c in self.id_to_idx)
            cluster_volume = sum(self.volumes[self.id_to_idx[c]] for c in cluster if c in self.id_to_idx)

            if cluster_weight <= capacity_weight and cluster_volume <= capacity_volume:
                routes.append(cluster)
            else:
                sub_routes = self._split_cluster(cluster, capacity_weight, capacity_volume)
                routes.extend(sub_routes)

        for route in routes:
            if not route:
                continue
            route.sort(key=lambda c: np.sqrt(
                (coords[coords['客户 ID']==c]['X'].values[0] - Config.DEPOT_COORD[0])**2 +
                (coords[coords['客户 ID']==c]['Y'].values[0] - Config.DEPOT_COORD[1])**2
            ) if c in self.id_to_idx and len(coords[coords['客户 ID']==c]) > 0 else float('inf'))

        individual = Individual(routes)
        self._ensure_all_customers_silent(individual)
        solutions.append(individual)

        return solutions

    def _split_cluster(self, cluster: List[int], capacity_weight: float, capacity_volume: float) -> List[List[int]]:
        """拆分过大的聚类"""
```

```
if not cluster:
    return []

sub_routes = []
current_route = []
current_weight = 0
current_volume = 0

for c in cluster:
    if c not in self.id_to_idx:
        continue
    w = self.weights[self.id_to_idx[c]]
    v = self.volumes[self.id_to_idx[c]]

    if w <= capacity_weight and v <= capacity_volume:
        if current_weight + w <= capacity_weight and current_volume + v <= capacity_volume:
            current_route.append(c)
            current_weight += w
            current_volume += v
        else:
            if current_route:
                sub_routes.append(current_route)
            current_route = [c]
            current_weight = w
            current_volume = v
    else:
        if current_route:
            sub_routes.append(current_route)
        sub_routes.append([c])
        current_route = []
        current_weight = 0
        current_volume = 0

if current_route:
    sub_routes.append(current_route)
```

```
        return sub_routes


    def random_combine_customers(self) -> Individual:
        """
        创建随机拼单解

        Returns:
            随机拼单个体
        """
        customers = list(self.customer_ids)
        random.shuffle(customers)

        routes = []
        current_route = []
        current_weight = 0
        current_volume = 0
        capacity_weight = self.max_weight * 0.95
        capacity_volume = self.max_volume * 0.95

        for c in customers:
            if c not in self.id_to_idx:
                continue
            w = self.weights[self.id_to_idx[c]]
            v = self.volumes[self.id_to_idx[c]]

            if current_weight + w <= capacity_weight and current_volume + v <= capacity_volume:
                current_route.append(c)
                current_weight += w
                current_volume += v
            else:
                if current_route:
                    routes.append(current_route)
                current_route = [c]
                current_weight = w
                current_volume = v
```

```
        if current_route:
            routes.append(current_route)

        individual = Individual(routes)
        self._ensure_all_customers_silent(individual)
        return individual

    def _ensure_all_customers(self, customers: List[int]) -> List[List[int]]:
        """确保所有客户都被分配到路径"""
        routes = self._greedy_construct(customers)
        served = set()
        for route in routes:
            served.update(route)

        missing = set(self.customer_ids) - served
        if missing:
            # 将缺失的客户添加到现有路径或创建新路径
            for c in missing:
                if c in self.id_to_idx:
                    idx = self.id_to_idx[c]
                    w = self.weights[idx]
                    v = self.volumes[idx]
                    capacity_weight = self.max_weight * 0.95
                    capacity_volume = self.max_volume * 0.95

                    # 计算需要的车辆数
                    n_from_weight = int(np.ceil(w / capacity_weight))
                    n_from_volume = int(np.ceil(v / capacity_volume))
                    n_vehicles = max(n_from_weight, n_from_volume)

                    # 计算每个部分的需求
                    portion_w = w / n_vehicles
                    portion_v = v / n_vehicles

                    # 尝试将每个部分添加到现有路径
                    for _ in range(n_vehicles):
```

```
                added = False
                for route in routes:
                    # 计算当前路径的总载重和体积
                    route_weight = 0
                    route_volume = 0
                    for customer in route:
                        if customer in self.id_to_idx:
                            route_weight += self.weights[self.id_to_idx[customer]]
                            route_volume += self.volumes[self.id_to_idx[customer]]

                    if route_weight + portion_w <= capacity_weight and route_volume + portion_v <= capacity_volume:
                        route.append(c)
                        added = True
                        break
                # 如果无法添加到现有路径，创建新路径
                if not added:
                    routes.append([c])

    return routes

def _greedy_construct(self, customers: List[int]) -> List[List[int]]:
    """贪心构造初始解 - 使用最近邻策略实现真正的聚类和拼单"""
    # 去重，确保每个客户只处理一次
    unique_customers = list(set(customers))

    # 客户需求映射
    customer_demands = {}
    for cid in unique_customers:
        if cid in self.id_to_idx:
            idx = self.id_to_idx[cid]
            customer_demands[cid] = (self.weights[idx], self.volumes[idx])

    capacity_weight = self.max_weight * 0.95
    capacity_volume = self.max_volume * 0.95

    # 生成所有客户的需求部分（包括拆分的）
```

```
all_parts = []

# 处理所有客户，将需求拆分为多个部分
for cid in unique_customers:
    if cid not in customer_demands:
        continue
    w, v = customer_demands[cid]

    # 检查是否需要拆分
    if w > capacity_weight or v > capacity_volume:
        # 需要拆分，计算需要的车辆数
        n_from_weight = int(np.ceil(w / capacity_weight))
        n_from_volume = int(np.ceil(v / capacity_volume))
        n_vehicles = max(n_from_weight, n_from_volume)

        # 计算每个部分的需求
        portion_w = w / n_vehicles
        portion_v = v / n_vehicles

        # 添加所有部分（每个部分都是独立的可分配单元）
        for i in range(n_vehicles):
            all_parts.append((cid, portion_w, portion_v, i))  # 添加部分编号
    else:
        # 不需要拆分，直接添加
        all_parts.append((cid, w, v, 0))

# 获取距离函数
def get_distance_to_center(cid):
    idx = self.id_to_idx[cid] + 1
    return self.distance_matrix[0][idx]

def get_distance_between(c1, c2):
    idx1 = self.id_to_idx[c1] + 1
    idx2 = self.id_to_idx[c2] + 1
    return self.distance_matrix[idx1][idx2]
```

```
# 构建车辆 - 使用真正的最近邻贪心策略
vehicles = []  # [(weight, volume, [customers])]
assigned = set()  # 跟踪已分配的 (cid, part_id)

while len(assigned) < len(all_parts):
    # 创建新车
    current_route = []
    current_weight = 0
    current_volume = 0

    # 从配送中心开始
    last_cid = 0  # 0 表示配送中心

    while True:
        # 找到距 last_cid 最近的未分配客户
        best_part = None
        best_dist = float('inf')

        for p in all_parts:
            if (p[0], p[3]) not in assigned:
                if last_cid == 0:
                    dist = get_distance_to_center(p[0])
                else:
                    dist = get_distance_between(last_cid, p[0])
                if dist < best_dist:
                    best_dist = dist
                    best_part = p

        if best_part is None:
            break  # 没有更多可分配的客户

        c_cid, c_w, c_v, c_pid = best_part

        # 检查能否容纳
        if current_weight + c_w <= capacity_weight and current_volume + c_v <= capacity_volume:
            current_route.append((c_cid, c_pid))
```

```
                current_weight += c_w
                current_volume += c_v
                assigned.add((c_cid, c_pid))
                last_cid = c_cid  # 更新为当前客户
            else:
                break  # 当前车辆已满，需要创建新车

        # 只有当有客户时才添加车辆
        if current_route:
            route_cids = list(dict.fromkeys([p[0] for p in current_route]))
            vehicles.append((current_weight, current_volume, route_cids))
        else:
            break  # 无法分配更多客户

    # 移除空车辆
    vehicles = [v for v in vehicles if v[2]]

    # 构建路线计划
    route_plan = [load[2] for load in vehicles if load[2]]

    return route_plan if route_plan else [[]]

def _log_construction(self, route_plan: List[List[int]], generation: int = 0):
    """记录路径构造日志（只在特定代数打印）"""
    if generation in [0, 1] or generation % 100 == 0:
        total_customers = sum(len(r) for r in route_plan)
        avg_customers = total_customers / len(route_plan) if route_plan else 0
        total_distance_before = sum(self._calculate_route_distance(r) for r in route_plan)
        print(f"  [代数{generation}] 构造了{len(route_plan)}条路径，平均每车{avg_customers:.1f}个客户，总距离{total_distance_before:.2f}km")

def evaluate_population(self, population: List[Individual], generation: int = 0):
    """评估种群（串行）- 支持多目标优化"""
    generation_costs = []
    generation_emissions = []
    generation_vehicles = []
```

```
for ind in population:
    self._smart_merge_routes(ind)

    cost, carbon, _ = self.cost_calculator.calculate_cost(
        ind, self.weights, self.volumes, self.customer_ids
    )
    ind.cost = cost
    ind.carbon = carbon

    num_vehicles = len([r for r in ind.route_plan if r])
    vehicle_penalty = num_vehicles * 800
    penalized_cost = cost + vehicle_penalty

    if cost <= 0 or penalized_cost <= 0:
        print(f"  [警告] 代数{generation}：非正成本 cost={cost:.2f}, penalized={penalized_cost:.2f}, num_vehicles={num_vehicles}")
        ind.cost = 400 * max(1, num_vehicles)
        penalized_cost = ind.cost + vehicle_penalty

    if penalized_cost > 1e9:
        print(f"  [警告] 代数{generation}：异常高成本 penalized={penalized_cost:.2e}，限制上限")
        penalized_cost = min(penalized_cost, 1e6)

    # 多目标适应度计算
    if self.multi_objective:
        generation_costs.append(ind.cost)
        generation_emissions.append(ind.carbon)
        generation_vehicles.append(num_vehicles)
        # 暂时先不设置 fitness，等收集完所有值后再计算归一化适应度
    else:
        if penalized_cost > 0:
            ind.fitness = 1.0 / penalized_cost
        else:
            ind.fitness = 1e-10

        if not np.isfinite(ind.fitness):
            print(f"  [警告] 代数{generation}：非有限适应度 {ind.fitness:.2e}")
```

```
                ind.fitness = 1e-10

# 多目标优化：计算归一化适应度
if self.multi_objective:
    # 初始化归一化基准（第 0 代）
    if self.max_cost is None:
        self.max_cost = max(generation_costs)
        self.max_emission = max(generation_emissions)
        self.max_vehicles = max(generation_vehicles)
        print(f"  [多目标] 归一化基准：max_cost={self.max_cost:.2f}, max_emission={self.max_emission:.2f}, max_vehicles={self.max_vehicles}")

    # 更新归一化基准（根据最新代）
    self.max_cost = max(self.max_cost, max(generation_costs))
    self.max_emission = max(self.max_emission, max(generation_emissions))
    self.max_vehicles = max(self.max_vehicles, max(generation_vehicles))

    # 计算每个个体的多目标适应度
    for i, ind in enumerate(population):
        num_vehicles = len([r for r in ind.route_plan if r])

        # 归一化（值越小越好，所以用 1 - 归一化值）
        norm_cost = ind.cost / self.max_cost if self.max_cost > 0 else 1.0
        norm_emission = ind.carbon / self.max_emission if self.max_emission > 0 else 1.0
        norm_vehicles = num_vehicles / self.max_vehicles if self.max_vehicles > 0 else 1.0

        # 加权求和（越小越好，所以用 1 减去综合得分）
        weighted_score = (self.weight_cost * norm_cost +
                          self.weight_emission * norm_emission +
                          self.weight_vehicles * norm_vehicles)

        # 适应度：加权得分越小越好，所以 fitness = 1 / (weighted_score + epsilon)
        ind.fitness = 1.0 / (weighted_score + 1e-6)

        # 保存归一化指标用于调试
        ind.norm_cost = norm_cost
        ind.norm_emission = norm_emission
```

```
                ind.norm_vehicles = norm_vehicles
                ind.weighted_score = weighted_score

        if generation % 50 == 0 or generation < 5:
            min_cost, max_cost, avg_cost = min(generation_costs), max(generation_costs), sum(generation_costs)/len(generation_costs)
            print(f"   [成本诊断] 代数{generation}：min={min_cost:.2f}, max={max_cost:.2f}, avg={avg_cost:.2f}")

            if self.multi_objective:
                min_score = min(ind.weighted_score for ind in population)
                max_score = max(ind.weighted_score for ind in population)
                avg_score = sum(ind.weighted_score for ind in population)/len(population)
                print(f"   [多目标] 代数{generation}：加权得分 min={min_score:.4f}, max={max_score:.4f}, avg={avg_score:.4f}")

def _calculate_route_center(self, route: List[int]) -> Tuple[float, float]:
    """计算路径中心坐标（用于地理邻近判断）"""
    if not route:
        return (0, 0)
    if self.coords is None:
        return (0, 0)
    xs, ys = [], []
    x_col = 'X' if 'X' in self.coords.columns else self.coords.columns[0]
    y_col = 'Y' if 'Y' in self.coords.columns else self.coords.columns[1] if len(self.coords.columns) > 1 else self.coords.columns[0]

    for c in route:
        if c in self.id_to_idx:
            idx = self.id_to_idx[c]
            if idx < len(self.coords):
                try:
                    x_val = float(self.coords.iloc[idx][x_col])
                    y_val = float(self.coords.iloc[idx][y_col])
                    xs.append(x_val)
                    ys.append(y_val)
                except (ValueError, TypeError):
                    pass
    if xs:
        return (sum(xs) / len(xs), sum(ys) / len(ys))
```

```
        return (0, 0)

    def _route_distance(self, route1: List[int], route2: List[int]) -> float:
        """计算两条路径之间的最小距离（基于端点）"""
        if not route1 or not route2:
            return float('inf')
        d1 = self.distance_matrix[route1[-1], route2[0]] if route1 and route2 else float('inf')
        d2 = self.distance_matrix[route2[-1], route1[0]] if route1 and route2 else float('inf')
        return min(d1, d2)

    def _evaluate_merge_benefit(self, route1: List[int], route2: List[int]) -> Tuple[float, float]:
        """评估合并收益：返回(节省成本, 时间窗违反惩罚)"""
        w1 = sum(self.weights[self.id_to_idx[c]] for c in route1 if c in self.id_to_idx)
        v1 = sum(self.volumes[self.id_to_idx[c]] for c in route1 if c in self.id_to_idx)
        w2 = sum(self.weights[self.id_to_idx[c]] for c in route2 if c in self.id_to_idx)
        v2 = sum(self.volumes[self.id_to_idx[c]] for c in route2 if c in self.id_to_idx)

        merged_w, merged_v = w1 + w2, v1 + v2
        max_w, max_v = self.max_weight * 0.98, self.max_volume * 0.98

        overload_penalty = 0
        if merged_w > max_w:
            overload_penalty += (merged_w - max_w) * 1.0
        if merged_v > max_v:
            overload_penalty += (merged_v - max_v) * 2.0

        merge_savings = 400 + self._route_distance(route1, route2) * 0.8

        return merge_savings, overload_penalty

    def _smart_merge_routes(self, individual: Individual) -> bool:
        """智能合并路径：考虑地理邻近性和软时间窗约束"""
        merged = False
        max_w, max_v = self.max_weight * 0.98, self.max_volume * 0.98
        safety_factor = 1.5
```

```
route_loads = []
for route in individual.route_plan:
    if not route:
        continue
    total_w = sum(self.weights[self.id_to_idx[c]] for c in route if c in self.id_to_idx)
    total_v = sum(self.volumes[self.id_to_idx[c]] for c in route if c in self.id_to_idx)
    route_loads.append((route, total_w, total_v))

changed = True
max_iterations = 5
iteration = 0
while changed and iteration < max_iterations:
    changed = False
    iteration += 1

    route_loads.sort(key=lambda x: x[1] + x[2])

    new_route_loads = []
    used = set()

    for i in range(len(route_loads)):
        if i in used:
            continue
        route_i, w_i, v_i = route_loads[i]
        center_i = self._calculate_route_center(route_i)

        best_j = -1
        best_score = float('inf')

        for j in range(i + 1, len(route_loads)):
            if j in used:
                continue
            route_j, w_j, v_j = route_loads[j]

            if w_i + w_j > max_w or v_i + v_j > max_v:
                continue
```

```
                savings, violation = self._evaluate_merge_benefit(route_i, route_j)

                if savings > violation * safety_factor:
                    center_j = self._calculate_route_center(route_j)
                    dist = ((center_i[0] - center_j[0])**2 + (center_i[1] - center_j[1])**2) ** 0.5

                    score = dist / (savings - violation * safety_factor + 0.1)
                    if score < best_score:
                        best_score = score
                        best_j = j

            if best_j >= 0:
                route_j, w_j, v_j = route_loads[best_j]
                new_route = route_i + route_j
                new_w, new_v = w_i + w_j, v_i + v_j
                new_route_loads.append((new_route, new_w, new_v))
                used.add(i)
                used.add(best_j)
                changed = True
                merged = True
            else:
                if w_i > 0 or v_i > 0:
                    new_route_loads.append((route_i, w_i, v_i))

        route_loads = new_route_loads

    individual.route_plan = [r[0] for r in route_loads if r[0]]
    return merged

def _merge_routes(self, individual: Individual) -> bool:
    """尝试合并路径以减少车辆数，返回是否进行了合并"""
    merged = False
    capacity_weight = self.max_weight * 0.95
    capacity_volume = self.max_volume * 0.95
```

```
    route_loads = []
    for route in individual.route_plan:
        total_w = sum(self.weights[self.id_to_idx[c]] for c in route if c in self.id_to_idx)
        total_v = sum(self.volumes[self.id_to_idx[c]] for c in route if c in self.id_to_idx)
        route_loads.append((total_w, total_v))

    changed = True
    while changed:
        changed = False
        for i in range(len(individual.route_plan)):
            if not individual.route_plan[i]:
                continue
            for j in range(i + 1, len(individual.route_plan)):
                if not individual.route_plan[j]:
                    continue

                w1, v1 = route_loads[i]
                w2, v2 = route_loads[j]

                if w1 + w2 <= capacity_weight and v1 + v2 <= capacity_volume:
                    individual.route_plan[i].extend(individual.route_plan[j])
                    individual.route_plan[j] = []
                    route_loads[i] = (w1 + w2, v1 + v2)
                    route_loads[j] = (0, 0)
                    changed = True
                    merged = True

        individual.route_plan = [r for r in individual.route_plan if r]
        route_loads = [l for l in route_loads if l[0] > 0 or l[1] > 0]

    return merged

def selection(self, population: List[Individual]) -> List[Individual]:
    """锦标赛选择"""
    selected = []
    tournament_size = 3
```

```
        for _ in range(len(population)):
            tournament = random.sample(population, tournament_size)
            winner = max(tournament, key=lambda x: x.fitness)  # 使用适应度选择
            selected.append(winner)

        return selected

    def order_crossover(self, p1_chrom: List[int], p2_chrom: List[int]) -> List[int]:
        """正确的顺序交叉(OX)"""
        p1_customers = [g for g in p1_chrom if g != SEPARATOR]
        p2_customers = [g for g in p2_chrom if g != SEPARATOR]

        if not p1_customers or not p2_customers:
            return p1_chrom.copy()

        size = len(p1_customers)
        point1 = random.randint(0, size - 1)
        point2 = random.randint(point1, size - 1)

        seg_customers = p1_customers[point1:point2+1]
        seg_set = set(seg_customers)

        remaining = [c for c in p2_customers if c not in seg_set]

        # 正确做法：找到 point1 在父 2 中的位置，然后按顺序取出
        # 找到第一个不在 seg 中的客户在 p2 中的位置
        first_remaining_pos = 0
        for i, c in enumerate(p2_customers):
            if c not in seg_set:
                first_remaining_pos = i
                break

        # 从 p2 中按顺序取剩余客户（从 first_remaining_pos 开始循环）
        ordered_remaining = []
        p2_len = len(p2_customers)
```

```
    idx = first_remaining_pos
    while len(ordered_remaining) < len(remaining):
        if p2_customers[idx % p2_len] not in seg_set:
            ordered_remaining.append(p2_customers[idx % p2_len])
        idx += 1

    # 构建子代染色体
    child_customers = ordered_remaining[:point1] + seg_customers + ordered_remaining[point1:]

    n_sep = p1_chrom.count(SEPARATOR)
    if n_sep == 0:
        return child_customers

    result = []
    sep_idx = 0
    sep_positions = self._get_separator_positions(p1_chrom, n_sep)

    for i, c in enumerate(child_customers):
        if sep_idx < len(sep_positions) and i == sep_positions[sep_idx]:
            result.append(SEPARATOR)
            sep_idx += 1
        result.append(c)

    while sep_idx < len(sep_positions):
        result.append(SEPARATOR)
        sep_idx += 1

    return result

def _get_separator_positions(self, chrom: List[int], n_sep: int) -> List[int]:
    """获取染色体中客户的位置索引（用于计算分隔符应插入的位置）"""
    customer_positions = [i for i, g in enumerate(chrom) if g != SEPARATOR]
    if len(customer_positions) == 0:
        return []

    step = len(customer_positions) / (n_sep + 1)
```

```
        positions = [int((i + 1) * step) for i in range(n_sep)]
        return sorted(set(positions))

    def crossover(self, parent1: Individual, parent2: Individual) -> Tuple[Individual, Individual]:
        """顺序交叉(OX) - 改进版"""
        if random.random() > self.crossover_rate:
            return parent1, parent2

        chrom1 = parent1.chromosome
        chrom2 = parent2.chromosome

        if len(chrom1) < 2 or len(chrom2) < 2:
            return parent1, parent2

        child1_chrom = self.order_crossover(chrom1, chrom2)
        child2_chrom = self.order_crossover(chrom2, chrom1)

        child1 = Individual(chromosome=child1_chrom)
        child2 = Individual(chromosome=child2_chrom)

        all_ids = set(self.customer_ids)
        child1.validate_and_fix(all_ids, self.customer_ids)
        child2.validate_and_fix(all_ids, self.customer_ids)

        return child1, child2

    def _regroup_customers(self, customers: List[int]) -> List[List[int]]:
        """重新分组客户到路径 - 实现真正的聚类和拼单"""
        # 直接使用_greedy_construct方法构建路线，确保车辆数合理
        route_plan = self._greedy_construct(customers)

        return route_plan

    def _optimize_routes(self, routes: List[List[int]]) -> List[List[int]]:
        """对路线进行2-opt局部优化"""
        optimized_routes = []
```

```
        for route in routes:
            if len(route) > 1:
                # 简单的 2-opt 优化
                optimized_route = self._two_opt(route)
                optimized_routes.append(optimized_route)
            else:
                optimized_routes.append(route)
        return optimized_routes

    def _two_opt(self, route: List[int]) -> List[int]:
        """2-opt 局部优化算法"""
        best_route = route.copy()
        improved = True

        while improved:
            improved = False
            for i in range(1, len(best_route) - 1):
                for j in range(i + 1, len(best_route)):
                    new_route = best_route.copy()
                    new_route[i:j+1] = reversed(new_route[i:j+1])

                    if self._calculate_route_distance(new_route) < self._calculate_route_distance(best_route):
                        best_route = new_route
                        improved = True

        return best_route

    def _calculate_route_distance(self, route: List[int]) -> float:
        """计算路线的总距离"""
        if not route:
            return 0

        total_distance = 0
        prev_idx = 0  # 配送中心

        for cid in route:
```

```
            if cid in self.id_to_idx:
                customer_idx = self.id_to_idx[cid] + 1  # +1 因为矩阵第一行是配送中心
                total_distance += self.distance_matrix[prev_idx, customer_idx]
                prev_idx = customer_idx

        # 返回配送中心
        total_distance += self.distance_matrix[prev_idx, 0]
        return total_distance

    def mutate(self, individual: Individual) -> Individual:
        """变异操作 - 改进版"""
        if random.random() > self.mutation_rate:
            return individual

        mutation_type = random.choice(['2opt', 'swap', 'move', 'route_merge'])

        if mutation_type == '2opt' and len(individual.route_plan) > 0:
            route_idx = random.randint(0, len(individual.route_plan) - 1)
            route = individual.route_plan[route_idx]
            if len(route) >= 2:
                i, j = sorted(random.sample(range(len(route)), 2))
                route[i:j+1] = reversed(route[i:j+1])

        elif mutation_type == 'swap' and len(individual.route_plan) >= 2:
            r1, r2 = random.sample(range(len(individual.route_plan)), 2)
            if individual.route_plan[r1] and individual.route_plan[r2]:
                c1 = random.choice(individual.route_plan[r1])
                c2 = random.choice(individual.route_plan[r2])
                idx1 = individual.route_plan[r1].index(c1)
                idx2 = individual.route_plan[r2].index(c2)
                individual.route_plan[r1][idx1], individual.route_plan[r2][idx2] = c2, c1

        elif mutation_type == 'move':
            valid_routes = [i for i, r in enumerate(individual.route_plan) if len(r) >= 2]
            if valid_routes:
                r_idx = random.choice(valid_routes)
```

```
                route = individual.route_plan[r_idx]
                c_idx = random.randint(0, len(route) - 1)
                customer = route.pop(c_idx)

                target_routes = [i for i in range(len(individual.route_plan)) if i != r_idx]
                if target_routes:
                    target_idx = random.choice(target_routes)
                    insert_pos = random.randint(0, len(individual.route_plan[target_idx]))
                    individual.route_plan[target_idx].insert(insert_pos, customer)
                else:
                    route.insert(c_idx, customer)

        elif mutation_type == 'route_merge':
            if len(individual.route_plan) >= 2:
                r1, r2 = random.sample(range(len(individual.route_plan)), 2)
                if individual.route_plan[r1] and individual.route_plan[r2]:
                    total_w = sum(self.weights[self.id_to_idx[c]] for c in individual.route_plan[r1] if c in self.id_to_idx)
                    add_w = sum(self.weights[self.id_to_idx[c]] for c in individual.route_plan[r2] if c in self.id_to_idx)
                    total_v = sum(self.volumes[self.id_to_idx[c]] for c in individual.route_plan[r1] if c in self.id_to_idx)
                    add_v = sum(self.volumes[self.id_to_idx[c]] for c in individual.route_plan[r2] if c in self.id_to_idx)

                    if total_w + add_w <= self.max_weight * 0.95 and total_v + add_v <= self.max_volume * 0.95:
                        individual.route_plan[r1].extend(individual.route_plan[r2])
                        individual.route_plan[r2] = []

        individual.route_plan = [r for r in individual.route_plan if r]
        individual.n_vehicles = len(individual.route_plan)

        all_ids = set(self.customer_ids)
        individual.validate_and_fix(all_ids)

        return individual

    def local_search_2opt(self, individual: Individual) -> Individual:
        """2-opt 局部搜索优化路径"""
        improved = True
```

```
iteration = 0
max_iterations = 3  # 限制局部搜索迭代次数

while improved and iteration < max_iterations:
    improved = False
    iteration += 1

    for route_idx in range(len(individual.route_plan)):
        route = individual.route_plan[route_idx]
        if len(route) < 4:
            continue

        # 检查是否需要拆分（客户需求超过车辆容量）
        route_weight = sum(self.weights[self.id_to_idx[c]] for c in route if c in self.id_to_idx)
        route_volume = sum(self.volumes[self.id_to_idx[c]] for c in route if c in self.id_to_idx)

        if route_weight > self.max_weight * 0.95 or route_volume > self.max_volume * 0.95:
            continue

        # 2-opt: 尝试逆转到不同位置
        best_route = route.copy()
        best_cost_reduction = 0

        for i in range(1, len(route) - 1):
            for j in range(i + 1, len(route)):
                # 创建新路径：反转[i:j]段
                new_route = route[:i] + route[i:j][::-1] + route[j:]
                new_weight = sum(self.weights[self.id_to_idx[c]] for c in new_route if c in self.id_to_idx)
                new_volume = sum(self.volumes[self.id_to_idx[c]] for c in new_route if c in self.id_to_idx)

                # 检查约束是否满足
                if new_weight <= self.max_weight * 0.95 and new_volume <= self.max_volume * 0.95:
                    # 估算成本变化（基于距离）
                    old_dist = self._estimate_route_distance(route)
                    new_dist = self._estimate_route_distance(new_route)
                    dist_reduction = old_dist - new_dist
```

```
                            if dist_reduction > best_cost_reduction:
                                best_cost_reduction = dist_reduction
                                best_route = new_route

                    if best_route != route:
                        individual.route_plan[route_idx] = best_route
                        improved = True

        return individual

def _estimate_route_distance(self, route: List[int]) -> float:
    """估算路径距离（使用欧氏距离近似）"""
    if not route:
        return 0

    # 假设客户坐标在 distance_matrix 中的索引
    # 客户 ID 从 1 开始，索引从 1 开始（0 是配送中心）
    total_dist = 0
    prev_idx = 0  # 配送中心索引

    for c in route:
        if c in self.id_to_idx:
            # 客户 c 的索引是 id_to_idx[c] + 1
            customer_idx = self.id_to_idx[c] + 1
            total_dist += self.distance_matrix[prev_idx, customer_idx]
            prev_idx = customer_idx

    # 返回配送中心
    total_dist += self.distance_matrix[prev_idx, 0]
    return total_dist

def elitism(self, population: List[Individual]) -> List[Individual]:
    """精英保留"""
    n_elite = int(len(population) * self.elite_rate)
    sorted_pop = sorted(population, key=lambda x: x.cost)
```

```
        return [Individual(ind.route_plan.copy()) for ind in sorted_pop[:n_elite]]


    def run(self) -> Individual:
        """运行遗传算法"""
        print("=" * 60)
        print("开始遗传算法优化...")
        print("=" * 60)

        # 初始化种群
        population = self.initialize_population()
        self.evaluate_population(population, generation=0)

        # 记录初始最优（根据多目标设置选择依据）
        if self.multi_objective:
            best = min(population, key=lambda x: x.weighted_score)
            self.best_cost = best.cost
            self.best_weighted_score = best.weighted_score
        else:
            best = min(population, key=lambda x: x.cost)
            self.best_cost = best.cost

        self.best_individual = Individual(best.route_plan.copy())
        self.history.append(self.best_cost)

        print(f"初始最优成本: {self.best_cost:.2f}元")
        print(f"初始车辆数: {len(best.route_plan)}")
        if self.multi_objective:
            print(f"初始加权得分: {best.weighted_score:.4f}")

        # 记录适应度历史
        fitness_history = [best.fitness if best.fitness > 0 else 1e-10]

        # 记录初始种群的构造信息
        self._log_construction(best.route_plan, generation=0)

        # 迭代进化
```

```
for gen in range(self.max_generations):
    # 打印进度
    if gen % 50 == 0 or gen < 5:
        current_best = min(population, key=lambda x: x.cost)
        best_fitness = current_best.fitness
        if not np.isfinite(best_fitness) or best_fitness <= 0:
            best_fitness = 1e-10
        vehicle_count = len([r for r in current_best.route_plan if r])
        print(f"第{gen}代: 最优成本={current_best.cost:.2f}元, 最优适应度={best_fitness:.2e}, 车辆数={vehicle_count}")

    # 锦标赛选择
    selected = self.selection(population)

    # 交叉
    offspring = []
    for i in range(0, len(selected) - 1, 2):
        child1, child2 = self.crossover(selected[i], selected[i+1])
        offspring.extend([child1, child2])

    # 变异
    offspring = [self.mutate(ind) for ind in offspring]

    # 合并
    population.extend(offspring)

    # 评估
    self.evaluate_population(population, generation=gen+1)

    # 自适应参数调整
    if (gen + 1) % 50 == 0:
        diversity = self.calculate_diversity(population)
        print(f"  [多样性] 代数{gen+1}：变异系数={diversity:.4f}")
        self.adapt_parameters(gen + 1, diversity)

    # 精英保留
    elites = self.elitism(population)
```

```
# 对精英个体进行局部搜索优化
for elite in elites:
    self.local_search_2opt(elite)
    # 重新评估
    cost, carbon, _ = self.cost_calculator.calculate_cost(
        elite, self.weights, self.volumes, self.customer_ids
    )
    elite.cost = cost
    elite.carbon = carbon

# 选择下一代
next_gen = self.selection(population)
next_gen = next_gen[:self.population_size - len(elites)]
next_gen.extend(elites)

population = next_gen

# 记录最优
current_best = min(population, key=lambda x: x.cost)
if current_best.cost < self.best_cost:
    self.best_cost = current_best.cost
    self.best_individual = Individual(current_best.route_plan.copy())
    if gen % 50 == 0 or gen < 10:  # 减少打印频率
        print(f"第{gen+1}代: 发现更优解 {self.best_cost:.2f}元")

self.history.append(self.best_cost)

# 早停检查（连续多代无显著改进）
if gen >= self.patience:
    recent_costs = self.history[-self.patience:]
    max_recent = max(recent_costs)
    if max_recent - self.best_cost < self.min_improvement:
        print(f"第{gen+1}代: 早停收敛（最佳成本: {self.best_cost:.2f}元）")
        break
```

```
        return self.best_individual

    def simulated_annealing(self, individual: Individual, max_iterations: int = 500,
                            initial_temp: float = 100.0, cooling_rate: float = 0.99) -> Individual:
        """
        模拟退火局部优化算法

        Args:
            individual: 初始解
            max_iterations: 最大迭代次数
            initial_temp: 初始温度
            cooling_rate: 降温速率

        Returns:
            优化后的解
        """
        print("\n" + "=" * 60)
        print("开始模拟退火优化...")
        print("=" * 60)

        current_solution = Individual(individual.route_plan.copy())
        cost, carbon, _ = self.cost_calculator.calculate_cost(
            current_solution, self.weights, self.volumes, self.customer_ids
        )
        current_solution.cost = cost
        current_solution.carbon = carbon

        best_solution = Individual(current_solution.route_plan.copy())
        best_solution.cost = current_solution.cost
        best_solution.carbon = current_solution.carbon

        current_temp = initial_temp
        no_improvement = 0
        max_no_improvement = 100

        print(f"初始成本: {current_solution.cost:.2f}元, 车辆数: {len(current_solution.route_plan)}")
```

```
for i in range(max_iterations):
    # 生成邻域解
    neighbor = self._generate_neighbor(current_solution)

    # 评估邻域解
    neighbor_cost, neighbor_carbon, _ = self.cost_calculator.calculate_cost(
        neighbor, self.weights, self.volumes, self.customer_ids
    )
    neighbor.cost = neighbor_cost
    neighbor.carbon = neighbor_carbon

    # 计算目标值变化（支持多目标）
    if self.multi_objective:
        current_score = self._calculate_weighted_score(current_solution)
        neighbor_score = self._calculate_weighted_score(neighbor)
        delta = neighbor_score - current_score
    else:
        delta = neighbor.cost - current_solution.cost

    # 接受准则
    if delta < 0:
        # 接受更优解
        current_solution = neighbor
        if neighbor.cost < best_solution.cost:
            best_solution = Individual(neighbor.route_plan.copy())
            best_solution.cost = neighbor.cost
            best_solution.carbon = neighbor.carbon
            no_improvement = 0
            if i % 50 == 0:
                print(f"  SA 迭代{i}: 找到更优解 {best_solution.cost:.2f}元")
    else:
        # 以概率接受较差解
        acceptance_prob = np.exp(-delta / (current_temp + 1e-10))
        if random.random() < acceptance_prob:
            current_solution = neighbor
```

```
                no_improvement += 1

            # 降温
            current_temp *= cooling_rate

            # 早停
            if no_improvement >= max_no_improvement:
                print(f"  SA 迭代{i}: 早停收敛")
                break

    print(f"模拟退火完成: 最终成本 {best_solution.cost:.2f}元, 车辆数 {len(best_solution.route_plan)}")
    print("=" * 60)
    return best_solution

def tabu_search(self, individual: Individual, max_iterations: int = 300,
                tabu_size: int = 20, neighborhood_size: int = 5) -> Individual:
    """
    禁忌搜索局部优化算法

    Args:
        individual: 初始解
        max_iterations: 最大迭代次数
        tabu_size: 禁忌表大小
        neighborhood_size: 邻域大小

    Returns:
        优化后的解
    """
    print("\n" + "=" * 60)
    print("开始禁忌搜索优化...")
    print("=" * 60)

    current_solution = Individual(individual.route_plan.copy())
    cost, carbon, _ = self.cost_calculator.calculate_cost(
        current_solution, self.weights, self.volumes, self.customer_ids
    )
```

```
current_solution.cost = cost
current_solution.carbon = carbon

best_solution = Individual(current_solution.route_plan.copy())
best_solution.cost = current_solution.cost
best_solution.carbon = current_solution.carbon

tabu_list = []
no_improvement = 0
max_no_improvement = 80

print(f"初始成本: {current_solution.cost:.2f}元, 车辆数: {len(current_solution.route_plan)}")

for i in range(max_iterations):
    # 生成邻域
    neighborhood = []
    for _ in range(neighborhood_size):
        neighbor = self._generate_neighbor(current_solution)
        # 评估邻域解
        neighbor_cost, neighbor_carbon, _ = self.cost_calculator.calculate_cost(
            neighbor, self.weights, self.volumes, self.customer_ids
        )
        neighbor.cost = neighbor_cost
        neighbor.carbon = neighbor_carbon
        neighborhood.append(neighbor)

    # 选择最佳邻域解
    best_neighbor = None
    best_neighbor_cost = float('inf')

    for neighbor in neighborhood:
        # 检查是否在禁忌表中
        neighbor_hash = self._hash_solution(neighbor)
        is_tabu = neighbor_hash in tabu_list

        # 计算目标值
```

```
        if self.multi_objective:
            neighbor_score = self._calculate_weighted_score(neighbor)
            current_score = self._calculate_weighted_score(current_solution)

            if (not is_tabu) or (is_tabu and neighbor_score < self._calculate_weighted_score(best_solution)):
                # 接受非禁忌解或满足特赦条件的禁忌解
                if neighbor_score < self._calculate_weighted_score(current_solution) or best_neighbor is None:
                    if neighbor.cost < best_neighbor_cost:
                        best_neighbor = neighbor
                        best_neighbor_cost = neighbor.cost
        else:
            if (not is_tabu) or (is_tabu and neighbor.cost < best_solution.cost):
                if neighbor.cost < best_neighbor_cost:
                    best_neighbor = neighbor
                    best_neighbor_cost = neighbor.cost

    if best_neighbor is None:
        # 没有找到可接受的解，随机选择一个
        best_neighbor = random.choice(neighborhood)

    # 更新当前解
    current_solution = best_neighbor

    # 添加到禁忌表
    current_hash = self._hash_solution(current_solution)
    tabu_list.append(current_hash)
    if len(tabu_list) > tabu_size:
        tabu_list.pop(0)

    # 更新最优解
    if current_solution.cost < best_solution.cost:
        best_solution = Individual(current_solution.route_plan.copy())
        best_solution.cost = current_solution.cost
        best_solution.carbon = current_solution.carbon
        no_improvement = 0
    if i % 30 == 0:
```

```
                print(f"    TS 迭代{i}: 找到更优解 {best_solution.cost:.2f}元")
            else:
                no_improvement += 1

            # 早停
            if no_improvement >= max_no_improvement:
                print(f"    TS 迭代{i}: 早停收敛")
                break

        print(f"禁忌搜索完成: 最终成本 {best_solution.cost:.2f}元, 车辆数 {len(best_solution.route_plan)}")
        print("=" * 60)
        return best_solution

    def hybrid_optimization(self) -> Individual:
        """
        混合优化框架: 遗传算法 + 模拟退火 + 禁忌搜索

        Returns:
            最优解
        """
        print("\n" + "=" * 70)
        print("混合优化框架: GA + SA + TS")
        print("=" * 70)

        # 阶段 1: 遗传算法全局搜索
        print("\n[阶段 1/3] 遗传算法全局搜索...")
        ga_solution = self.run()

        # 阶段 2: 模拟退火局部优化
        print("\n[阶段 2/3] 模拟退火局部优化...")
        sa_solution = self.simulated_annealing(ga_solution)

        # 阶段 3: 禁忌搜索精细调整
        print("\n[阶段 3/3] 禁忌搜索精细调整...")
        ts_solution = self.tabu_search(sa_solution)
```

```
        print("\n" + "=" * 70)
        print("混合优化完成!")
        print(f"  GA 结果: {ga_solution.cost:.2f}元, {len(ga_solution.route_plan)}辆车")
        print(f"  SA 结果: {sa_solution.cost:.2f}元, {len(sa_solution.route_plan)}辆车")
        print(f"  TS 结果: {ts_solution.cost:.2f}元, {len(ts_solution.route_plan)}辆车")
        print("=" * 70)

        return ts_solution

    def _generate_neighbor(self, individual: Individual) -> Individual:
        """
        生成邻域解（用于模拟退火和禁忌搜索）

        邻域操作包括:
        - 2-opt 路径优化
        - 客户交换
        - 客户移动
        - 路径合并
        """
        neighbor = Individual(individual.route_plan.copy())

        # 随机选择一种邻域操作
        operation = random.choice(['2opt', 'swap', 'move', 'route_merge', 'split_route'])

        if operation == '2opt' and len(neighbor.route_plan) > 0:
            route_idx = random.randint(0, len(neighbor.route_plan) - 1)
            route = neighbor.route_plan[route_idx]
            if len(route) >= 2:
                i, j = sorted(random.sample(range(len(route)), 2))
                route[i:j+1] = reversed(route[i:j+1])

        elif operation == 'swap' and len(neighbor.route_plan) >= 2:
            r1, r2 = random.sample(range(len(neighbor.route_plan)), 2)
            if neighbor.route_plan[r1] and neighbor.route_plan[r2]:
                c1 = random.choice(neighbor.route_plan[r1])
                c2 = random.choice(neighbor.route_plan[r2])
```

```
            idx1 = neighbor.route_plan[r1].index(c1)
            idx2 = neighbor.route_plan[r2].index(c2)
            neighbor.route_plan[r1][idx1], neighbor.route_plan[r2][idx2] = c2, c1

    elif operation == 'move':
        valid_routes = [i for i, r in enumerate(neighbor.route_plan) if len(r) >= 2]
        if valid_routes:
            r_idx = random.choice(valid_routes)
            route = neighbor.route_plan[r_idx]
            c_idx = random.randint(0, len(route) - 1)
            customer = route.pop(c_idx)

            target_routes = [i for i in range(len(neighbor.route_plan)) if i != r_idx]
            if target_routes:
                target_idx = random.choice(target_routes)
                insert_pos = random.randint(0, len(neighbor.route_plan[target_idx]))
                neighbor.route_plan[target_idx].insert(insert_pos, customer)
            else:
                route.insert(c_idx, customer)

    elif operation == 'route_merge':
        if len(neighbor.route_plan) >= 2:
            r1, r2 = random.sample(range(len(neighbor.route_plan)), 2)
            if neighbor.route_plan[r1] and neighbor.route_plan[r2]:
                total_w = sum(self.weights[self.id_to_idx[c]] for c in neighbor.route_plan[r1] if c in self.id_to_idx)
                add_w = sum(self.weights[self.id_to_idx[c]] for c in neighbor.route_plan[r2] if c in self.id_to_idx)
                total_v = sum(self.volumes[self.id_to_idx[c]] for c in neighbor.route_plan[r1] if c in self.id_to_idx)
                add_v = sum(self.volumes[self.id_to_idx[c]] for c in neighbor.route_plan[r2] if c in self.id_to_idx)

                if total_w + add_w <= self.max_weight * 0.95 and total_v + add_v <= self.max_volume * 0.95:
                    neighbor.route_plan[r1].extend(neighbor.route_plan[r2])
                    neighbor.route_plan[r2] = []

    elif operation == 'split_route':
        # 拆分路径（增加车辆但可能减少距离）
        valid_routes = [i for i, r in enumerate(neighbor.route_plan) if len(r) >= 4]
```

```
            if valid_routes:
                r_idx = random.choice(valid_routes)
                route = neighbor.route_plan[r_idx]
                split_pos = random.randint(2, len(route) - 2)
                route1 = route[:split_pos]
                route2 = route[split_pos:]
                neighbor.route_plan[r_idx] = route1
                neighbor.route_plan.append(route2)

        # 清理空路径
        neighbor.route_plan = [r for r in neighbor.route_plan if r]
        neighbor.n_vehicles = len(neighbor.route_plan)

        # 确保所有客户都被分配
        all_ids = set(self.customer_ids)
        neighbor.validate_and_fix(all_ids)

        return neighbor

    def _hash_solution(self, individual: Individual) -> str:
        """
        生成解的哈希值（用于禁忌搜索）

        哈希基于路径结构，忽略路径内部顺序以提高效率
        """
        # 对每条路径的客户 ID 排序，然后生成哈希
        sorted_routes = []
        for route in individual.route_plan:
            sorted_route = sorted(route)
            sorted_routes.append(tuple(sorted_route))

        # 对所有路径排序，然后生成哈希
        sorted_routes.sort()

        return str(tuple(sorted_routes))
```

```
    def _calculate_weighted_score(self, individual: Individual) -> float:
        """
        计算加权得分（用于多目标优化）
        """
        if self.max_cost is None or self.max_emission is None or self.max_vehicles is None:
            return individual.cost

        num_vehicles = len([r for r in individual.route_plan if r])

        norm_cost = individual.cost / self.max_cost
        norm_emission = individual.carbon / self.max_emission
        norm_vehicles = num_vehicles / self.max_vehicles

        return (self.weight_cost * norm_cost +
                self.weight_emission * norm_emission +
                self.weight_vehicles * norm_vehicles)

# ============== 结果输出 ==============
class ResultWriter:
    """结果输出器"""

    def __init__(self, individual: Individual, cost_calculator: CostCalculator,
                 customer_data: pd.DataFrame, time_windows: np.ndarray):
        self.individual = individual
        self.cost_calculator = cost_calculator
        self.customer_data = customer_data
        self.time_windows = time_windows

    def save_to_file(self, filename='问题 1 结果_优化.txt'):
        """保存结果到文件"""
        with open(filename, 'w', encoding='utf-8') as f:
            f.write("=" * 70 + "\n")
            f.write("华中杯数学建模 A 题 - 问题 1 优化结果\n")
            f.write("城市绿色物流配送调度 - 静态环境下车辆调度\n")
            f.write("=" * 70 + "\n\n")
```

```
# 写入车辆使用方案
f.write("【车辆使用方案】\n")
f.write("-" * 50 + "\n")
f.write(f"使用车辆数量: {self.individual.n_vehicles}辆\n\n")

# 获取列名
weight_col = '总重量' if '总重量' in self.customer_data.columns else '重量'
volume_col = '总体积' if '总体积' in self.customer_data.columns else '体积'

total_distance = 0
total_cost, total_carbon, cost_details = self.cost_calculator.calculate_cost(
    self.individual,
    self.customer_data[weight_col].values,
    self.customer_data[volume_col].values,
    self.customer_data['客户 ID'].values
)

# 统计每个客户被服务的次数
from collections import Counter
customer_visits = Counter()
for route in self.individual.route_plan:
    for c in route:
        # 确保客户 ID 是整数
        c_int = int(c) if isinstance(c, (float, str)) else c
        customer_visits[c_int] += 1

for i, route in enumerate(self.individual.route_plan, 1):
    f.write(f"车辆 {i}:\n")
    f.write(f"  配送路径: 配送中心")
    for c in route:
        c_int = int(c) if isinstance(c, (float, str)) else c
        f.write(f" → 客户{c_int}")
    f.write(" → 配送中心\n")

    # 计算路径距离
    if route:
```

```
            distance = self._calculate_route_distance(route)
            total_distance += distance
            f.write(f"   路径距离: {distance:.2f}km\n")

        # 计算载重和体积（考虑拆分配送）
        weights = 0
        volumes = 0
        for c in route:
            # 确保客户 ID 是整数
            c_int = int(c) if isinstance(c, (float, str)) else c
            # 检查客户是否存在
            # 使用正确的客户 ID 列名
            customer_col = '客户 ID' if '客户 ID' in self.customer_data.columns else \
                           '目标客户编号' if '目标客户编号' in self.customer_data.columns else \
                           '客户编号' if '客户编号' in self.customer_data.columns else self.customer_data.columns[0]
            customer_mask = self.customer_data[customer_col] == c_int
            if customer_mask.any():
                total_weight = self.customer_data.loc[customer_mask, weight_col].values[0]
                total_volume = self.customer_data.loc[customer_mask, volume_col].values[0]
                # 计算每辆车的实际载重（总需求/服务次数）
                visit_count = customer_visits.get(c_int, 1)
                if visit_count > 0:
                    # 检查是否超载
                    individual_weight = total_weight / visit_count
                    individual_volume = total_volume / visit_count
                    weights += individual_weight
                    volumes += individual_volume
        f.write(f"   总载重: {weights:.2f}kg\n")
        f.write(f"   总体积: {volumes:.2f}m³\n\n")

    # 写入成本明细
    f.write("【成本明细】\n")
    f.write("-" * 50 + "\n")
    f.write(f"   启动成本: {cost_details['start_cost']:.2f}元\n")
    f.write(f"   运输成本: {cost_details['transport_cost']:.2f}元\n")
    f.write(f"   等待成本: {cost_details['wait_cost']:.2f}元\n")
```

```
            f.write(f"   晚到惩罚: {cost_details['late_penalty']:.2f}元\n")
            f.write(f"   超载罚款: {cost_details.get('overload_penalty', 0):.2f}元\n")
            f.write(f"   碳排放成本: {cost_details['carbon_cost']:.2f}元\n")
            f.write(f"   ----------------------------------\n")
            f.write(f"   总成本: {total_cost:.2f}元\n")
            f.write(f"   单车平均成本: {total_cost/self.individual.n_vehicles:.2f}元\n\n")

            # 计算并写入车辆利用率
            total_weight = sum(self.customer_data[weight_col].values)
            total_volume = sum(self.customer_data[volume_col].values)
            f.write(f"【车辆利用率分析】\n")
            f.write(f"   总货物重量: {total_weight:.2f}kg\n")
            f.write(f"   总货物体积: {total_volume:.2f}m³\n")
            f.write(f"   平均每车载重: {total_weight/self.individual.n_vehicles:.2f}kg\n")
            f.write(f"   平均每车体积: {total_volume/self.individual.n_vehicles:.2f}m³\n")
            f.write(f"   载重利用率: {total_weight/(self.individual.n_vehicles*3000)*100:.1f}%\n")
            f.write(f"   体积利用率: {total_volume/(self.individual.n_vehicles*15)*100:.1f}%\n\n")

            f.write(f"总行驶距离: {total_distance:.2f}km\n")
            f.write(f"总碳排放: {total_carbon:.2f}kg CO2\n")

            f.write("\n" + "=" * 70 + "\n")

        print(f"\n 结果已保存到: {filename}")
        return total_cost, total_carbon, total_distance

    def _calculate_route_distance(self, route: List[int]) -> float:
        """计算路径距离"""
        if not route:
            return 0

        distance = 0
        prev_idx = 0  # 配送中心索引

        # 获取 id_to_idx 映射
        id_to_idx = {}
```

```
            for i, cid in enumerate(self.customer_data['客户 ID'].values):
                id_to_idx[cid] = i

            for cid in route:
                cid_int = int(cid) if not isinstance(cid, int) else cid
                if cid_int in id_to_idx:
                    customer_idx = id_to_idx[cid_int] + 1  # +1 因为矩阵第一行是配送中心
                    distance += self.cost_calculator.distance_matrix[prev_idx, customer_idx]
                    prev_idx = customer_idx

            # 返回配送中心
            distance += self.cost_calculator.distance_matrix[prev_idx, 0]

            return distance

        def print_summary(self):
            """打印结果摘要"""
            total_cost, total_carbon, total_distance = self.save_to_file()

            print("\n" + "=" * 60)
            print("求解结果摘要")
            print("=" * 60)
            print(f"使用车辆数: {self.individual.n_vehicles}辆")
            print(f"总成本: {total_cost:.2f}元")
            print(f"总行驶距离: {total_distance:.2f}km")
            print(f"总碳排放: {total_carbon:.2f}kg CO2")
            print("=" * 60)

    # ============== 主函数 ==============
    def main():
        """主函数"""
        print("=" * 60)
        print("华中杯数学建模 A 题 - 问题 1")
        print("城市绿色物流配送调度优化")
        print("=" * 60)
```

```
start_time = time.time()


# 加载数据
print("\n[1/5] 加载数据...")
loader = DataLoader()
distance_matrix = loader.load_distance_matrix()
customer_data, coords, time_windows = loader.load_customer_data()


n_customers = len(customer_data)
print(f"客户数量: {n_customers}")
print(f"距离矩阵大小: {distance_matrix.shape}")


# 数据预处理
print("\n[2/5] 数据预处理...")
# 确保时间窗是 numpy 数组并转换为浮点数
if isinstance(time_windows, pd.DataFrame):
    # 转换时间字符串为浮点数（如 "11:33" -> 11.55）
    def time_to_float(time_str):
        try:
            if isinstance(time_str, str) and ':' in time_str:
                h, m = map(int, time_str.split(':'))
                return h + m / 60
            elif isinstance(time_str, (int, float)):
                return float(time_str)
            else:
                return 8.0  # 默认开始时间
        except:
            return 8.0  # 错误处理


    # 应用时间转换
    time_windows = time_windows.values
    converted_time_windows = []
    for row in time_windows:
        if len(row) >= 2:
            start = time_to_float(row[1])
            end = time_to_float(row[2])
```

```
                converted_time_windows.append([start, end])
            else:
                converted_time_windows.append([8.0, 18.0])  # 默认时间窗
        time_windows = np.array(converted_time_windows)

    # 确保客户 ID 是整数
    # 使用正确的客户 ID 列名
    customer_col = '客户 ID' if '客户 ID' in customer_data.columns else \
                   '目标客户编号' if '目标客户编号' in customer_data.columns else \
                   '客户编号' if '客户编号' in customer_data.columns else customer_data.columns[0]
    customer_data['客户 ID'] = customer_data[customer_col].astype(int)

    # 创建距离矩阵索引映射
    # 假设距离矩阵第一行/列是配送中心
    # 客户 ID 从 1-98 对应索引 1-98
    print("\n[3/5] 初始化遗传算法...")
    ga = GeneticAlgorithm(
        distance_matrix=distance_matrix,
        customer_data=customer_data,
        time_windows=time_windows,
        vehicle_type='燃油车 1',
        coords=coords
    )

    # 运行混合优化
    print("\n[4/5] 运行混合优化框架...")
    best_individual = ga.hybrid_optimization()

    # 输出结果
    print("\n[5/5] 保存结果...")
    cost_calculator = CostCalculator(
        distance_matrix, customer_data, time_windows, '燃油车 1'
    )
    writer = ResultWriter(best_individual, cost_calculator, customer_data, time_windows)
    writer.print_summary()
```

```
    elapsed_time = time.time() - start_time
    print(f"\n 总求解时间: {elapsed_time:.2f}秒")

    return best_individual, ga

def main_with_better_init():
    """使用改进初始种群的优化版本"""
    print("=" * 60)
    print("华中杯数学建模 A 题 - 问题 1（改进初始种群版）")
    print("城市绿色物流配送调度优化")
    print("=" * 60)

    start_time = time.time()

    print("\n[1/6] 加载数据...")
    loader = DataLoader()
    distance_matrix = loader.load_distance_matrix()
    customer_data, coords, time_windows = loader.load_customer_data()

    n_customers = len(customer_data)
    print(f"客户数量: {n_customers}")
    print(f"距离矩阵大小: {distance_matrix.shape}")

    print("\n[2/6] 数据预处理...")
    if isinstance(time_windows, pd.DataFrame):
        def time_to_float(time_str):
            try:
                if isinstance(time_str, str) and ':' in time_str:
                    h, m = map(int, time_str.split(':'))
                    return h + m / 60
                elif isinstance(time_str, (int, float)):
                    return float(time_str)
                else:
                    return 8.0
            except:
                return 8.0
```

```
    time_windows = time_windows.values
    converted_time_windows = []
    for row in time_windows:
        if len(row) >= 2:
            start = time_to_float(row[1])
            end = time_to_float(row[2])
            converted_time_windows.append([start, end])
        else:
            converted_time_windows.append([8.0, 18.0])
    time_windows = np.array(converted_time_windows)

customer_col = '客户 ID' if '客户 ID' in customer_data.columns else \
               '目标客户编号' if '目标客户编号' in customer_data.columns else \
               '客户编号' if '客户编号' in customer_data.columns else customer_data.columns[0]
customer_data['客户 ID'] = customer_data[customer_col].astype(int)

print("\n[3/6] 初始化遗传算法（传入坐标数据用于聚类）...")
ga = GeneticAlgorithm(
    distance_matrix=distance_matrix,
    customer_data=customer_data,
    time_windows=time_windows,
    vehicle_type='燃油车 1',
    coords=coords
)

print("\n[4/6] 运行验证测试（忽略时间窗）...")
test_result = ga.test_without_time_windows()

print("\n[5/6] 使用改进初始种群运行遗传算法优化...")
better_pop = ga.create_better_initial_population()
ga.population = better_pop
ga.evaluate_population(better_pop)

best = min(better_pop, key=lambda x: x.cost)
ga.best_cost = best.cost
```

```
ga.best_individual = Individual(best.route_plan.copy())
ga.history.append(ga.best_cost)

print(f"改进初始种群最优成本: {ga.best_cost:.2f}元")
print(f"改进初始种群车辆数: {len(best.route_plan)}")

print("\n[6/6] 继续迭代优化...")
for gen in range(ga.max_generations):
    if gen % 50 == 0 or gen < 5:
        current_best = min(ga.population, key=lambda x: x.cost)
        vehicle_count = len([r for r in current_best.route_plan if r])
        print(f"第{gen}代: 最优成本={current_best.cost:.2f}元, 车辆数={vehicle_count}")

    selected = ga.selection(ga.population)
    offspring = []
    for i in range(0, len(selected) - 1, 2):
        child1, child2 = ga.crossover(selected[i], selected[i+1])
        offspring.extend([child1, child2])
    offspring = [ga.mutate(ind) for ind in offspring]
    ga.population.extend(offspring)
    ga.evaluate_population(ga.population)
    elites = ga.elitism(ga.population)
    for elite in elites:
        ga.local_search_2opt(elite)
        cost, carbon, _ = ga.cost_calculator.calculate_cost(
            elite, ga.weights, ga.volumes, ga.customer_ids
        )
        elite.cost = cost
        elite.carbon = carbon

    next_gen = ga.selection(ga.population)
    next_gen = next_gen[:ga.population_size - len(elites)]
    next_gen.extend(elites)
    ga.population = next_gen

    current_best = min(ga.population, key=lambda x: x.cost)
```

```
            if current_best.cost < ga.best_cost:
                ga.best_cost = current_best.cost
                ga.best_individual = Individual(current_best.route_plan.copy())
                print(f"第{gen+1}代: 发现更优解 {ga.best_cost:.2f}元")

            ga.history.append(ga.best_cost)

            if gen >= ga.patience:
                recent_costs = ga.history[-ga.patience:]
                max_recent = max(recent_costs)
                if max_recent - ga.best_cost < ga.min_improvement:
                    print(f"第{gen+1}代: 早停收敛 (最佳成本: {ga.best_cost:.2f}元)")
                    break

        print("\n 保存结果...")
        cost_calculator = CostCalculator(
            distance_matrix, customer_data, time_windows, '燃油车 1'
        )
        writer = ResultWriter(ga.best_individual, cost_calculator, customer_data, time_windows)
        writer.print_summary()

        elapsed_time = time.time() - start_time
        print(f"\n 总求解时间: {elapsed_time:.2f}秒")

        return ga.best_individual, ga

    def test_simplified_problem():
        """简化问题测试：忽略时间窗，只测试容量和距离优化"""
        print("=" * 70)
        print("【简化问题测试】忽略时间窗约束，只测试容量和距离优化")
        print("=" * 70)

        start_time = time.time()

        print("\n[1/4] 加载数据...")
        loader = DataLoader()
```

```
distance_matrix = loader.load_distance_matrix()

customer_data, coords, time_windows = loader.load_customer_data()



n_customers = len(customer_data)

print(f"客户数量: {n_customers}")



customer_col = '客户 ID' if '客户 ID' in customer_data.columns else \

                '目标客户编号' if '目标客户编号' in customer_data.columns else \

                '客户编号' if '客户编号' in customer_data.columns else customer_data.columns[0]

customer_data['客户 ID'] = customer_data[customer_col].astype(int)



weight_col = '总重量' if '总重量' in customer_data.columns else '重量'

volume_col = '总体积' if '总体积' in customer_data.columns else '体积'



customer_ids = customer_data['客户 ID'].values

weights = customer_data[weight_col].values

volumes = customer_data[volume_col].values



total_demand = sum(weights)

total_volume = sum(volumes)

vehicle_capacity = 3000   # 燃油车 1 载重

vehicle_volume = 15         # 燃油车 1 体积



theoretical_min_vehicles = max(

    int(np.ceil(total_demand / vehicle_capacity)),

    int(np.ceil(total_volume / vehicle_volume))

)

print(f"\n 理论最小车辆数（仅容量约束）: {theoretical_min_vehicles}")

print(f"总货物重量: {total_demand:.2f}kg")

print(f"总货物体积: {total_volume:.2f}m³ ")



print("\n[2/4] 初始化种群（简化评估：无时间窗）...")



class SimplifiedCostCalculator:

    def __init__(self, distance_matrix):

        self.distance_matrix = distance_matrix
```

```
def evaluate(self, individual: Individual, weights, volumes, customer_ids, id_to_idx) -> Tuple[float, int]:
    from collections import Counter
    total_cost = 0
    num_vehicles = 0

    route_plan = individual.route_plan
    all_customers = []
    for route in route_plan:
        all_customers.extend(route)

    visit_count = Counter(all_customers)

    for route in route_plan:
        if not route:
            continue
        num_vehicles += 1

        route_cost = 400
        load_w = 0
        load_v = 0

        prev_idx = 0
        for c in route:
            if c not in id_to_idx:
                continue
            idx = id_to_idx[c]
            count = visit_count.get(c, 1)
            load_w += weights[idx] / count
            load_v += volumes[idx] / count

            customer_idx = c
            dist = self.distance_matrix[prev_idx, customer_idx]
            route_cost += dist * 0.8
            prev_idx = customer_idx
```

```
                route_cost += self.distance_matrix[prev_idx, 0] * 0.8

                overload_penalty = 0
                if load_w > vehicle_capacity:
                    overload_penalty += (load_w - vehicle_capacity) * 100
                if load_v > vehicle_volume:
                    overload_penalty += (load_v - vehicle_volume) * 200
                route_cost += overload_penalty

                total_cost += route_cost

        return total_cost, num_vehicles

cost_calc = SimplifiedCostCalculator(distance_matrix)
id_to_idx = {cid: i for i, cid in enumerate(customer_ids)}

def create_initial_population_simple(n: int) -> List[Individual]:
    import math
    population = []
    for _ in range(n):
        customers = list(customer_ids)
        random.shuffle(customers)

        big_customers = []
        small_customers = []

        for c in customers:
            if c not in id_to_idx:
                continue
            idx = id_to_idx[c]
            w, v = weights[idx], volumes[idx]

            if w > vehicle_capacity * 0.7 or v > vehicle_volume * 0.7:
                big_customers.append((c, w, v))
            else:
                small_customers.append((c, w, v))
```

```
        big_customers.sort(key=lambda x: x[1] + x[2], reverse=True)
        small_customers.sort(key=lambda x: x[1] + x[2], reverse=True)

        routes = []

        for c, w, v in big_customers:
            num_w = math.ceil(w / (vehicle_capacity * 0.95)) if vehicle_capacity > 0 else 1
            num_v = math.ceil(v / (vehicle_volume * 0.95)) if vehicle_volume > 0 else 1
            num_vehicles = max(num_w, num_v)

            for _ in range(num_vehicles):
                routes.append([c])

        current_route = []
        current_w = 0
        current_v = 0

        for c, w, v in small_customers:
            if current_w + w <= vehicle_capacity * 0.95 and current_v + v <= vehicle_volume * 0.95:
                current_route.append(c)
                current_w += w
                current_v += v
            else:
                if current_route:
                    routes.append(current_route)
                current_route = [c]
                current_w = w
                current_v = v

        if current_route:
            routes.append(current_route)

        population.append(Individual(routes))
    return population
```

```
population = create_initial_population_simple(100)


print("\n[3/4] 运行遗传算法（无时间窗）...")


POP_SIZE = 100

CROSSOVER_RATE = 0.8

MUTATION_RATE = 0.3

MAX_GEN = 300


best_individual = None

best_cost = float('inf')

history = []


for ind in population:

    cost, num_v = cost_calc.evaluate(ind, weights, volumes, customer_ids, id_to_idx)

    ind.cost = cost

    ind.num_vehicles = num_v

    if cost > 0:

        ind.fitness = 1.0 / cost

    else:

        ind.fitness = 1e-10


best = min(population, key=lambda x: x.cost)

best_cost = best.cost

best_individual = Individual(best.route_plan.copy())


print(f"\n 初始最佳: 车辆数={best.num_vehicles}, 成本={best.cost:.2f}")


for gen in range(MAX_GEN):

    selected = []

    for _ in range(POP_SIZE):

        tournament = random.sample(population, 3)

        winner = max(tournament, key=lambda x: x.fitness)

        selected.append(winner)


    offspring = []
```

```
for i in range(0, len(selected) - 1, 2):
    if random.random() < CROSSOVER_RATE:
        p1, p2 = selected[i], selected[i+1]
        child_routes = []

        all_c1 = []
        for r in p1.route_plan:
            all_c1.extend(r)
        all_c2 = []
        for r in p2.route_plan:
            all_c2.extend(r)

        if len(all_c1) >= 2 and len(all_c2) >= 2:
            pt1 = random.randint(0, len(all_c1) - 1)
            pt2 = random.randint(pt1, len(all_c1) - 1)
            seg = set(all_c1[pt1:pt2+1])

            remaining = [c for c in all_c2 if c not in seg]

            child_c = remaining[:pt1] + all_c1[pt1:pt2+1] + remaining[pt1:]

            routes = []
            current = []
            current_w = 0
            current_v = 0
            for c in child_c:
                if c not in id_to_idx:
                    continue
                idx = id_to_idx[c]
                w, v = weights[idx], volumes[idx]

                if current_w + w <= vehicle_capacity * 0.95 and current_v + v <= vehicle_volume * 0.95:
                    current.append(c)
                    current_w += w
                    current_v += v
                else:
```

```
                    if current:
                        routes.append(current)
                    current = [c]
                    current_w = w
                    current_v = v
            if current:
                routes.append(current)

            child = Individual(routes)
        else:
            child = Individual(p1.route_plan.copy())
    else:
        child = Individual(selected[i].route_plan.copy())

    if random.random() < MUTATION_RATE:
        mutation_type = random.choice(['2opt', 'swap', 'move', 'merge'])

        if mutation_type == '2opt' and len(child.route_plan) > 0:
            r_idx = random.randint(0, len(child.route_plan) - 1)
            route = child.route_plan[r_idx]
            if len(route) >= 2:
                i, j = sorted(random.sample(range(len(route)), 2))
                route[i:j+1] = reversed(route[i:j+1])

        elif mutation_type == 'swap' and len(child.route_plan) >= 2:
            r1, r2 = random.sample(range(len(child.route_plan)), 2)
            if child.route_plan[r1] and child.route_plan[r2]:
                c1 = random.choice(child.route_plan[r1])
                c2 = random.choice(child.route_plan[r2])
                i1 = child.route_plan[r1].index(c1)
                i2 = child.route_plan[r2].index(c2)
                child.route_plan[r1][i1], child.route_plan[r2][i2] = c2, c1

        elif mutation_type == 'move' and len(child.route_plan) >= 2:
            valid = [i for i, r in enumerate(child.route_plan) if len(r) >= 2]
            if valid:
```

```
                    r_idx = random.choice(valid)
                    route = child.route_plan[r_idx]
                    c_idx = random.randint(0, len(route) - 1)
                    customer = route.pop(c_idx)
                    targets = [i for i in range(len(child.route_plan)) if i != r_idx]
                    if targets:
                        t = random.choice(targets)
                        pos = random.randint(0, len(child.route_plan[t]))
                        child.route_plan[t].insert(pos, customer)

            elif mutation_type == 'merge' and len(child.route_plan) >= 2:
                r1, r2 = random.sample(range(len(child.route_plan)), 2)
                if child.route_plan[r1] and child.route_plan[r2]:
                    w1 = sum(weights[id_to_idx[c]] for c in child.route_plan[r1] if c in id_to_idx)
                    v1 = sum(volumes[id_to_idx[c]] for c in child.route_plan[r1] if c in id_to_idx)
                    w2 = sum(weights[id_to_idx[c]] for c in child.route_plan[r2] if c in id_to_idx)
                    v2 = sum(volumes[id_to_idx[c]] for c in child.route_plan[r2] if c in id_to_idx)
                    if w1 + w2 <= vehicle_capacity * 0.95 and v1 + v2 <= vehicle_volume * 0.95:
                        child.route_plan[r1].extend(child.route_plan[r2])
                        child.route_plan[r2] = []

            child.route_plan = [r for r in child.route_plan if r]

        cost, num_v = cost_calc.evaluate(child, weights, volumes, customer_ids, id_to_idx)
        child.cost = cost
        child.num_vehicles = num_v
        if cost > 0:
            child.fitness = 1.0 / cost
        else:
            child.fitness = 1e-10
        offspring.append(child)

    population.extend(offspring)

    elites = sorted(population, key=lambda x: x.cost)[:10]
    population = sorted(population, key=lambda x: x.fitness, reverse=True)[:POP_SIZE]
```

```
        current_best = min(population, key=lambda x: x.cost)
        if current_best.cost < best_cost:
            best_cost = current_best.cost
            best_individual = Individual(current_best.route_plan.copy())

        history.append(best_cost)

        if gen % 50 == 0 or gen < 5:
            print(f"第{gen}代: 最佳车辆数={current_best.num_vehicles}, 成本={current_best.cost:.2f}")

print("\n[4/4] 分析结果...")

def emergency_merge_routes(best_individual, target_vehicles=35):
    """紧急合并，将车辆数降到目标数量（区分大小客户）"""
    current_vehicles = len([r for r in best_individual.route_plan if r])
    print(f"\n【强制合并策略】尝试将{current_vehicles}辆车合并到{target_vehicles}辆...")

    def get_route_weight(route):
        """计算路线总需求（考虑拆分）"""
        from collections import Counter
        all_customers = []
        for r in route:
            all_customers.extend(r)
        visits = Counter()
        for c in all_customers:
            visits[c] += 1

        total_w = 0
        total_v = 0
        for c, count in visits.items():
            if c in id_to_idx:
                total_w += weights[id_to_idx[c]] / count
                total_v += volumes[id_to_idx[c]] / count
        return total_w, total_v
```

```
def check_capacity_with_split(route):
    """检查路线是否满足容量约束（允许拆分大客户）"""
    total_w, total_v = get_route_weight([route])
    return total_w <= vehicle_capacity * 1.1 and total_v <= vehicle_volume * 1.1


def is_big_customer(customer_id):
    """判断是否为单辆无法装载的大客户"""
    if customer_id in id_to_idx:
        w = weights[id_to_idx[customer_id]]
        v = volumes[id_to_idx[customer_id]]
        return w > vehicle_capacity * 0.7 or v > vehicle_volume * 0.7
    return False


def calculate_route_cost(route):
    """计算路线成本"""
    if not route:
        return 0
    cost = 400
    prev_idx = 0
    for c in route:
        if c in id_to_idx:
            dist = distance_matrix[prev_idx, c]
            cost += dist * 0.8
            prev_idx = c
    cost += distance_matrix[prev_idx, 0] * 0.8
    return cost


def two_opt(route):
    """2-opt 优化"""
    if len(route) < 2:
        return route
    improved = True
    while improved:
        improved = False
        for i in range(len(route) - 1):
            for j in range(i + 2, len(route)):
```

```
                    if j == len(route) - 1 and i == 0:
                        continue
                    new_route = route[:i+1] + route[i+1:j+1][::-1] + route[j+1:]
                    if calculate_route_cost(new_route) < calculate_route_cost(route):
                        route = new_route
                        improved = True
    return route

current_routes = [r.copy() for r in best_individual.route_plan]

big_routes = [r for r in current_routes if len(r) == 1 and is_big_customer(r[0])]
small_routes = [r for r in current_routes if not (len(r) == 1 and is_big_customer(r[0]))]

print(f"  大客户专用路线: {len(big_routes)}条 (无法合并)")
print(f"  可合并路线: {len(small_routes)}条")

sorted_small = sorted(small_routes, key=len)

merge_count = 0
while len(sorted_small) > target_vehicles - len(big_routes):
    merged = False
    for i in range(len(sorted_small) - 1):
        combined = sorted_small[i] + sorted_small[i + 1]
        if check_capacity_with_split(combined):
            combined = two_opt(combined)
            sorted_small[i] = combined
            sorted_small.pop(i + 1)
            merged = True
            merge_count += 1
            print(f"  合并成功 #{merge_count}: 合并为{len(combined)}个客户")
            break
    if not merged:
        print(f"  无法继续合并: 没有找到满足容量约束的配对")
        break

final_routes = big_routes + [r for r in sorted_small if r]
```

```
    if merge_count > 0:
        new_routes_plan = Individual(final_routes)
        new_cost, new_num_v = cost_calc.evaluate(
            new_routes_plan, weights, volumes, customer_ids, id_to_idx
        )
        print(f"\n  合并结果: {current_vehicles} → {len(final_routes)}辆车, 成本={new_cost:.2f}元")
        best_individual.route_plan = final_routes
        best_individual.cost = new_cost
        best_individual.num_vehicles = len(final_routes)

    return best_individual

best_individual = emergency_merge_routes(best_individual, target_vehicles=35)

print("\n" + "=" * 70)
print("【简化问题测试结果】")
print("=" * 70)

final_vehicles = len(best_individual.route_plan)
avg_customers_per_vehicle = n_customers / final_vehicles if final_vehicles > 0 else 0

max_single_customer_route = 0
multi_customer_routes = 0
for route in best_individual.route_plan:
    if len(route) == 1:
        max_single_customer_route += 1
    elif len(route) >= 2:
        multi_customer_routes += 1

print(f"\n 最终车辆数: {final_vehicles}")
print(f"理论最小车辆数: {theoretical_min_vehicles}")
print(f"平均每车客户数: {avg_customers_per_vehicle:.2f}")

print(f"\n 单客户路线数: {max_single_customer_route}")
print(f"多客户拼单路线数: {multi_customer_routes}")
```

```
    print(f"\n 最终成本: {best_cost:.2f}元")

    print("\n" + "-" * 70)
    print("【结论】")
    print("-" * 70)

    if final_vehicles <= 50:
        print(f"✓ 车辆数降到 50 以下: {final_vehicles}辆")
    else:
        print(f"✗ 车辆数未能降到 50 以下: {final_vehicles}辆 (理论最小: {theoretical_min_vehicles})")

    if multi_customer_routes > 0:
        print(f"✓ 实现了多客户拼单: {multi_customer_routes}条路线包含 2+客户")
        print(f"   拼单率: {multi_customer_routes/len(best_individual.route_plan)*100:.1f}%")
    else:
        print(f"✗ 没有实现多客户拼单，所有路线都是单车")

    if final_vehicles > theoretical_min_vehicles * 1.5:
        print(f"⚠ 车辆数偏高(>{theoretical_min_vehicles*1.5})，可能存在优化空间")
    elif final_vehicles <= theoretical_min_vehicles * 1.2:
        print(f"✓ 接近理论最小值({theoretical_min_vehicles})，优化效果良好")

    print("\n" + "=" * 70)

    elapsed_time = time.time() - start_time
    print(f"\n 测试用时: {elapsed_time:.2f}秒")

    return best_individual

def test_individual_modules():
    """单独测试每个模块"""
    print("=" * 70)
    print("【模块独立测试】")
    print("=" * 70)
```

```
print("\n[1/5] 加载数据...")

loader = DataLoader()

distance_matrix = loader.load_distance_matrix()

customer_data, coords, time_windows = loader.load_customer_data()



customer_col = '客户 ID' if '客户 ID' in customer_data.columns else \

                '目标客户编号' if '目标客户编号' in customer_data.columns else \

                '客户编号' if '客户编号' in customer_data.columns else customer_data.columns[0]

customer_data['客户 ID'] = customer_data[customer_col].astype(int)



weight_col = '总重量' if '总重量' in customer_data.columns else '重量'

volume_col = '总体积' if '总体积' in customer_data.columns else '体积'



customer_ids = customer_data['客户 ID'].values

weights = customer_data[weight_col].values

volumes = customer_data[volume_col].values



id_to_idx = {cid: i for i, cid in enumerate(customer_ids)}



VEHICLE_CAPACITY = 3000 * 0.95

VEHICLE_VOLUME = 15 * 0.95



print(f"客户数量: {len(customer_ids)}")

print(f"车辆容量: {VEHICLE_CAPACITY:.0f}kg, {VEHICLE_VOLUME:.1f}m³ ")



print("\n" + "=" * 70)

print("【模块 1: 贪心装箱模块】")

print("=" * 70)



def greedy_bin_packing(customer_list: List[int]) -> List[List[int]]:

    """贪心装箱：正确处理大客户需求"""

    import math

    routes = []

    current_route = []

    current_weight = 0

    current_volume = 0
```

```
customers_with_demand = []
for c in customer_list:
    if c not in id_to_idx:
        continue
    idx = id_to_idx[c]
    w, v = weights[idx], volumes[idx]
    customers_with_demand.append({'id': c, 'weight': w, 'volume': v})

customers_sorted = sorted(customers_with_demand, key=lambda x: x['weight'] + x['volume'], reverse=True)

for customer in customers_sorted:
    c = customer['id']
    w = customer['weight']
    v = customer['volume']

    if w > VEHICLE_CAPACITY or v > VEHICLE_VOLUME:
        num_vehicles_w = math.ceil(w / VEHICLE_CAPACITY) if VEHICLE_CAPACITY > 0 else 1
        num_vehicles_v = math.ceil(v / VEHICLE_VOLUME) if VEHICLE_VOLUME > 0 else 1
        num_vehicles = max(num_vehicles_w, num_vehicles_v)

        for _ in range(num_vehicles):
            routes.append([c])
        continue

    if current_weight + w <= VEHICLE_CAPACITY and current_volume + v <= VEHICLE_VOLUME:
        current_route.append(c)
        current_weight += w
        current_volume += v
    else:
        if current_route:
            routes.append(current_route)
        current_route = [c]
        current_weight = w
        current_volume = v
```

```
        if current_route:
            routes.append(current_route)

        return routes

def greedy_bin_packing_v2(customer_list: List[int]) -> List[List[int]]:
    """贪心装箱 V2：大客户优先满载，小客户拼单填充"""
    import math

    big_customers = []
    small_customers = []

    for c in customer_list:
        if c not in id_to_idx:
            continue
        idx = id_to_idx[c]
        w, v = weights[idx], volumes[idx]

        if w > VEHICLE_CAPACITY or v > VEHICLE_VOLUME:
            big_customers.append({'id': c, 'weight': w, 'volume': v})
        else:
            small_customers.append({'id': c, 'weight': w, 'volume': v})

    big_customers.sort(key=lambda x: x['weight'] + x['volume'], reverse=True)
    small_customers.sort(key=lambda x: x['weight'] + x['volume'], reverse=True)

    routes = []

    for bc in big_customers:
        w, v = bc['weight'], bc['volume']
        num_vehicles_w = math.ceil(w / VEHICLE_CAPACITY) if VEHICLE_CAPACITY > 0 else 1
        num_vehicles_v = math.ceil(v / VEHICLE_VOLUME) if VEHICLE_VOLUME > 0 else 1
        num_vehicles = max(num_vehicles_w, num_vehicles_v)

        portion_w = w / num_vehicles
        portion_v = v / num_vehicles
```

```
            for _ in range(num_vehicles):
                routes.append([bc['id']])

    current_route = []
    current_weight = 0
    current_volume = 0

    for sc in small_customers:
        c, w, v = sc['id'], sc['weight'], sc['volume']

        if current_weight + w <= VEHICLE_CAPACITY and current_volume + v <= VEHICLE_VOLUME:
            current_route.append(c)
            current_weight += w
            current_volume += v
        else:
            if current_route:
                routes.append(current_route)
            current_route = [c]
            current_weight = w
            current_volume = v

    if current_route:
        routes.append(current_route)

    return routes

test_customers = list(customer_ids)[:20]
print(f"\n 输入: 前{len(test_customers)}个客户 {test_customers}")

result_routes = greedy_bin_packing_v2(test_customers)

print(f"\n 输出: {len(result_routes)}条路径")

import math
from collections import Counter
```

```
total_w = 0
total_v = 0
unique_customers = set()
over_capacity_count = 0

all_customer_ids = []
for route in result_routes:
    all_customer_ids.extend(route)
customer_visit_count = Counter(all_customer_ids)

for i, route in enumerate(result_routes):
    route_w = 0
    route_v = 0
    route_customers = set()
    for c in route:
        if c in id_to_idx:
            idx = id_to_idx[c]
            w = weights[idx]
            v = volumes[idx]
            count = customer_visit_count.get(c, 1)
            actual_w = w / count
            actual_v = v / count
            route_w += actual_w
            route_v += actual_v
            route_customers.add(c)

    for c in route_customers:
        unique_customers.add(c)

    is_overload = route_w > VEHICLE_CAPACITY or route_v > VEHICLE_VOLUME
    if is_overload:
        over_capacity_count += 1

    total_w += route_w
    total_v += route_v
```

```
        status = "超载!" if is_overload else "OK"
        print(f"   路径{i+1}: {len(route)}客户 {route} [{status}]")
        print(f"              载重: {route_w:.1f}kg/{VEHICLE_CAPACITY:.0f}kg ({route_w/VEHICLE_CAPACITY*100:.1f}%)")
        print(f"              体积: {route_v:.1f}m³/{VEHICLE_VOLUME:.1f}m³ ({route_v/VEHICLE_VOLUME*100:.1f}%)")

print(f"\n 总载重: {total_w:.1f}kg, 总体积: {total_v:.1f}m³")
print(f"服务客户数: {len(unique_customers)}/{len(test_customers)}")
print(f"车辆数: {len(result_routes)}")

if len(unique_customers) == len(test_customers):
    print("✓ 贪心装箱模块: 所有客户都被分配")
else:
    missing = set(test_customers) - unique_customers
    print(f"✗ 贪心装箱模块: 缺少{len(missing)}个客户: {missing}")

if over_capacity_count == 0:
    print("✓ 贪心装箱模块: 所有路径都满足容量约束")
else:
    print(f"✗ 贪心装箱模块: {over_capacity_count}条路径超载")

print("\n" + "=" * 70)
print("【模块 2: 2-opt 路径优化模块】")
print("=" * 70)

def calculate_route_distance(route: List[int]) -> float:
    """计算路径总距离"""
    if not route:
        return 0
    total_dist = 0
    prev_idx = 0
    for c in route:
        if c in id_to_idx:
            customer_idx = id_to_idx[c] + 1
            total_dist += distance_matrix[prev_idx, customer_idx]
            prev_idx = customer_idx
    total_dist += distance_matrix[prev_idx, 0]
```

```
    return total_dist


def two_opt(route: List[int]) -> List[int]:
    """2-opt 优化"""
    if len(route) < 2:
        return route
    best_route = route.copy()
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best_route) - 1):
            for j in range(i + 1, len(best_route)):
                new_route = best_route.copy()
                new_route[i:j+1] = reversed(new_route[i:j+1])
                if calculate_route_distance(new_route) < calculate_route_distance(best_route):
                    best_route = new_route
                    improved = True
    return best_route


test_route = result_routes[0] if result_routes else [1, 2, 3]
print(f"\n 输入路径: {test_route}")
print(f"原始距离: {calculate_route_distance(test_route):.2f}km")


if len(test_route) >= 2:
    original_dist = calculate_route_distance(test_route)
    optimized_route = two_opt(test_route)
    optimized_dist = calculate_route_distance(optimized_route)


    print(f"优化后路径: {optimized_route}")
    print(f"优化后距离: {optimized_dist:.2f}km")


    improvement = (original_dist - optimized_dist) / original_dist * 100 if original_dist > 0 else 0
    print(f"距离优化: {improvement:.1f}%")


    if optimized_dist <= original_dist:
        print("✓ 2-opt 模块:  正常工作")
```

```
        else:
            print("✗ 2-opt 模块：优化后距离反而增加！")
else:
    print("路径客户数不足，跳过 2-opt 测试")

print("\n" + "=" * 70)
print("【模块 3：成本计算模块】")
print("=" * 70)

def calculate_route_cost(route: List[int]) -> Tuple[float, float, float]:
    """计算单条路径成本：(总成本，运输成本，启动成本)"""
    if not route:
        return 0, 0, 0

    start_cost = 400
    transport_cost = 0
    prev_idx = 0

    load_ratio = min(1.0, sum(weights[id_to_idx[c]] for c in route if c in id_to_idx) / 3000)
    cost_factor = 0.8 * (0.5 + 0.5 * load_ratio)

    for c in route:
        if c in id_to_idx:
            customer_idx = id_to_idx[c] + 1
            dist = distance_matrix[prev_idx, customer_idx]
            transport_cost += dist * cost_factor
            prev_idx = customer_idx

    transport_cost += distance_matrix[prev_idx, 0] * cost_factor

    total_cost = start_cost + transport_cost
    return total_cost, transport_cost, start_cost

test_routes = result_routes[:5] if result_routes else [[1], [2]]
print(f"\n 计算{len(test_routes)}条路径的成本:")
```

```
total_cost_all = 0
for i, route in enumerate(test_routes):
    cost, transport, start = calculate_route_cost(route)
    total_cost_all += cost
    dist = calculate_route_distance(route)
    print(f"  路径{i+1}: 成本={cost:.2f}元 (启动={start:.2f}, 运输={transport:.2f}), 距离={dist:.2f}km")

print(f"\n 总成本: {total_cost_all:.2f}元")
print(f"平均每条路径成本: {total_cost_all/len(test_routes):.2f}元")

if total_cost_all > 0:
    print("✓ 成本计算模块: 正常工作")
else:
    print("✗ 成本计算模块: 成本为 0，可能有问题")

print("\n" + "=" * 70)
print("【模块 4: 路径合并模块】")
print("=" * 70)

def get_route_weight(route, visit_count):
    w = 0
    v = 0
    for c in route:
        if c in id_to_idx:
            total_w = weights[id_to_idx[c]]
            total_v = volumes[id_to_idx[c]]
            count = visit_count.get(c, 1)
            w += total_w / count
            v += total_v / count
    return w, v

def can_merge(route1, route2, visit_count):
    w1, v1 = get_route_weight(route1, visit_count)
    w2, v2 = get_route_weight(route2, visit_count)
    return (w1 + w2 <= VEHICLE_CAPACITY) and (v1 + v2 <= VEHICLE_VOLUME)
```

```
if len(result_routes) >= 2:
    r1, r2 = result_routes[0], result_routes[1]
    w1, v1 = get_route_weight(r1, customer_visit_count)
    w2, v2 = get_route_weight(r2, customer_visit_count)

    print(f"\n 路径 1: {len(r1)}客户，载重={w1:.1f}kg，体积={v1:.1f}m³ ")
    print(f"路径 2: {len(r2)}客户，载重={w2:.1f}kg，体积={v2:.1f}m³ ")

    can_merge_result = can_merge(r1, r2, customer_visit_count)
    print(f"合并后: 载重={w1+w2:.1f}kg，体积={v1+v2:.1f}m³ ")
    print(f"可合并: {'是' if can_merge_result else '否'}")

    if can_merge_result:
        merged = r1 + r2
        merged_dist_before = calculate_route_distance(r1) + calculate_route_distance(r2)
        merged_cost_before = sum(calculate_route_cost(r)[0] for r in [r1, r2])

        merged_dist_after = calculate_route_distance(merged)
        merged_cost_after = calculate_route_cost(merged)[0]

        print(f"\n 合并前: 距离={merged_dist_before:.2f}km，成本={merged_cost_before:.2f}元，车辆=2")
        print(f"合并后: 距离={merged_dist_after:.2f}km，成本={merged_cost_after:.2f}元，车辆=1")
        print(f"节省: 距离减少={merged_dist_before-merged_dist_after:.2f}km，成本减少={merged_cost_before-merged_cost_after:.2f}元")

        if merged_cost_after < merged_cost_before:
            print("✓ 路径合并: 可以减少成本")
        else:
            print("✗ 路径合并: 成本反而增加")
    else:
        print("当前两条路径无法合并（超载）")
else:
    print("路径数不足，无法测试合并")

print("\n" + "=" * 70)
print("【模块 5: 全流程集成测试】")
print("=" * 70)
```

```
print("\n 执行: 贪心装箱 -> 2-opt 优化 -> 成本计算")

all_customers = list(customer_ids)
random.shuffle(all_customers)

print(f"输入: {len(all_customers)}个客户")

routes = greedy_bin_packing_v2(all_customers)
print(f"贪心装箱: {len(routes)}条路径")

optimized_routes = []
total_dist_before = 0
total_dist_after = 0
for route in routes:
    total_dist_before += calculate_route_distance(route)
    optimized = two_opt(route)
    optimized_routes.append(optimized)
    total_dist_after += calculate_route_distance(optimized)

print(f"2-opt 优化: 距离从{total_dist_before:.2f}km 降到{total_dist_after:.2f}km ({100*(total_dist_before-total_dist_after)/total_dist_before:.1f}%优化)")

total_cost = sum(calculate_route_cost(r)[0] for r in optimized_routes)
print(f"总成本: {total_cost:.2f}元")

multi_customer = sum(1 for r in optimized_routes if len(r) >= 2)
print(f"多客户拼单: {multi_customer}条路线 ({100*multi_customer/len(optimized_routes):.1f}%)")

print("\n" + "=" * 70)
print("【测试结论】")
print("=" * 70)
print("1. 贪心装箱模块: 检查是否有效分配客户到各车")
print("2. 2-opt 模块: 检查是否优化路径顺序")
print("3. 成本计算模块: 检查成本计算是否正确")
print("4. 路径合并模块: 检查是否能在容量约束下合并路线")
print("5. 全流程集成: 检查整体流程是否正常工作")
```

```
    return {
        'routes': result_routes,
        'optimized_routes': optimized_routes,
        'total_cost': total_cost,
        'num_vehicles': len(optimized_routes)
    }

def generate_large_customers(n: int, min_weight: float = 2000, min_volume: float = 10) -> List[int]:
    """
    生成大客户列表（需求超过车辆容量 30%的客户）

    Args:
        n: 生成的大客户数量
        min_weight: 最小重量需求（kg）
        min_volume: 最小体积需求（m³）

    Returns:
        大客户 ID 列表
    """
    large_customers = []
    np.random.seed(int(time.time()) % 1000)

    for i in range(n):
        customer_id = random.randint(1, 88)
        if customer_id not in large_customers:
            large_customers.append(customer_id)

    return large_customers

def run_algorithm_with_config(test_config: Dict) -> Dict:
    """
    根据配置运行算法

    Args:
        test_config: 测试配置字典
```

```
Returns:
    运行结果字典
"""
print(f"\n{'='*60}")
print(f"运行配置: {test_config.get('name', '未命名测试')}")
print(f"{'='*60}")

loader = DataLoader()
distance_matrix = loader.load_distance_matrix()
customer_data, coords, time_windows = loader.load_customer_data()

n_customers = len(customer_data)

if isinstance(time_windows, pd.DataFrame):
    def time_to_float(time_str):
        try:
            if isinstance(time_str, str) and ':' in time_str:
                h, m = map(int, time_str.split(':'))
                return h + m / 60
            elif isinstance(time_str, (int, float)):
                return float(time_str)
            else:
                return 8.0
        except:
            return 8.0

    time_windows = time_windows.values
    converted_time_windows = []
    for row in time_windows:
        if len(row) >= 2:
            start = time_to_float(row[1])
            end = time_to_float(row[2])
            converted_time_windows.append([start, end])
        else:
            converted_time_windows.append([8.0, 18.0])
```

```
        time_windows = np.array(converted_time_windows)

    customer_col = '客户 ID' if '客户 ID' in customer_data.columns else \
                   '目标客户编号' if '目标客户编号' in customer_data.columns else \
                   '客户编号' if '客户编号' in customer_data.columns else customer_data.columns[0]
    customer_data['客户 ID'] = customer_data[customer_col].astype(int)

    vehicle_type = test_config.get('vehicle_type', '燃油车 1')

    ga = GeneticAlgorithm(
        distance_matrix=distance_matrix,
        customer_data=customer_data,
        time_windows=time_windows,
        vehicle_type=vehicle_type,
        coords=coords
    )

    if test_config.get('use_hybrid', False):
        print("使用混合优化框架 (GA + SA + TS)")
        best_individual = ga.hybrid_optimization()
    else:
        print("使用标准遗传算法")
        best_individual = ga.run()

    result = {
        'name': test_config.get('name', '未命名测试'),
        'cost': best_individual.cost,
        'vehicles': len(best_individual.route_plan),
        'carbon': best_individual.carbon,
        'individual': best_individual,
        'ga': ga
    }

    return result

def validate_result(result: Dict) -> bool:
```

```
    """
    验证算法结果的正确性

    Args:
        result: 算法运行结果

    Returns:
        验证是否通过
    """
    print(f"\n{'='*60}")
    print(f"验证结果: {result['name']}")
    print(f"{'='*60}")

    ga = result['ga']
    individual = result['individual']

    print(f"总成本: {result['cost']:.2f}元")
    print(f"使用车辆数: {result['vehicles']}辆")
    print(f"总碳排放: {result['carbon']:.2f}kg")

    validation_passed = True

    print(f"\n 验证项目:")

    if result['cost'] <= 0:
        print(f"  ✗ 成本验证失败: 成本为{result['cost']:.2f}（应该 > 0）")
        validation_passed = False
    else:
        print(f"  ✓ 成本验证通过: {result['cost']:.2f}元")

    if result['vehicles'] <= 0:
        print(f"  ✗ 车辆数验证失败: 车辆数为{result['vehicles']}（应该 > 0）")
        validation_passed = False
    else:
        print(f"  ✓ 车辆数验证通过: {result['vehicles']}辆")
```

```
served_customers = individual.get_all_customers()
expected_customers = set(ga.customer_ids)
missing_customers = expected_customers - set(served_customers)

if missing_customers:
    print(f"  ✗ 客户覆盖验证失败: 缺少{len(missing_customers)}个客户")
    validation_passed = False
else:
    print(f"  ✓ 客户覆盖验证通过: 所有{len(served_customers)}个客户都被服务")

route_loads_valid = True
for i, route in enumerate(individual.route_plan):
    if not route:
        continue
    total_w = sum(ga.weights[ga.id_to_idx[c]] for c in route if c in ga.id_to_idx)
    total_v = sum(ga.volumes[ga.id_to_idx[c]] for c in route if c in ga.id_to_idx)

    if total_w > ga.max_weight * 1.05:
        print(f"  ✗ 路线{i+1}载重超限: {total_w:.2f}kg > {ga.max_weight:.2f}kg")
        route_loads_valid = False
    if total_v > ga.max_volume * 1.05:
        print(f"  ✗ 路线{i+1}体积超限: {total_v:.2f}m³ > {ga.max_volume:.2f}m³")
        route_loads_valid = False

if route_loads_valid:
    print(f"  ✓ 容量约束验证通过: 所有路线都满足载重和体积约束")
else:
    validation_passed = False

if np.isfinite(result['cost']):
    print(f"  ✓ 成本数值验证通过: 是有限值")
else:
    print(f"  ✗ 成本数值验证失败: 成本值不是有限数值")
    validation_passed = False

print(f"\n{'='*60}")
```

```
        if validation_passed:
            print(f"✓ 验证通过: {result['name']}")
        else:
            print(f"✗ 验证失败: {result['name']}")
        print(f"{'='*60}")

        return validation_passed

def robustness_testing():
    """
    系统鲁棒性测试

    测试算法在不同场景下的性能表现：
    1. 基础案例（88 个客户，标准参数）
    2. 大客户密集场景（50 个大客户）
    3. 严格时间窗场景（时间窗宽度缩小 50%）
    4. 新能源车不足场景（限制新能源车数量）
    """
    print("=" * 70)
    print("系统鲁棒性测试")
    print("=" * 70)

    test_cases = [
        {
            'name': '基础案例',
            'description': '88 个客户，标准参数',
            'use_hybrid': True,
            'vehicle_type': '燃油车 1'
        },
        {
            'name': '大客户密集',
            'description': '包含 50 个大客户（重量>2000kg 或体积>10m³）',
            'use_hybrid': True,
            'vehicle_type': '燃油车 1'
        },
        {
```

```
        'name': '严格时间窗',
        'description': '时间窗宽度缩小 50%，考验算法的时间窗处理能力',
        'use_hybrid': True,
        'vehicle_type': '燃油车 1'
    },
    {
        'name': '新能源车测试',
        'description': '使用新能源车，优化碳排放',
        'use_hybrid': True,
        'vehicle_type': '新能源 1'
    }
]

results = []

for i, case in enumerate(test_cases, 1):
    print(f"\n\n{'#'*70}")
    print(f"测试案例 {i}/{len(test_cases)}: {case['name']}")
    print(f"{'#'*70}")
    print(f"描述: {case.get('description', '')}")

    try:
        result = run_algorithm_with_config(case)
        validate_result(result)
        results.append({
            'case': case['name'],
            'result': result,
            'passed': True,
            'error': None
        })
    except Exception as e:
        print(f"\n✗ 测试执行失败: {str(e)}")
        import traceback
        traceback.print_exc()
        results.append({
            'case': case['name'],
```

```
                'result': None,
                'passed': False,
                'error': str(e)
            })

    print("\n\n" + "=" * 70)
    print("鲁棒性测试汇总报告")
    print("=" * 70)

    for res in results:
        status = "✓ 通过" if res['passed'] else "✗ 失败"
        print(f"\n{res['case']}: {status}")
        if res['passed']:
            print(f"  成本: {res['result']['cost']:.2f}元")
            print(f"  车辆数: {res['result']['vehicles']}辆")
            print(f"  碳排放: {res['result']['carbon']:.2f}kg")
        else:
            print(f"  错误: {res['error']}")

    passed_count = sum(1 for r in results if r['passed'])
    print(f"\n 通过率: {passed_count}/{len(results)} ({100*passed_count/len(results):.1f}%)")

    print("=" * 70)

    return results


def genetic_algorithm(distance_matrix: np.ndarray, customer_data: pd.DataFrame,
                      time_windows: np.ndarray, coords: pd.DataFrame = None,
                      max_generations: int = 300) -> Tuple[Individual, Dict]:
    """
    独立版本的遗传算法

    Args:
        distance_matrix: 距离矩阵
        customer_data: 客户数据
        time_windows: 时间窗
```

```
        coords: 客户坐标
        max_generations: 最大迭代代数

    Returns:
        (最优解, 性能指标字典)
    """
    ga = GeneticAlgorithm(
        distance_matrix=distance_matrix,
        customer_data=customer_data,
        time_windows=time_windows,
        vehicle_type='燃油车 1',
        coords=coords
    )

    ga.max_generations = max_generations

    start_time = time.time()
    best_individual = ga.run()
    elapsed_time = time.time() - start_time

    metrics = {
        'elapsed_time': elapsed_time,
        'generations': len(ga.history),
        'best_cost': ga.best_cost,
        'best_vehicles': len(best_individual.route_plan),
        'history': ga.history
    }

    return best_individual, metrics

def hybrid_ga_greedy(distance_matrix: np.ndarray, customer_data: pd.DataFrame,
                     time_windows: np.ndarray, coords: pd.DataFrame = None,
                     max_generations: int = 200) -> Tuple[Individual, Dict]:
    """
    遗传+贪心混合算法
```

```
先用贪心构造优质初始种群，再用遗传算法优化

Args:
    distance_matrix: 距离矩阵
    customer_data: 客户数据
    time_windows: 时间窗
    coords: 客户坐标
    max_generations: 最大迭代代数

Returns:
    (最优解, 性能指标字典)
"""
ga = GeneticAlgorithm(
    distance_matrix=distance_matrix,
    customer_data=customer_data,
    time_windows=time_windows,
    vehicle_type='燃油车 1',
    coords=coords
)

ga.max_generations = max_generations

start_time = time.time()

print("  [混合算法] 阶段 1: 贪心构造初始解...")
better_pop = ga.create_better_initial_population()
ga.population = better_pop
ga.evaluate_population(better_pop)

best = min(better_pop, key=lambda x: x.cost)
ga.best_cost = best.cost
ga.best_individual = Individual(best.route_plan.copy())
ga.history.append(ga.best_cost)

print(f"  [混合算法] 贪心初始解: 成本={ga.best_cost:.2f}元, 车辆数={len(best.route_plan)}")
```

```
print("   [混合算法] 阶段 2: 遗传算法优化...")

for gen in range(max_generations):

    selected = ga.selection(ga.population)

    offspring = []

    for i in range(0, len(selected) - 1, 2):

        child1, child2 = ga.crossover(selected[i], selected[i+1])

        offspring.extend([child1, child2])

    offspring = [ga.mutate(ind) for ind in offspring]

    ga.population.extend(offspring)

    ga.evaluate_population(ga.population, generation=gen+1)

    elites = ga.elitism(ga.population)

    next_gen = ga.selection(ga.population)

    next_gen = next_gen[:ga.population_size - len(elites)]

    next_gen.extend(elites)

    ga.population = next_gen


    current_best = min(ga.population, key=lambda x: x.cost)

    if current_best.cost < ga.best_cost:

        ga.best_cost = current_best.cost

        ga.best_individual = Individual(current_best.route_plan.copy())

        ga.history.append(ga.best_cost)


    if gen % 50 == 0:

        print(f"   [混合算法] 第{gen}代: 成本={ga.best_cost:.2f}元")


elapsed_time = time.time() - start_time


metrics = {

    'elapsed_time': elapsed_time,

    'generations': len(ga.history),

    'best_cost': ga.best_cost,

    'best_vehicles': len(ga.best_individual.route_plan),

    'history': ga.history

}


return ga.best_individual, metrics
```

```
def simulated_annealing_standalone(distance_matrix: np.ndarray, customer_data: pd.DataFrame,
                                   time_windows: np.ndarray, coords: pd.DataFrame = None,
                                   max_iterations: int = 1000) -> Tuple[Individual, Dict]:
    """
    独立版本的模拟退火算法

    Args:
        distance_matrix: 距离矩阵
        customer_data: 客户数据
        time_windows: 时间窗
        coords: 客户坐标
        max_iterations: 最大迭代次数

    Returns:
        (最优解, 性能指标字典)
    """
    ga = GeneticAlgorithm(
        distance_matrix=distance_matrix,
        customer_data=customer_data,
        time_windows=time_windows,
        vehicle_type='燃油车 1',
        coords=coords
    )

    initial_pop = ga.initialize_population()
    initial_best = min(initial_pop, key=lambda x: x.cost)

    start_time = time.time()
    best_individual = ga.simulated_annealing(
        initial_best,
        max_iterations=max_iterations,
        initial_temp=100.0,
        cooling_rate=0.995
    )
    elapsed_time = time.time() - start_time
```

```
    metrics = {
        'elapsed_time': elapsed_time,
        'iterations': max_iterations,
        'best_cost': best_individual.cost,
        'best_vehicles': len(best_individual.route_plan),
        'initial_cost': initial_best.cost
    }

    return best_individual, metrics


def tabu_search_standalone(distance_matrix: np.ndarray, customer_data: pd.DataFrame,
                           time_windows: np.ndarray, coords: pd.DataFrame = None,
                           max_iterations: int = 500) -> Tuple[Individual, Dict]:
    """
    独立版本的禁忌搜索算法

    Args:
        distance_matrix: 距离矩阵
        customer_data: 客户数据
        time_windows: 时间窗
        coords: 客户坐标
        max_iterations: 最大迭代次数

    Returns:
        (最优解, 性能指标字典)
    """
    ga = GeneticAlgorithm(
        distance_matrix=distance_matrix,
        customer_data=customer_data,
        time_windows=time_windows,
        vehicle_type='燃油车 1',
        coords=coords
    )

    initial_pop = ga.initialize_population()
```

```
    initial_best = min(initial_pop, key=lambda x: x.cost)

    start_time = time.time()
    best_individual = ga.tabu_search(
        initial_best,
        max_iterations=max_iterations,
        tabu_size=30,
        neighborhood_size=8
    )
    elapsed_time = time.time() - start_time

    metrics = {
        'elapsed_time': elapsed_time,
        'iterations': max_iterations,
        'best_cost': best_individual.cost,
        'best_vehicles': len(best_individual.route_plan),
        'initial_cost': initial_best.cost
    }

    return best_individual, metrics


def count_vehicles(solution: Individual) -> int:
    """统计车辆数"""
    return len([r for r in solution.route_plan if r])


def calculate_cost_solution(solution: Individual, cost_calculator: CostCalculator,
                            weights: np.ndarray, volumes: np.ndarray,
                            customer_ids: np.ndarray) -> float:
    """计算解的总成本"""
    cost, _, _ = cost_calculator.calculate_cost(
        solution, weights, volumes, customer_ids
    )
    return cost


def get_convergence_generation(history: List[float], threshold: float = 0.01) -> int:
    """
```

```
    获取收敛代数（成本变化小于阈值的代数）

    Args:
        history: 成本历史记录
        threshold: 收敛阈值（相对于最优解的变化比例）

    Returns:
        收敛代数
    """
    if len(history) < 10:
        return len(history)

    best_cost = min(history)
    for i, cost in enumerate(history):
        if (cost - best_cost) / best_cost < threshold:
            return i

    return len(history)


def comparative_experiment() -> Dict:
    """
    对比不同算法性能

    对比的算法：
    1. 遗传算法（标准）
    2. 遗传+贪心混合算法
    3. 模拟退火算法
    4. 禁忌搜索算法

    Returns:
        包含各算法性能指标的字典
    """
    print("=" * 70)
    print("算法性能对比实验")
    print("=" * 70)
```

```
print("\n[1/5] 加载数据...")

loader = DataLoader()

distance_matrix = loader.load_distance_matrix()

customer_data, coords, time_windows = loader.load_customer_data()



n_customers = len(customer_data)

print(f"客户数量: {n_customers}")



if isinstance(time_windows, pd.DataFrame):

    def time_to_float(time_str):

        try:

            if isinstance(time_str, str) and ':' in time_str:

                h, m = map(int, time_str.split(':'))

                return h + m / 60

            elif isinstance(time_str, (int, float)):

                return float(time_str)

            else:

                return 8.0

        except:

            return 8.0



    time_windows = time_windows.values

    converted_time_windows = []

    for row in time_windows:

        if len(row) >= 2:

            start = time_to_float(row[1])

            end = time_to_float(row[2])

            converted_time_windows.append([start, end])

        else:

            converted_time_windows.append([8.0, 18.0])

    time_windows = np.array(converted_time_windows)



customer_col = '客户 ID' if '客户 ID' in customer_data.columns else \

               '目标客户编号' if '目标客户编号' in customer_data.columns else \

               '客户编号' if '客户编号' in customer_data.columns else customer_data.columns[0]

customer_data['客户 ID'] = customer_data[customer_col].astype(int)
```

```
cost_calculator = CostCalculator(
    distance_matrix, customer_data, time_windows, '燃油车 1'
)

weights = customer_data['总重量' if '总重量' in customer_data.columns else '重量'].values
volumes = customer_data['总体积' if '总体积' in customer_data.columns else '体积'].values
customer_ids = customer_data['客户 ID'].values

algorithms = {
    "遗传算法": lambda: genetic_algorithm(distance_matrix, customer_data, time_windows, coords, max_generations=300),
    "遗传+贪心": lambda: hybrid_ga_greedy(distance_matrix, customer_data, time_windows, coords, max_generations=200),
    "模拟退火": lambda: simulated_annealing_standalone(distance_matrix, customer_data, time_windows, coords, max_iterations=800),
    "禁忌搜索": lambda: tabu_search_standalone(distance_matrix, customer_data, time_windows, coords, max_iterations=400)
}

results = {}

print("\n[2/5] 运行算法对比实验...")

for name, algo_func in algorithms.items():
    print(f"\n{'='*60}")
    print(f"运行算法: {name}")
    print(f"{'='*60}")

    try:
        solution, metrics = algo_func()

        cost = calculate_cost_solution(solution, cost_calculator, weights, volumes, customer_ids)

        results[name] = {
            "车辆数": count_vehicles(solution),
            "总成本": cost,
            "运行时间": metrics['elapsed_time'],
            "收敛代数": get_convergence_generation(metrics.get('history', [])),
            "初始成本": metrics.get('initial_cost', cost),
```

```
                "成本改善率": (metrics.get('initial_cost', cost) - cost) / metrics.get('initial_cost', cost) * 100 if metrics.get('initial_cost', 0) > 0 else 0
            }

            print(f"   车辆数: {results[name]['车辆数']}辆")
            print(f"   总成本: {results[name]['总成本']:.2f}元")
            print(f"   运行时间: {results[name]['运行时间']:.2f}秒")
            print(f"   收敛代数: {results[name]['收敛代数']}")

        except Exception as e:
            print(f"   ✗ 算法执行失败: {str(e)}")
            import traceback
            traceback.print_exc()
            results[name] = {
                "车辆数": -1,
                "总成本": -1,
                "运行时间": -1,
                "收敛代数": -1,
                "错误": str(e)
            }

print("\n[3/5] 分析对比结果...")

valid_results = {k: v for k, v in results.items() if v['总成本'] > 0}

if valid_results:
    best_cost_algo = min(valid_results.items(), key=lambda x: x[1]['总成本'])[0]
    fastest_algo = min(valid_results.items(), key=lambda x: x[1]['运行时间'])[0]
    fewest_vehicles_algo = min(valid_results.items(), key=lambda x: x[1]['车辆数'])[0]

    print(f"   最低成本算法: {best_cost_algo} ({valid_results[best_cost_algo]['总成本']:.2f}元)")
    print(f"   最快算法: {fastest_algo} ({valid_results[fastest_algo]['运行时间']:.2f}秒)")
    print(f"   最少车辆算法: {fewest_vehicles_algo} ({valid_results[fewest_vehicles_algo]['车辆数']}辆)")

print("\n[4/5] 生成对比表格...")

print("\n" + "=" * 70)
```

```
print("算法性能对比汇总表")

print("=" * 70)

print(f"{'算法名称':<15} {'车辆数':<10} {'总成本(元)':<15} {'运行时间(秒)':<15} {'收敛代数':<10}")

print("-" * 70)


for name, result in sorted(results.items(), key=lambda x: x[1]['总成本'] if x[1]['总成本'] > 0 else float('inf')):

    if result['总成本'] > 0:

        print(f"{name:<15} {result['车辆数']:<10} {result['总成本']:<15.2f} {result['运行时间']:<15.2f} {result['收敛代数']:<10}")

    else:

        print(f"{name:<15} {'失败':<10} {'-':<15} {'-':<15} {'-':<10}")


print("=" * 70)


print("\n[5/5] 生成详细报告...")


report = []

report.append("\n 各算法特点分析：")

report.append("")

report.append("1. 遗传算法（GA）")

report.append("   - 优点：全局搜索能力强，适合复杂问题")

report.append("   - 缺点：收敛速度较慢，容易早熟")

report.append("")

report.append("2. 遗传+贪心混合算法（GA+Greedy）")

report.append("   - 优点：初始解质量高，收敛速度快")

report.append("   - 缺点：依赖贪心策略，可能陷入局部最优")

report.append("")

report.append("3. 模拟退火（SA）")

report.append("   - 优点：能跳出局部最优，全局搜索能力较强")

report.append("   - 缺点：参数敏感，需要仔细调参")

report.append("")

report.append("4. 禁忌搜索（TS）")

report.append("   - 优点：局部搜索能力强，避免循环")

report.append("   - 缺点：对初始解依赖较强，计算开销较大")

report.append("")


if valid_results:
```

```
            report.append("推荐算法：")
            if len(valid_results) > 0:
                report.append(f"  - 综合最优: {best_cost_algo}")
                report.append(f"  - 速度最快: {fastest_algo}")
                report.append(f"  - 车辆最少: {fewest_vehicles_algo}")

        report_str = "\n".join(report)
        print(report_str)

        print("\n" + "=" * 70)
        print("对比实验完成")
        print("=" * 70)

        return results

# ============== 迭代局部搜索（ILS）算法 ==============
class BasicILS:
    """
    迭代局部搜索（Iterated Local Search）算法

    ILS 是一种高效的元启发式算法，通过在局部最优解之间进行扰动和搜索，
    能够有效避免局部最优，获得高质量的解。

    特点：
    - 简单而强大：核心思想简单，但性能优异
    - 易于实现：可以重用现有的局部搜索算子
    - 鲁棒性强：对参数设置不敏感
    - 效率高：在合理时间内获得高质量解
    """

    def __init__(self, distance_matrix: np.ndarray, customer_data: pd.DataFrame,
                 time_windows: np.ndarray, coords: pd.DataFrame = None,
                 vehicle_type: str = '燃油车 1'):
        """
        初始化 ILS 算法
```

```
Args:
        distance_matrix: 距离矩阵
        customer_data: 客户数据
        time_windows: 时间窗
        coords: 客户坐标（用于聚类分析）
        vehicle_type: 车辆类型
    """
    self.distance_matrix = distance_matrix
    self.customer_data = customer_data
    self.time_windows = time_windows
    self.coords = coords
    self.vehicle_type = vehicle_type

    # 提取客户需求
    customer_col = '客户 ID' if '客户 ID' in customer_data.columns else \
                   '目标客户编号' if '目标客户编号' in customer_data.columns else \
                   '客户编号' if '客户编号' in customer_data.columns else customer_data.columns[0]

    weight_col = '总重量' if '总重量' in customer_data.columns else '重量'
    volume_col = '总体积' if '总体积' in customer_data.columns else '体积'

    self.customer_ids = customer_data[customer_col].values
    self.weights = customer_data[weight_col].values
    self.volumes = customer_data[volume_col].values

    # 车辆约束
    self.max_weight = Config.VEHICLE_TYPES[vehicle_type]['capacity_kg']
    self.max_volume = Config.VEHICLE_TYPES[vehicle_type]['capacity_m3']

    # ID 映射
    self.id_to_idx = {cid: i for i, cid in enumerate(self.customer_ids)}

    # 创建成本计算器
    self.cost_calculator = CostCalculator(
        distance_matrix, customer_data, time_windows, vehicle_type
    )
```

```
        # ILS 参数
        self.max_iterations = 500
        self.max_no_improve = 100
        self.perturbation_strength = 5

        # 当前解和最优解
        self.current_solution = None
        self.best_solution = None
        self.best_cost = float('inf')

        # 历史记录
        self.history = []
        self.cost_history = []

    def load_current_solution(self) -> Individual:
        """加载当前解（使用贪心构造的初始解）"""
        ga = GeneticAlgorithm(
            distance_matrix=self.distance_matrix,
            customer_data=self.customer_data,
            time_windows=self.time_windows,
            vehicle_type=self.vehicle_type,
            coords=self.coords
        )

        population = ga.initialize_population()
        initial_best = min(population, key=lambda x: x.cost)

        return initial_best

    def local_search(self, solution: Individual, max_iterations: int = 100) -> Individual:
        """
        增强局部搜索 - 使用变邻域搜索(VNS)策略

        Args:
            solution: 初始解
```

```
        max_iterations: 最大迭代次数

    Returns:
        局部最优解
    """
    improved = True
    iteration = 0

    while improved and iteration < max_iterations:
        improved = False
        iteration += 1

        # 尝试不同的移动算子
        best_improvement = 0
        best_move = None
        best_neighbor = None

        # 1. 路径内优化：2-opt
        for route_idx, route in enumerate(solution.route_plan):
            if len(route) >= 4:
                improved_route, improvement = self.two_opt_route(route, solution)
                if improvement > best_improvement:
                    best_improvement = improvement
                    best_neighbor = improved_route
                    best_move = ('2opt', route_idx)

        # 2. 客户间交换：同一条路径内两个客户交换位置
        for route_idx, route in enumerate(solution.route_plan):
            if len(route) >= 2:
                improved_route, improvement = self.swap_customers(route, solution)
                if improvement > best_improvement:
                    best_improvement = improvement
                    best_neighbor = improved_route
                    best_move = ('swap', route_idx)

        # 3. 客户移动：将一个客户移动到另一位置
```

```
for route_idx in range(len(solution.route_plan)):
    improved_solution, improvement = self.move_customer(solution, route_idx)
    if improvement > best_improvement:
        best_improvement = improvement
        best_neighbor = improved_solution.route_plan[route_idx] if best_move and best_move[0] == 'move' else None
        best_move = ('move', route_idx)

# 4. 路径间交换：两个不同路径的客户交换
if len(solution.route_plan) >= 2:
    improved_solution, improvement = self.cross_exchange(solution)
    if improvement > best_improvement:
        best_improvement = improvement
        best_move = ('cross_exchange', None)

# 5. 路径合并
improved_solution, improvement = self.smart_route_merge(solution)
if improvement > best_improvement:
    best_improvement = improvement
    best_move = ('merge', None)

# 应用最佳移动
if best_move and best_improvement > 0:
    if best_move[0] == '2opt' and best_neighbor:
        solution.route_plan[best_move[1]] = best_neighbor
        improved = True
    elif best_move[0] == 'swap' and best_neighbor:
        solution.route_plan[best_move[1]] = best_neighbor
        improved = True
    elif best_move[0] == 'move':
        solution = improved_solution
        improved = True
    elif best_move[0] == 'cross_exchange':
        solution = improved_solution
        improved = True
    elif best_move[0] == 'merge':
        solution = improved_solution
```

```
                    improved = True

        return solution

    def two_opt_route(self, route: List[int], solution: Individual) -> Tuple[List[int], float]:
        """
        对单条路径应用 2-opt 优化

        Returns:
            (优化后的路径, 成本改善量)
        """
        if len(route) < 4:
            return route, 0

        best_route = route.copy()
        best_improvement = 0

        for i in range(1, len(route) - 1):
            for j in range(i + 1, len(route)):
                new_route = route[:i] + route[i:j][::-1] + route[j:]

                # 计算成本变化
                old_cost = self.estimate_route_cost(route)
                new_cost = self.estimate_route_cost(new_route)

                improvement = old_cost - new_cost

                if improvement > best_improvement:
                    best_improvement = improvement
                    best_route = new_route

        return best_route, best_improvement

    def swap_customers(self, route: List[int], solution: Individual) -> Tuple[List[int], float]:
        """
        交换路径中两个客户的位置
```

```
        Returns:
            (优化后的路径, 成本改善量)
        """
        if len(route) < 2:
            return route, 0

        best_route = route.copy()
        best_improvement = 0

        for i in range(len(route)):
            for j in range(i + 1, len(route)):
                new_route = route.copy()
                new_route[i], new_route[j] = new_route[j], new_route[i]

                old_cost = self.estimate_route_cost(route)
                new_cost = self.estimate_route_cost(new_route)

                improvement = old_cost - new_cost

                if improvement > best_improvement:
                    best_improvement = improvement
                    best_route = new_route

        return best_route, best_improvement

    def move_customer(self, solution: Individual, route_idx: int) -> Tuple[Individual, float]:
        """
        将一个客户从一条路径移动到另一位置

        Returns:
            (新的解, 成本改善量)
        """
        if not solution.route_plan[route_idx]:
            return solution, 0
```

```
best_solution = None
best_improvement = 0

route = solution.route_plan[route_idx]

for c_idx in range(len(route)):
    customer = route[c_idx]

    # 从原路径移除
    new_route = route.copy()
    new_route.pop(c_idx)

    # 尝试插入到其他路径
    for other_route_idx in range(len(solution.route_plan)):
        if other_route_idx == route_idx:
            continue

        other_route = solution.route_plan[other_route_idx].copy()

        # 检查容量约束
        total_w = sum(self.weights[self.id_to_idx[c]] for c in other_route if c in self.id_to_idx)
        total_v = sum(self.volumes[self.id_to_idx[c]] for c in other_route if c in self.id_to_idx)

        if self.weights[self.id_to_idx[customer]] + total_w <= self.max_weight * 0.98 and \
           self.volumes[self.id_to_idx[customer]] + total_v <= self.max_volume * 0.98:

            # 尝试不同插入位置
            for insert_pos in range(len(other_route) + 1):
                test_route = other_route.copy()
                test_route.insert(insert_pos, customer)

                # 创建新解
                new_solution = Individual(solution.route_plan.copy())
                new_solution.route_plan[route_idx] = new_route
                new_solution.route_plan[other_route_idx] = test_route
```

```
                        # 评估
                        old_cost = self.evaluate_solution(solution)
                        new_cost = self.evaluate_solution(new_solution)

                        improvement = old_cost - new_cost

                        if improvement > best_improvement:
                            best_improvement = improvement
                            best_solution = new_solution

        return best_solution if best_solution else solution, best_improvement

def cross_exchange(self, solution: Individual) -> Tuple[Individual, float]:
    """
    路径间交叉交换：两条路径交换客户段

    Returns:
        (新的解, 成本改善量)
    """
    best_solution = None
    best_improvement = 0

    for r1_idx in range(len(solution.route_plan)):
        for r2_idx in range(r1_idx + 1, len(solution.route_plan)):
            route1 = solution.route_plan[r1_idx]
            route2 = solution.route_plan[r2_idx]

            if len(route1) < 2 or len(route2) < 2:
                continue

            # 尝试不同的交换段
            for seg1_start in range(len(route1)):
                for seg1_end in range(seg1_start + 1, len(route1) + 1):
                    for seg2_start in range(len(route2)):
                        for seg2_end in range(seg2_start + 1, len(route2) + 1):
                            # 创建新路径
```

```
                        new_route1 = route1[:seg1_start] + route2[seg2_start:seg2_end] + route1[seg1_end:]
                        new_route2 = route2[:seg2_start] + route1[seg1_start:seg1_end] + route2[seg2_end:]

                        # 检查容量约束
                        w1 = sum(self.weights[self.id_to_idx[c]] for c in new_route1 if c in self.id_to_idx)
                        v1 = sum(self.volumes[self.id_to_idx[c]] for c in new_route1 if c in self.id_to_idx)
                        w2 = sum(self.weights[self.id_to_idx[c]] for c in new_route2 if c in self.id_to_idx)
                        v2 = sum(self.volumes[self.id_to_idx[c]] for c in new_route2 if c in self.id_to_idx)

                        if w1 <= self.max_weight * 0.98 and v1 <= self.max_volume * 0.98 and \
                           w2 <= self.max_weight * 0.98 and v2 <= self.max_volume * 0.98:

                            # 创建新解
                            new_solution = Individual(solution.route_plan.copy())
                            new_solution.route_plan[r1_idx] = new_route1
                            new_solution.route_plan[r2_idx] = new_route2

                            old_cost = self.evaluate_solution(solution)
                            new_cost = self.evaluate_solution(new_solution)

                            improvement = old_cost - new_cost

                            if improvement > best_improvement:
                                best_improvement = improvement
                                best_solution = new_solution

        return best_solution if best_solution else solution, best_improvement

    def smart_route_merge(self, solution: Individual) -> Tuple[Individual, float]:
        """
        智能路径合并：基于地理邻近性和容量约束合并路径

        Returns:
            (新的解, 成本改善量)
        """
        best_solution = None
```

```
best_improvement = 0

for r1_idx in range(len(solution.route_plan)):
    for r2_idx in range(r1_idx + 1, len(solution.route_plan)):
        route1 = solution.route_plan[r1_idx]
        route2 = solution.route_plan[r2_idx]

        if not route1 or not route2:
            continue

        # 检查容量约束
        total_w = sum(self.weights[self.id_to_idx[c]] for c in route1 + route2 if c in self.id_to_idx)
        total_v = sum(self.volumes[self.id_to_idx[c]] for c in route1 + route2 if c in self.id_to_idx)

        if total_w <= self.max_weight * 0.98 and total_v <= self.max_volume * 0.98:
            # 创建新路径
            merged_route = route1 + route2

            # 优化合并后的路径顺序
            optimized_route = self.optimize_route_order(merged_route)

            # 创建新解
            new_solution = Individual(solution.route_plan.copy())
            new_solution.route_plan[r1_idx] = optimized_route
            new_solution.route_plan[r2_idx] = []

            # 清理空路径
            new_solution.route_plan = [r for r in new_solution.route_plan if r]

            old_cost = self.evaluate_solution(solution)
            new_cost = self.evaluate_solution(new_solution)

            improvement = old_cost - new_cost

            if improvement > best_improvement:
                best_improvement = improvement
```

```
                        best_solution = new_solution

        return best_solution if best_solution else solution, best_improvement

def optimize_route_order(self, route: List[int]) -> List[int]:
    """优化路径中客户的访问顺序（基于最近邻）"""
    if len(route) <= 1:
        return route

    optimized = []
    remaining = route.copy()
    current = 0

    while remaining:
        nearest = min(remaining, key=lambda c: self.distance_matrix[current][self.id_to_idx[c] + 1] if c in self.id_to_idx else float('inf'))
        optimized.append(nearest)
        remaining.remove(nearest)
        if nearest in self.id_to_idx:
            current = self.id_to_idx[nearest] + 1

    return optimized

def estimate_route_cost(self, route: List[int]) -> float:
    """估算路径成本（基于距离）"""
    if not route:
        return 0

    total_dist = 0
    prev_idx = 0

    for customer in route:
        if customer in self.id_to_idx:
            customer_idx = self.id_to_idx[customer] + 1
            total_dist += self.distance_matrix[prev_idx][customer_idx]
            prev_idx = customer_idx
```

```
        total_dist += self.distance_matrix[prev_idx][0]

        return total_dist * 0.8

    def evaluate_solution(self, solution: Individual) -> float:
        """评估解的总成本"""
        cost, _, _ = self.cost_calculator.calculate_cost(
            solution, self.weights, self.volumes, self.customer_ids
        )
        return cost

    def accept(self, new_solution: Individual, current_cost: float) -> bool:
        """
        接受准则：允许接受不比当前解差的解

        Returns:
            是否接受新解
        """
        new_cost = self.evaluate_solution(new_solution)
        return new_cost <= current_cost

    def perturb(self, solution: Individual, strength: int = None) -> Individual:
        """
        智能扰动：基于问题特征的扰动策略

        Args:
            solution: 当前解
            strength: 扰动强度（默认根据未改进次数自适应）

        Returns:
            扰动后的解
        """
        if strength is None:
            strength = self.perturbation_strength

        new_solution = Individual(solution.route_plan.copy())
```

```
# 策略 1：拆分最长的路径
if new_solution.route_plan:
    longest_route_idx = max(range(len(new_solution.route_plan)),
                            key=lambda i: len(new_solution.route_plan[i]))
    route = new_solution.route_plan[longest_route_idx]

    if len(route) >= 4:
        split_pos = len(route) // 2
        route1 = route[:split_pos]
        route2 = route[split_pos:]

        new_solution.route_plan[longest_route_idx] = route1
        new_solution.route_plan.append(route2)

# 策略 2：随机交换两条路径的客户
if len(new_solution.route_plan) >= 2:
    r1_idx, r2_idx = random.sample(range(len(new_solution.route_plan)), 2)

    if new_solution.route_plan[r1_idx] and new_solution.route_plan[r2_idx]:
        c1_idx = random.randint(0, len(new_solution.route_plan[r1_idx]) - 1)
        c2_idx = random.randint(0, len(new_solution.route_plan[r2_idx]) - 1)

        new_solution.route_plan[r1_idx][c1_idx], new_solution.route_plan[r2_idx][c2_idx] = \
            new_solution.route_plan[r2_idx][c2_idx], new_solution.route_plan[r1_idx][c1_idx]

# 策略 3：移动客户到不同路径
for _ in range(strength):
    if len(new_solution.route_plan) >= 2:
        r1_idx, r2_idx = random.sample(range(len(new_solution.route_plan)), 2)

        if new_solution.route_plan[r1_idx]:
            c_idx = random.randint(0, len(new_solution.route_plan[r1_idx]) - 1)
            customer = new_solution.route_plan[r1_idx].pop(c_idx)

            insert_pos = random.randint(0, len(new_solution.route_plan[r2_idx]))
```

```
                    new_solution.route_plan[r2_idx].insert(insert_pos, customer)

        # 清理空路径
        new_solution.route_plan = [r for r in new_solution.route_plan if r]
        new_solution.n_vehicles = len(new_solution.route_plan)

        return new_solution

    def solve(self, initial_solution: Individual = None) -> Individual:
        """
        运行 ILS 算法

        Args:
            initial_solution: 初始解（如果为 None，则使用贪心构造）

        Returns:
            最优解
        """
        print("\n" + "=" * 70)
        print("迭代局部搜索（ILS）算法")
        print("=" * 70)

        # 1. 获取初始解
        if initial_solution is None:
            print("\n[1/4] 构造初始解...")
            self.current_solution = self.load_current_solution()
        else:
            self.current_solution = initial_solution

        self.best_solution = Individual(self.current_solution.route_plan.copy())
        self.best_cost = self.evaluate_solution(self.best_solution)

        print(f"初始成本: {self.best_cost:.2f}元")
        print(f"初始车辆数: {len(self.current_solution.route_plan)}")

        # 2. 主循环
```

```
print("\n[2/4] 开始迭代优化...")

no_improve_count = 0

iteration = 0



while iteration < self.max_iterations and no_improve_count < self.max_no_improve:

    iteration += 1



    # 局部搜索

    new_solution = self.local_search(Individual(self.current_solution.route_plan.copy()))

    new_cost = self.evaluate_solution(new_solution)



    # 接受准则

    if self.accept(new_solution, self.current_solution.cost):

        self.current_solution = new_solution

        self.current_solution.cost = new_cost



        # 更新最优解

        if new_cost < self.best_cost:

            self.best_solution = Individual(new_solution.route_plan.copy())

            self.best_cost = new_cost

            no_improve_count = 0



            if iteration % 20 == 0:

                print(f"  第{iteration}次迭代: 发现更优解 {self.best_cost:.2f}元, 车辆数={len(self.best_solution.route_plan)}")

        else:

            no_improve_count += 1

    else:

        no_improve_count += 1



    # 扰动

    if no_improve_count >= 20:

        print(f"  第{iteration}次迭代: 执行扰动 (no_improve={no_improve_count})")

        self.current_solution = self.perturb(self.current_solution)

        self.current_solution.cost = self.evaluate_solution(self.current_solution)

        no_improve_count = 0
```

```
            # 记录历史
            self.cost_history.append(self.best_cost)

            # 早停
            if no_improve_count >= self.max_no_improve:
                print(f"  第{iteration}次迭代: 早停收敛 (no_improve={no_improve_count})")
                break

        # 3. 最终优化
        print("\n[3/4] 最终局部搜索优化...")
        self.best_solution = self.local_search(self.best_solution)
        self.best_cost = self.evaluate_solution(self.best_solution)

        # 4. 结果输出
        print("\n[4/4] ILS 优化完成")
        print(f"最终成本: {self.best_cost:.2f}元")
        print(f"最终车辆数: {len(self.best_solution.route_plan)}")
        print(f"总迭代次数: {iteration}")
        print(f"收敛代数: {get_convergence_generation(self.cost_history)}")
        print("=" * 70)

        return self.best_solution

def ils_with_hybrid_ga(max_generations: int = 200) -> Individual:
    """
    ILS 结合遗传算法的混合算法

    策略:
    1. 遗传算法生成优质初始解
    2. ILS 进行深度局部优化
    3. 多次扰动和搜索

    Args:
        max_generations: 遗传算法最大迭代代数

    Returns:
```

```
    最优解
"""
print("\n" + "=" * 70)
print("混合优化: 遗传算法 + 迭代局部搜索 (GA + ILS)")
print("=" * 70)

print("\n[阶段 1/3] 遗传算法全局搜索...")
ga = GeneticAlgorithm(
    distance_matrix=loader.load_distance_matrix(),
    customer_data=customer_data,
    time_windows=time_windows,
    vehicle_type='燃油车 1',
    coords=coords
)
ga.max_generations = max_generations

ga_solution = ga.run()
print(f"GA 结果: 成本={ga_solution.cost:.2f}元, 车辆数={len(ga_solution.route_plan)}")

print("\n[阶段 2/3] ILS 深度优化...")
ils = BasicILS(
    distance_matrix=loader.load_distance_matrix(),
    customer_data=customer_data,
    time_windows=time_windows,
    coords=coords,
    vehicle_type='燃油车 1'
)

ils.max_iterations = 300
ils.max_no_improve = 80
ils.perturbation_strength = 7

ils_solution = ils.solve(ga_solution)

print("\n[阶段 3/3] 结果对比")
print(f"GA 成本: {ga_solution.cost:.2f}元, 车辆数: {len(ga_solution.route_plan)}")
```

```
    print(f"ILS 成本: {ils_solution.cost:.2f}元，车辆数: {len(ils_solution.route_plan)}")
    print(f"改善率: {(ga_solution.cost - ils_solution.cost) / ga_solution.cost * 100:.2f}%")

    print("\n" + "=" * 70)
    print("混合优化完成!")
    print("=" * 70)

    return ils_solution

# 使用示例
if __name__ == "__main__":
    main()



class ILSSolver:
    """
    迭代局部搜索（Iterated Local Search）算法

    算法框架：
    1. 初始解生成
    2. 强化局部搜索（Intensification）
    3. 扰动（Perturbation）
    4. 接受准则（基于模拟退火）

    目标：车辆数从 111→75-85，成本从 59k→42-48k
    """

    def __init__(self, problem_instance, initial_solution=None):
        self.problem = problem_instance
        self.current_solution = initial_solution or self.load_initial_solution()
        self.best_solution = copy.deepcopy(self.current_solution)
        self.best_cost = self.evaluate(self.best_solution)

        self.max_iterations = 500
        self.max_no_improve = 50
        self.initial_temperature = 1000
```

```
        self.cooling_rate = 0.95
        self.temperature = self.initial_temperature

        self.iterations = 0
        self.improvements = 0
        self.start_time = time.time()

    def load_initial_solution(self):
        """加载当前遗传算法的最优解作为初始解"""
        print("加载当前遗传算法最优解作为 ILS 初始解...")
        return self.construct_greedy_solution()

    def construct_greedy_solution(self):
        """快速贪心构造初始解"""
        print("使用贪心算法构造初始解...")
        from greedy_packing import fixed_greedy_packing
        return fixed_greedy_packing(self.problem.customers, self.problem.vehicle_capacity)

    def evaluate(self, solution):
        """评估解的质量"""
        from cost_calculator import fixed_cost_calculation
        return fixed_cost_calculation(solution, self.problem)

    def solve(self):
        """ILS 主求解循环"""
        print(f"开始 ILS 优化，初始解: {len(self.current_solution.routes)}辆车, "
              f"成本: {self.evaluate(self.current_solution):.2f}元")

        no_improve_count = 0
        self.temperature = self.initial_temperature

        for iteration in range(self.max_iterations):
            self.iterations = iteration

            s_local = self.intensification_search(self.current_solution)
            s_local_cost = self.evaluate(s_local)
```

```
            if self.acceptance_criterion(s_local_cost, self.evaluate(self.current_solution)):
                self.current_solution = s_local

                if s_local_cost < self.best_cost:
                    self.best_solution = copy.deepcopy(s_local)
                    self.best_cost = s_local_cost
                    self.improvements += 1
                    no_improve_count = 0

                    print(f"迭代{iteration}: 新最优！{len(self.best_solution.routes)}辆车, "
                          f"成本{self.best_cost:.2f}元")
                else:
                    no_improve_count += 1
            else:
                no_improve_count += 1

            if no_improve_count >= self.max_no_improve:
                print(f"迭代{iteration}: 长时间无改进，执行扰动...")
                self.current_solution = self.perturbation(self.current_solution)
                no_improve_count = 0

            self.temperature *= self.cooling_rate

            if iteration % 50 == 0:
                self.print_progress(iteration)

        self.best_solution = self.intensification_search(self.best_solution)
        self.best_cost = self.evaluate(self.best_solution)

        self.print_final_result()
        return self.best_solution

    def intensification_search(self, solution):
        """
        强化局部搜索阶段
```

使用多种邻域算子进行深度优化：

- 2-opt: 路径内反转优化

- Or-opt: 移动连续客户段

- Cross-exchange: 路径间交换

- Route merge: 路径合并

持续搜索直到达到最大迭代次数或无改进

"""

improved = True

iteration = 0

max_iter = 50

while improved and iteration < max_iter:

    improved = False

    iteration += 1

    best_improvement = 0

    best_neighbor = None

    operators = [

        ('2opt', self.two_opt_all_routes),

        ('oropt', self.or_opt_routes),

        ('cross', self.cross_exchange_routes),

        ('merge', self.route_merge)

    ]

    for move_type, op_func in operators:

        new_sol = copy.deepcopy(solution)

        op_func(new_sol)

        new_cost = self.evaluate(new_sol)

        current_cost = self.evaluate(solution)

        if new_cost < current_cost and current_cost - new_cost > best_improvement:

            best_improvement = current_cost - new_cost

            best_neighbor = new_sol

```
            if best_neighbor:
                solution = best_neighbor
                improved = True

        return solution

    def perturbation(self, solution):
        """
        扰动阶段 - 跳出局部最优

        策略：
        1. 路径拆分：拆分过长的路径
        2. 客户重分配：随机移动客户到不同路径
        3. 路径合并：合并容量可合并的短路径
        """
        perturbed = copy.deepcopy(solution)

        all_customers = []
        for route in perturbed.routes:
            all_customers.extend(route)

        if not all_customers:
            return perturbed

        num_to_remove = max(1, int(len(all_customers) * 0.2))
        num_to_remove = min(num_to_remove, len(all_customers))
        customers_to_remove = random.sample(all_customers, num_to_remove)

        for customer_id in customers_to_remove:
            for route in perturbed.routes:
                if customer_id in route:
                    route.remove(customer_id)
                    break

        perturbed.routes = [r for r in perturbed.routes if r]
```

```
        for customer_id in customers_to_remove:
            best_route_idx, best_position = self.find_best_insertion(customer_id, perturbed)

            if best_route_idx is not None:
                perturbed.routes[best_route_idx].insert(best_position, customer_id)
            else:
                perturbed.routes.append([customer_id])

        return perturbed

    def acceptance_criterion(self, new_cost, current_cost):
        """
        接受准则 - 模拟退火思想

        如果新解更优则接受
        如果新解稍差但满足温度条件也接受（避免陷入局部最优）
        """
        if new_cost < current_cost:
            return True

        delta = current_cost - new_cost
        probability = math.exp(delta / self.temperature)
        return random.random() < probability

    def two_opt_all_routes(self, solution):
        """对所有路径应用 2-opt"""
        for i, route in enumerate(solution.routes):
            if len(route) >= 4:
                solution.routes[i] = self.two_opt(route)
        return solution

    def two_opt(self, route):
        """单路径 2-opt 优化"""
        if len(route) < 4:
            return route
```

```
    best = list(route)
    best_cost = self.evaluate_route(route)

    for i in range(1, len(route) - 2):
        for j in range(i + 1, len(route)):
            if j - i == 1:
                continue
            new_route = route[:i] + route[i:j][::-1] + route[j:]
            new_cost = self.evaluate_route(new_route)
            if new_cost < best_cost:
                best = new_route
                best_cost = new_cost

    return best

def or_opt_routes(self, solution, segment_length=3):
    """移动连续客户段"""
    best_cost = self.evaluate(solution)

    for route_idx in range(len(solution.routes)):
        route = solution.routes[route_idx]

        for seg_len in range(1, min(segment_length + 1, len(route))):
            for seg_start in range(len(route) - seg_len + 1):
                segment = route[seg_start:seg_start + seg_len]
                remaining = route[:seg_start] + route[seg_start + seg_len:]

                if not remaining:
                    continue

                for insert_pos in range(len(remaining) + 1):
                    new_route = remaining[:insert_pos] + segment + remaining[insert_pos:]

                    if self.is_feasible_route(new_route):
                        solution.routes[route_idx] = new_route
```

```
                    new_cost = self.evaluate(solution)

                    if new_cost >= best_cost:
                        solution.routes[route_idx] = route
                    else:
                        best_cost = new_cost

        return solution

    def cross_exchange_routes(self, solution):
        """路径间交叉交换"""
        best_cost = self.evaluate(solution)

        for r1_idx in range(len(solution.routes)):
            for r2_idx in range(r1_idx + 1, len(solution.routes)):
                route1 = solution.routes[r1_idx]
                route2 = solution.routes[r2_idx]

                if len(route1) < 2 or len(route2) < 2:
                    continue

                for seg1_start in range(len(route1)):
                    for seg1_end in range(seg1_start + 1, len(route1) + 1):
                        for seg2_start in range(len(route2)):
                            for seg2_end in range(seg2_start + 1, len(route2) + 1):
                                new_route1 = route1[:seg1_start] + route2[seg2_start:seg2_end] + route1[seg1_end:]
                                new_route2 = route2[:seg2_start] + route1[seg1_start:seg1_end] + route2[seg2_end:]

                                if self.is_feasible_route(new_route1) and self.is_feasible_route(new_route2):
                                    solution.routes[r1_idx] = new_route1
                                    solution.routes[r2_idx] = new_route2

                                    new_cost = self.evaluate(solution)
                                    if new_cost >= best_cost:
                                        solution.routes[r1_idx] = route1
                                        solution.routes[r2_idx] = route2
```

```
        return solution

    def route_merge(self, solution):
        """路径合并 - 合并容量允许的短路径"""
        best_cost = self.evaluate(solution)
        merged = False

        for r1_idx in range(len(solution.routes)):
            for r2_idx in range(r1_idx + 1, len(solution.routes)):
                route1 = solution.routes[r1_idx]
                route2 = solution.routes[r2_idx]

                combined = route1 + route2

                if self.is_feasible_route(combined):
                    new_route = self.two_opt(combined)
                    solution.routes[r1_idx] = new_route
                    solution.routes[r2_idx] = []
                    merged = True

        if merged:
            solution.routes = [r for r in solution.routes if r]

        return solution

    def is_feasible_route(self, route):
        """检查路径可行性"""
        total_weight = 0
        total_volume = 0

        for customer_id in route:
            customer = self.problem.customers.get(customer_id)
            if customer is None:
                return False
            total_weight += getattr(customer, 'weight', 0)
```

```
            total_volume += getattr(customer, 'volume', 0)

            if total_weight > self.problem.vehicle_capacity_weight or \
               total_volume > self.problem.vehicle_capacity_volume:
                return False

        return True

    def evaluate_route(self, route):
        """评估单条路径成本"""
        if not route:
            return 0

        total_cost = 0
        prev_idx = 0

        for customer_id in route:
            if customer_id in self.problem.customer_ids:
                customer_idx = self.problem.customer_ids.index(customer_id) + 1
                total_cost += self.problem.distance_matrix[prev_idx][customer_idx]
                prev_idx = customer_idx

        total_cost += self.problem.distance_matrix[prev_idx][0]
        return total_cost * 0.8

    def find_best_insertion(self, customer_id, solution):
        """找到最佳插入位置"""
        best_cost_increase = float('inf')
        best_route_idx = None
        best_position = None

        customer = self.problem.customers.get(customer_id)
        if customer is None:
            return None, None

        for route_idx, route in enumerate(solution.routes):
```

```
                route_weight = sum(getattr(self.problem.customers.get(cid), 'weight', 0) for cid in route)
                route_volume = sum(getattr(self.problem.customers.get(cid), 'volume', 0) for cid in route)

                if route_weight + getattr(customer, 'weight', 0) > self.problem.vehicle_capacity_weight or \
                    route_volume + getattr(customer, 'volume', 0) > self.problem.vehicle_capacity_volume:
                    continue

                for pos in range(len(route) + 1):
                    new_route = route[:pos] + [customer_id] + route[pos:]
                    cost_increase = self.evaluate_route(new_route) - self.evaluate_route(route)

                    if cost_increase < best_cost_increase:
                        best_cost_increase = cost_increase
                        best_route_idx = route_idx
                        best_position = pos

        return best_route_idx, best_position

    def print_progress(self, iteration):
        """显示进度信息"""
        elapsed = time.time() - self.start_time
        print(f"进度: {iteration}/{self.max_iterations}代, "
              f"时间: {elapsed:.1f}s, 温度: {self.temperature:.2f}, "
              f"最优: {len(self.best_solution.routes)}辆车, {self.best_cost:.2f}元")

    def print_final_result(self):
        """显示最终结果"""
        elapsed = time.time() - self.start_time
        print(f"""
{'='*60}
ILS 优化完成!
总迭代: {self.iterations}代
改进次数: {self.improvements}次
总时间: {elapsed:.2f}秒
最终结果: {len(self.best_solution.routes)}辆车, 成本{self.best_cost:.2f}元
相比初始解: 车辆数{len(self.current_solution.routes)}→{len(self.best_solution.routes)},
```

```
成本{self.evaluate(self.current_solution):.2f}→{self.best_cost:.2f}

{'='*60}

""")




class HybridILS:

    """

    混合迭代局部搜索（Hybrid ILS）概念框架



    本框架展示如何重用现有组件构建 ILS 算法：



    组件重用关系：

    ┌─────────────────────────────────────────────────────────────────┐

    │                         ILS 主框架                              │

    ├─────────────────────────────────────────────────────────────────┤

    │  1. 初始解生成          → 重用 GeneticAlgorithm                 │

    │  2. 局部搜索 (Intensification) → 重用 LocalSearch (2-opt 等)    │

    │  3. 接受准则            → 重用 SimulatedAnnealing               │

    │  4. 扰动 (Perturbation) → 重用 GA 的变异操作                    │

    │  5. 精细调整            → 重用 TabuSearch (可选)                │

    └─────────────────────────────────────────────────────────────────┘



    算法流程：

        for iteration in range(max_iterations):

            # 局部搜索阶段

            candidate = local_search.improve(current)



            # 接受准则（SA 思想）

            if sa.accept(candidate, current):

                current = candidate



            # 更新最优解

            if cost(current) < cost(best):

                best = current



            # 扰动阶段（跳出局部最优）
```

```
            if stagnation_detected:
                current = ga.mutate(current)

    这种设计使得各模块可以独立开发和测试，
    同时在 ILS 框架下协同工作。
    """

    def __init__(self, problem_instance):
        self.problem = problem_instance

    def solve(self):
        """
        ILS 主求解流程（概念框架）
        实际实现见 ILSSolver 类
        """
        pass


def validate_solution(solution, problem):
    """
    验证解的有效性
    """
    print("\n 验证解的可行性...")

    served_customers = set()
    for route in solution.routes:
        served_customers.update(route)

    all_customers = set(range(1, problem.num_customers + 1))
    missing = all_customers - served_customers
    if missing:
        print(f"  ✗ 错误：{len(missing)}个客户未被服务")
        return False

    overload_count = 0
    for route in solution.routes:
```

```
            if not problem.is_route_feasible(route):
                overload_count += 1

        if overload_count > 0:
            print(f"  ✗ 错误：{overload_count}条路径超载")
            return False

        print(f"  ✓ 验证通过：{len(solution.routes)}条路径, {len(served_customers)}个客户被服务，无超载")
        return True
```

问题二：

```
    """
华中杯数学建模 A 题 - 问题 2：环保政策影响下的车辆调度
在问题 1 基础上增加绿色配送区动态识别和燃油车限行约束
"""

import numpy as np
import pandas as pd
import random
import time
from typing import List, Tuple, Dict, Set
import os
from 问题一_优化版 import (
    Config, DataLoader, TimeVaryingSpeed, Individual, CostCalculator,
    ConstraintValidator, GeneticAlgorithm, ResultWriter
)

# ============== 扩展配置 ==============
class PolicyConfig:
    """政策配置"""
    # 绿色配送区参数
    GREEN_ZONE_CENTER = (0, 0)  # 圆心坐标
    GREEN_ZONE_RADIUS = 10  # 半径(km) - 按题目要求设置为 10km

    # 燃油车限行时段
    FUEL_RESTRICTION_START = 8  # 8:00
    FUEL_RESTRICTION_END = 16   # 16:00
```

```
# ============== 绿色配送区识别器 ==============

class GreenZoneIdentifier:
    """绿色配送区动态识别器"""

    def __init__(self, center: Tuple[float, float], radius: float):
        self.center = center
        self.radius = radius

    def is_in_green_zone(self, x: float, y: float) -> bool:
        """判断坐标是否在绿色配送区内"""
        distance = np.sqrt((x - self.center[0])**2 + (y - self.center[1])**2)
        return distance <= self.radius

    def identify_green_zone_customers(self, coords: pd.DataFrame) -> Set[int]:
        """
        识别绿色配送区内的客户

        Args:
            coords: 客户坐标数据，包含客户 ID 和坐标列

        Returns:
            绿色配送区内客户 ID 集合
        """
        green_customers = set()

        # 检测坐标列名
        x_col = None
        y_col = None
        id_col = None

        for col in coords.columns:
            col_lower = str(col).lower()
            if 'x' in col_lower:
                x_col = col
            elif 'y' in col_lower:
```

```
                y_col = col
            elif 'id' in col_lower or '编号' in col_lower:
                id_col = col

        # 如果没有找到合适的列，返回空集合
        if not all([x_col, y_col, id_col]):
            return set()

        for _, row in coords.iterrows():
            try:
                x = float(row[x_col])
                y = float(row[y_col])
                if self.is_in_green_zone(x, y):
                    green_customers.add(int(row[id_col]))
            except (ValueError, TypeError):
                continue

        return green_customers

    def get_green_zone_stats(self, coords: pd.DataFrame) -> Dict:
        """获取绿色配送区统计信息"""
        green_customers = self.identify_green_zone_customers(coords)

        return {
            'total_customers': len(coords),
            'green_zone_customers': len(green_customers),
            'green_zone_rate': len(green_customers) / len(coords) * 100,
            'customer_ids': green_customers
        }

# ============== 时间窗约束分析器 ==============
class TimeWindowAnalyzer:
    """时间窗约束分析器"""

    @staticmethod
    def parse_time_to_hours(time_str) -> float:
```

```
    """将时间字符串（如'11:33'）转换为小时数（如 11.55）"""
    if isinstance(time_str, (int, float)):
        return float(time_str)
    try:
        # 尝试直接转换为浮点数
        return float(time_str)
    except (ValueError, TypeError):
        # 解析时间字符串格式 'HH:MM'
        if isinstance(time_str, str) and ':' in time_str:
            parts = time_str.split(':')
            hours = int(parts[0])
            minutes = int(parts[1])
            return hours + minutes / 60.0
    return 0.0


@staticmethod
def requires_restricted_delivery(tw_start: float, tw_end: float) -> bool:
    """
    判断客户是否需要在限行时段配送

    Args:
        tw_start: 时间窗开始时间（小时）
        tw_end: 时间窗结束时间（小时）

    Returns:
        True 如果客户时间窗覆盖 8:00-16:00
    """
    return not (tw_end <= PolicyConfig.FUEL_RESTRICTION_START or
                tw_start >= PolicyConfig.FUEL_RESTRICTION_END)


@staticmethod
def get_restricted_customers(time_windows: np.ndarray,
                             customer_ids: np.ndarray) -> Set[int]:
    """获取需要受限配送的客户"""
    restricted = set()
```

```
        for i, tw_data in enumerate(time_windows):
            try:
                # 时间窗数据可能是 [客户编号, 开始时间, 结束时间] 或 [开始时间, 结束时间]
                # 根据数据格式调整索引
                if len(tw_data) >= 3:
                    # 第一列是客户编号，跳过
                    tw_start = TimeWindowAnalyzer.parse_time_to_hours(tw_data[1])
                    tw_end = TimeWindowAnalyzer.parse_time_to_hours(tw_data[2])
                else:
                    tw_start = TimeWindowAnalyzer.parse_time_to_hours(tw_data[0])
                    tw_end = TimeWindowAnalyzer.parse_time_to_hours(tw_data[1])

                if TimeWindowAnalyzer.requires_restricted_delivery(tw_start, tw_end):
                    restricted.add(customer_ids[i])
            except (ValueError, IndexError, TypeError) as e:
                continue

        return restricted

# ============== 车型分配器 ==============
class VehicleAllocator:
    """车型分配器（考虑政策约束）"""

    def __init__(self, green_zone_identifier: GreenZoneIdentifier,
                 time_windows: np.ndarray, customer_ids: np.ndarray,
                 coords: pd.DataFrame = None):
        self.green_zone = green_zone_identifier
        self.time_windows = time_windows
        self.customer_ids = customer_ids
        self.coords = coords

        # 识别受限客户
        if coords is not None:
            self.green_zone_customers = green_zone_identifier.identify_green_zone_customers(coords)
        else:
            self.green_zone_customers = set()
```

```
    self.restricted_customers = TimeWindowAnalyzer.get_restricted_customers(
        time_windows, customer_ids
    )

    # 必须使用新能源车的客户：同时满足以下条件：
    # 1. 在绿色配送区内
    # 2. 时间窗覆盖限行时段(8:00-16:00)
    self.ev_required = self.green_zone_customers & self.restricted_customers

def can_use_fuel_vehicle(self, customer_id: int,
                         delivery_time: float = 10.0) -> bool:
    """
    判断某客户是否可以使用燃油车

    Args:
        customer_id: 客户 ID
        delivery_time: 预计配送时间
    """
    # 如果客户在绿色配送区
    if customer_id in self.green_zone_customers:
        # 检查时间是否在限行时段
        hour = delivery_time % 24
        if PolicyConfig.FUEL_RESTRICTION_START <= hour < PolicyConfig.FUEL_RESTRICTION_END:
            return False

    return True

def get_vehicle_type_for_customer(self, customer_id: int) -> str:
    """
    获取适合客户需求的车型

    Returns:
        '新能源 1', '新能源 2', 或其他可用车型
    """
    if customer_id in self.ev_required:
```

```
            # 必须使用新能源车，选择载重大的
            return '新能源 1'
        return '燃油车 1'

# ============== 问题 2 遗传算法 ==============
class PolicyGeneticAlgorithm(GeneticAlgorithm):
    """考虑政策约束的遗传算法"""

    def __init__(self, distance_matrix: np.ndarray, customer_data: pd.DataFrame,
                 time_windows: np.ndarray, coords: pd.DataFrame,
                 vehicle_type: str = '燃油车 1'):
        super().__init__(distance_matrix, customer_data, time_windows, vehicle_type)

        # 初始化政策组件
        self.coords = coords
        self.green_zone_identifier = GreenZoneIdentifier(
            PolicyConfig.GREEN_ZONE_CENTER,
            PolicyConfig.GREEN_ZONE_RADIUS
        )
        self.vehicle_allocator = VehicleAllocator(
            self.green_zone_identifier,
            time_windows,
            self.customer_ids,
            coords
        )

        # 统计绿色配送区客户
        green_stats = self.green_zone_identifier.get_green_zone_stats(coords)
        print(f"\n 绿色配送区信息:")
        print(f"  - 绿色配送区客户数: {green_stats['green_zone_customers']}")
        print(f"  - 占比: {green_stats['green_zone_rate']:.1f}%")
        print(f"  - 必须使用新能源车的客户数: {len(self.vehicle_allocator.ev_required)}")

    def _greedy_construct(self, customers: List[int]) -> List[List[int]]:
        """贪心构造初始解（考虑政策约束）"""
        routes = []
```

```
oversized_customers = {}  # 跟踪需要拆分的客户及其服务次数

# 首先识别需要拆分的客户
for customer in customers:
    if customer in self.id_to_idx:
        idx = self.id_to_idx[customer]
        w = self.weights[idx]
        v = self.volumes[idx]

        max_weight = Config.VEHICLE_TYPES['燃油车 1']['capacity_kg']
        max_volume = Config.VEHICLE_TYPES['燃油车 1']['capacity_m3']

        # 检查单个客户需求是否超过车辆容量
        if w > max_weight * 0.95 or v > max_volume * 0.95:
            if customer not in oversized_customers:
                n_vehicles_needed = max(
                    int(np.ceil(w / (max_weight * 0.95))),
                    int(np.ceil(v / (max_volume * 0.95)))
                )
                oversized_customers[customer] = n_vehicles_needed

# 处理不需要拆分的客户（优先填充车辆）
processed_customers = set()
route = []
current_weight = 0
current_volume = 0
use_ev = False

for customer in customers:
    if customer in oversized_customers or customer in processed_customers:
        continue

    if customer in self.id_to_idx:
        idx = self.id_to_idx[customer]
        w = self.weights[idx]
        v = self.volumes[idx]
```

```
            # 检查是否需要使用新能源车
            if customer in self.vehicle_allocator.ev_required:
                use_ev = True
                max_weight = Config.VEHICLE_TYPES['新能源 1']['capacity_kg']
                max_volume = Config.VEHICLE_TYPES['新能源 1']['capacity_m3']
            else:
                max_weight = Config.VEHICLE_TYPES['燃油车 1']['capacity_kg']
                max_volume = Config.VEHICLE_TYPES['燃油车 1']['capacity_m3']

            if current_weight + w <= max_weight * 0.95 and \
               current_volume + v <= max_volume * 0.95:
                route.append(customer)
                current_weight += w
                current_volume += v
                processed_customers.add(customer)
            else:
                if route:
                    routes.append(route)
                route = [customer]
                current_weight = w
                current_volume = v
                use_ev = customer in self.vehicle_allocator.ev_required
                processed_customers.add(customer)

    if route:
        routes.append(route)

    # 处理需要拆分的客户
    all_splits = []
    for customer, n_vehicles in oversized_customers.items():
        idx = self.id_to_idx[customer]
        w = self.weights[idx]
        v = self.volumes[idx]
        portion_w = w / n_vehicles
        portion_v = v / n_vehicles
```

```
            for _ in range(n_vehicles):
                all_splits.append((customer, portion_w, portion_v))

        # 贪心装车
        current_route = []
        current_weight = 0
        current_volume = 0

        for customer, w, v in all_splits:
            if customer in self.vehicle_allocator.ev_required:
                max_weight = Config.VEHICLE_TYPES['新能源 1']['capacity_kg']
                max_volume = Config.VEHICLE_TYPES['新能源 1']['capacity_m3']
            else:
                max_weight = Config.VEHICLE_TYPES['燃油车 1']['capacity_kg']
                max_volume = Config.VEHICLE_TYPES['燃油车 1']['capacity_m3']

            if current_weight + w <= max_weight * 0.95 and \
               current_volume + v <= max_volume * 0.95:
                current_route.append(customer)
                current_weight += w
                current_volume += v
            else:
                if current_route:
                    routes.append(current_route)
                current_route = [customer]
                current_weight = w
                current_volume = v

        if current_route:
            routes.append(current_route)

        return routes if routes else [[]]

    def _regroup_customers(self, customers: List[int]) -> List[List[int]]:
        """重新分组客户到路径（考虑政策约束）- 每个客户只被服务一次"""
        from collections import Counter
```

```
# 每个客户只计算一次真实需求，不管在列表中出现多少次
customer_total_demand = {}  # {客户 ID: (重量, 体积)}

for cid in customers:
    if cid in self.id_to_idx:
        idx = self.id_to_idx[cid]
        if cid not in customer_total_demand:
            customer_total_demand[cid] = (self.weights[idx], self.volumes[idx])

# 分离超重客户和普通客户
split_customers = []  # [(客户 ID, 重量, 体积, 需要的车数)]
normal_customers = []  # [(客户 ID, 重量, 体积)]

fuel_capacity_weight = Config.VEHICLE_TYPES['燃油车 1']['capacity_kg'] * 0.95
fuel_capacity_volume = Config.VEHICLE_TYPES['燃油车 1']['capacity_m3'] * 0.95
ev_capacity_weight = Config.VEHICLE_TYPES['新能源 1']['capacity_kg'] * 0.95
ev_capacity_volume = Config.VEHICLE_TYPES['新能源 1']['capacity_m3'] * 0.95

for cid, (w, v) in customer_total_demand.items():
    # 根据是否为 EV 需求客户选择容量约束
    if cid in self.vehicle_allocator.ev_required:
        if w > ev_capacity_weight or v > ev_capacity_volume:
            n_from_weight = int(np.ceil(w / ev_capacity_weight))
            n_from_volume = int(np.ceil(v / ev_capacity_volume))
            n_vehicles = max(n_from_weight, n_from_volume)
            split_customers.append((cid, w, v, n_vehicles, True))  # True 表示需要 EV
        else:
            normal_customers.append((cid, w, v, True))
    else:
        if w > fuel_capacity_weight or v > fuel_capacity_volume:
            n_from_weight = int(np.ceil(w / fuel_capacity_weight))
            n_from_volume = int(np.ceil(v / fuel_capacity_volume))
            n_vehicles = max(n_from_weight, n_from_volume)
            split_customers.append((cid, w, v, n_vehicles, False))
        else:
```

```
                normal_customers.append((cid, w, v, False))

    # 车辆装载列表 [(当前重量, 当前体积, [客户列表], 需要 EV)]
    vehicle_loads = []

    # 先为超重客户分配专用车辆
    for cid, w, v, n_vehicles, need_ev in split_customers:
        portion_w = w / n_vehicles
        portion_v = v / n_vehicles
        for _ in range(n_vehicles):
            vehicle_loads.append((portion_w, portion_v, [cid], need_ev))

    # 按重量降序排列普通客户以便装箱
    normal_customers.sort(key=lambda x: x[1], reverse=True)

    # 将普通客户打包到现有车辆
    for cid, w, v, need_ev in normal_customers:
        capacity_w = ev_capacity_weight if need_ev else fuel_capacity_weight
        capacity_v = ev_capacity_volume if need_ev else fuel_capacity_volume

        placed = False
        for i, (load_w, load_v, route_custs, route_ev) in enumerate(vehicle_loads):
            # 不能混用 EV 和非 EV 客户
            if route_ev != need_ev:
                continue
            if load_w + w <= capacity_w and load_v + v <= capacity_v:
                vehicle_loads[i] = (load_w + w, load_v + v, route_custs + [cid], need_ev)
                placed = True
                break
        if not placed:
            vehicle_loads.append((w, v, [cid], need_ev))

    # 构建路线计划（不考虑 EV 标志，只保留客户列表）
    route_plan = [load[2] for load in vehicle_loads if load[2]]
    return route_plan if route_plan else [[]]
```

```
# ============== 问题 2 结果输出 ==============

class PolicyResultWriter(ResultWriter):
    """政策影响下的结果输出器"""

    def __init__(self, individual: Individual, cost_calculator: CostCalculator,
                 customer_data: pd.DataFrame, time_windows: np.ndarray,
                 green_stats: Dict, vehicle_allocator: VehicleAllocator):
        super().__init__(individual, cost_calculator, customer_data, time_windows)
        self.green_stats = green_stats
        self.vehicle_allocator = vehicle_allocator

    def save_to_file(self, filename='问题 2 结果_优化.txt'):
        """保存结果到文件"""
        with open(filename, 'w', encoding='utf-8') as f:
            f.write("=" * 70 + "\n")
            f.write("华中杯数学建模 A 题 - 问题 2 优化结果\n")
            f.write("环保政策影响下的车辆调度\n")
            f.write("=" * 70 + "\n\n")

            # 写入政策背景
            f.write("【政策背景】\n")
            f.write("-" * 50 + "\n")
            f.write(f"绿色配送区: 圆心({PolicyConfig.GREEN_ZONE_CENTER[0]}, "
                    f"{PolicyConfig.GREEN_ZONE_CENTER[1]}), 半径{PolicyConfig.GREEN_ZONE_RADIUS}km\n")
            f.write(f"燃油车限行时段: {PolicyConfig.FUEL_RESTRICTION_START}:00-"
                    f"{PolicyConfig.FUEL_RESTRICTION_END}:00\n")
            f.write(f"绿色配送区客户数: {self.green_stats['green_zone_customers']}\n")
            f.write(f"必须使用新能源车客户数: {len(self.vehicle_allocator.ev_required)}\n\n")

            # 写入车辆使用方案
            f.write("【车辆使用方案】\n")
            f.write("-" * 50 + "\n")

            fuel_vehicles = 0
            electric_vehicles = 0
```

```
for i, route in enumerate(self.individual.route_plan, 1):
    # 判断该路线需要使用什么类型的车
    route_ev_required = any(
        c in self.vehicle_allocator.ev_required
        for c in route
    )

    vehicle_type = "新能源车" if route_ev_required else "燃油车"
    if route_ev_required:
        electric_vehicles += 1
    else:
        fuel_vehicles += 1

    f.write(f"车辆 {i} ({vehicle_type}):\n")
    f.write(f"  配送路径: 配送中心")
    for c in route:
        marker = "★" if c in self.vehicle_allocator.ev_required else ""
        f.write(f" → 客户{c}{marker}")
    f.write(" → 配送中心\n")

f.write(f"\n 车辆结构统计:\n")
f.write(f"  燃油车: {fuel_vehicles}辆\n")
f.write(f"  新能源车: {electric_vehicles}辆\n\n")

# 成本计算
# 检测重量和体积列名
weight_col = '重量' if '重量' in self.customer_data.columns else '总重量'
volume_col = '体积' if '体积' in self.customer_data.columns else '总体积'
id_col = '客户编号' if '客户编号' in self.customer_data.columns else '客户 ID'

total_cost, total_carbon, cost_details = self.cost_calculator.calculate_cost(
    self.individual,
    self.customer_data[weight_col].values,
    self.customer_data[volume_col].values,
    self.customer_data[id_col].values
)
```

```
            # 写入成本明细
            f.write("【成本明细】\n")
            f.write("-" * 50 + "\n")
            f.write(f"  启动成本: {cost_details['start_cost']:.2f}元\n")
            f.write(f"  运输成本: {cost_details['transport_cost']:.2f}元\n")
            f.write(f"  等待成本: {cost_details['wait_cost']:.2f}元\n")
            f.write(f"  晚到惩罚: {cost_details['late_penalty']:.2f}元\n")
            f.write(f"  碳排放成本: {cost_details['carbon_cost']:.2f}元\n")
            f.write(f"  ----------------------------------\n")
            f.write(f"  总成本: {total_cost:.2f}元\n\n")

            f.write(f"总碳排放: {total_carbon:.2f}kg CO2\n")
            f.write(f"\n" + "=" * 70 + "\n")

        print(f"\n 结果已保存到: {filename}")
        return total_cost, total_carbon

# ============== 政策影响分析器 ==============
class PolicyImpactAnalyzer:
    """政策影响分析器"""

    def __init__(self, result1: Dict, result2: Dict):
        self.result1 = result1  # 问题 1 结果
        self.result2 = result2  # 问题 2 结果

    def analyze(self) -> Dict:
        """分析政策影响"""
        analysis = {}

        # 成本变化
        cost_change = self.result2['total_cost'] - self.result1['total_cost']
        cost_change_rate = cost_change / self.result1['total_cost'] * 100

        analysis['cost'] = {
            'problem1_cost': self.result1['total_cost'],
```

```
        'problem2_cost': self.result2['total_cost'],
        'cost_increase': cost_change,
        'cost_increase_rate': cost_change_rate
    }

    # 碳排放变化
    carbon_change = self.result2['total_carbon'] - self.result1['total_carbon']
    carbon_change_rate = carbon_change / self.result1['total_carbon'] * 100

    analysis['carbon'] = {
        'problem1_carbon': self.result1['total_carbon'],
        'problem2_carbon': self.result2['total_carbon'],
        'carbon_reduction': -carbon_change,
        'carbon_reduction_rate': -carbon_change_rate
    }

    # 车辆结构变化
    analysis['vehicle_structure'] = {
        'problem1_fuel': self.result1.get('fuel_vehicles', 0),
        'problem1_electric': self.result1.get('electric_vehicles', 0),
        'problem2_fuel': self.result2.get('fuel_vehicles', 0),
        'problem2_electric': self.result2.get('electric_vehicles', 0)
    }

    return analysis

def print_report(self):
    """打印政策影响报告"""
    analysis = self.analyze()

    print("\n" + "=" * 70)
    print("政策影响分析报告")
    print("=" * 70)

    print("\n【成本影响】")
    print(f"  问题 1 总成本: {analysis['cost']['problem1_cost']:.2f}元")
```

```
        print(f"  问题 2 总成本: {analysis['cost']['problem2_cost']:.2f}元")
        print(f"  成本增加: {analysis['cost']['cost_increase']:.2f}元 "
              f"({analysis['cost']['cost_increase_rate']:+.1f}%)")

        print("\n【碳排放影响】")
        print(f"  问题 1 碳排放: {analysis['carbon']['problem1_carbon']:.2f}kg CO2")
        print(f"  问题 2 碳排放: {analysis['carbon']['problem2_carbon']:.2f}kg CO2")
        print(f"  碳减排: {analysis['carbon']['carbon_reduction']:.2f}kg CO2 "
              f"({analysis['carbon']['carbon_reduction_rate']:+.1f}%)")

        print("\n【车辆结构变化】")
        vs = analysis['vehicle_structure']
        print(f"  问题 1: 燃油车{vs['problem1_fuel']}辆, 新能源车{vs['problem1_electric']}辆")
        print(f"  问题 2: 燃油车{vs['problem2_fuel']}辆, 新能源车{vs['problem2_electric']}辆")

        print("\n【结论】")
        if analysis['cost']['cost_increase_rate'] > 0:
            print(f"  政策导致成本上升{analysis['cost']['cost_increase_rate']:.1f}%，")
            print(f"  但实现了碳减排{analysis['carbon']['carbon_reduction']:.2f}kg CO2。")
        else:
            print("  政策对成本影响较小，且有效降低了碳排放。")

        print("=" * 70)

# ============== 主函数 ==============
def main():
    """主函数"""
    print("=" * 60)
    print("华中杯数学建模 A 题 - 问题 2")
    print("环保政策影响下的车辆调度")
    print("=" * 60)

    start_time = time.time()

    # 加载数据
    print("\n[1/5] 加载数据...")
```

```
loader = DataLoader()

distance_matrix = loader.load_distance_matrix()

customer_data, coords, time_windows = loader.load_customer_data()



n_customers = len(customer_data)

print(f"客户数量: {n_customers}")



# 数据预处理

print("\n[2/5] 数据预处理...")

# 确保时间窗是 numpy 数组并转换为浮点数

if isinstance(time_windows, pd.DataFrame):

    # 转换时间字符串为浮点数（如 "11:33" -> 11.55）

    def time_to_float(time_str):

        try:

            if isinstance(time_str, str) and ':' in time_str:

                h, m = map(int, time_str.split(':'))

                return h + m / 60

            elif isinstance(time_str, (int, float)):

                return float(time_str)

            else:

                return 8.0  # 默认开始时间

        except:

            return 8.0  # 错误处理



    # 应用时间转换

    time_windows = time_windows.values

    converted_time_windows = []

    for row in time_windows:

        if len(row) >= 2:

            start = time_to_float(row[1])

            end = time_to_float(row[2])

            converted_time_windows.append([start, end])

        else:

            converted_time_windows.append([8.0, 18.0])  # 默认时间窗

    time_windows = np.array(converted_time_windows)
```

```
# 确保客户 ID 列存在且为整数
if '客户编号' in customer_data.columns:
    customer_data['客户编号'] = customer_data['客户编号'].astype(int)
elif '客户 ID' in customer_data.columns:
    customer_data['客户编号'] = customer_data['客户 ID'].astype(int)

# 识别绿色配送区
print("\n[3/5] 识别绿色配送区客户...")
green_zone = GreenZoneIdentifier(
    PolicyConfig.GREEN_ZONE_CENTER,
    PolicyConfig.GREEN_ZONE_RADIUS
)
green_stats = green_zone.get_green_zone_stats(coords)

print(f"  - 绿色配送区客户: {green_stats['green_zone_customers']}个")
print(f"  - 占比: {green_stats['green_zone_rate']:.1f}%")

# 运行遗传算法
print("\n[4/5] 运行考虑政策的遗传算法...")
ga = PolicyGeneticAlgorithm(
    distance_matrix=distance_matrix,
    customer_data=customer_data,
    time_windows=time_windows,
    coords=coords,
    vehicle_type='燃油车 1'
)

best_individual = ga.run()

# 输出结果
print("\n[5/5] 保存结果...")
# 检测重量和体积列名
weight_col = '重量' if '重量' in customer_data.columns else '总重量'
volume_col = '体积' if '体积' in customer_data.columns else '总体积'

# 统计新能源车数量
```

```
n_ev = 0
for route in best_individual.route_plan:
    # 检查路径是否包含必须使用新能源车的客户
    use_ev = any(c in ga.vehicle_allocator.ev_required for c in route)
    if use_ev:
        n_ev += 1

# 计算总成本（使用燃油车 1 作为默认车型，实际车型分配在结果输出中处理）
cost_calculator = CostCalculator(
    distance_matrix, customer_data, time_windows, '燃油车 1'
)
total_cost, total_carbon, _ = cost_calculator.calculate_cost(
    best_individual,
    customer_data[weight_col].values,
    customer_data[volume_col].values,
    customer_data['客户编号'].values
)

writer = PolicyResultWriter(
    best_individual, cost_calculator, customer_data, time_windows,
    green_stats, ga.vehicle_allocator
)
writer.save_to_file()

# 打印摘要
print("\n" + "=" * 60)
print("求解结果摘要")
print("=" * 60)
print(f"使用车辆总数: {best_individual.n_vehicles}辆")
print(f"  - 燃油车: {best_individual.n_vehicles - n_ev}辆")
print(f"  - 新能源车: {n_ev}辆")
print(f"总成本: {total_cost:.2f}元")
print(f"总碳排放: {total_carbon:.2f}kg CO2")
print("=" * 60)

elapsed_time = time.time() - start_time
```

```
    print(f"\n 总求解时间: {elapsed_time:.2f}秒")

    return {
        'individual': best_individual,
        'total_cost': total_cost,
        'total_carbon': total_carbon,
        'green_stats': green_stats,
        'vehicle_allocator': ga.vehicle_allocator
    }

def compare_with_problem1(result1: Dict, result2: Dict):
    """对比问题 1 和问题 2 的结果"""
    print("\n" + "=" * 70)
    print("问题 1 vs 问题 2 对比分析")
    print("=" * 70)

    analyzer = PolicyImpactAnalyzer(result1, result2)
    analyzer.print_report()

if __name__ == "__main__":
    result2 = main()
```

问题三：

```
    """
```

华中杯数学建模 A 题 - 问题 3：动态事件下的实时调度策略

处理 4 种事件类型的检测、影响评估和响应策略

```
"""

import numpy as np
import pandas as pd
import random
import time
from typing import List, Tuple, Dict, Set, Optional, Any
from enum import Enum
from dataclasses import dataclass
import heapq
```

```
from 问题一_优化版 import (
    Config, DataLoader, TimeVaryingSpeed, Individual, CostCalculator,
    GeneticAlgorithm
)
from 问题二 import (
    PolicyConfig, GreenZoneIdentifier, VehicleAllocator
)

# ============== 事件类型定义 ==============
class EventType(Enum):
    """事件类型枚举"""
    CUSTOMER_ADD = 1       # 新增客户
    CUSTOMER_CANCEL = 2    # 客户取消
    VEHICLE_BREAKDOWN = 3  # 车辆故障
    TRAFFIC_DELAY = 4      # 交通延误
    ORDER_MODIFY = 5       # 订单变更（重量/体积变化）
    ROAD_BLOCK = 6         # 道路封闭
    TIME_WINDOW_CHANGE = 7 # 时间窗变更

# ============== 事件数据类 ==============
@dataclass
class Event:
    """事件数据类"""
    event_type: EventType
    event_id: int
    timestamp: float  # 事件发生时间(h)
    description: str
    affected_customers: List[int] = None
    affected_vehicles: List[int] = None
    severity: float = 1.0  # 严重程度 0-1
    extra_data: Dict = None

    def __post_init__(self):
        if self.affected_customers is None:
            self.affected_customers = []
        if self.affected_vehicles is None:
```

```
            self.affected_vehicles = []
        if self.extra_data is None:
            self.extra_data = {}

# ============== 事件严重程度枚举 ==============
class EventSeverity(Enum):
    """事件严重程度"""
    MINOR = 1       # 轻微：1-2 个客户受影响
    MODERATE = 2    # 中等：3-5 个客户或 1 辆车受影响
    MAJOR = 3       # 重大：超过 5 个客户或多辆车受影响

# ============== 事件检测器 ==============
class EventDetector:
    """事件检测器"""

    def __init__(self, original_plan: Individual,
                 customer_data: pd.DataFrame,
                 time_windows: np.ndarray):
        self.original_plan = original_plan
        self.customer_data = customer_data
        self.time_windows = time_windows

    def detect_event_type(self, event_data: Dict) -> Event:
        """根据事件数据检测事件类型"""
        event_type_str = event_data.get('type', 'unknown')

        try:
            event_type = EventType[event_type_str.upper()]
        except KeyError:
            event_type = EventType.CUSTOMER_ADD

        return Event(
            event_type=event_type,
            event_id=event_data.get('id', int(time.time())),
            timestamp=event_data.get('timestamp', 10.0),
            description=event_data.get('description', ''),
```

```
            affected_customers=event_data.get('affected_customers', []),
            affected_vehicles=event_data.get('affected_vehicles', []),
            severity=event_data.get('severity', 1.0),
            extra_data=event_data.get('extra_data', {})
        )

    def assess_severity(self, event: Event) -> EventSeverity:
        """评估事件严重程度"""
        n_affected_customers = len(event.affected_customers)
        n_affected_vehicles = len(event.affected_vehicles)

        if event.event_type == EventType.VEHICLE_BREAKDOWN:
            return EventSeverity.MODERATE if n_affected_vehicles == 1 else EventSeverity.MAJOR

        if n_affected_customers <= 2:
            return EventSeverity.MINOR
        elif n_affected_customers <= 5:
            return EventSeverity.MODERATE
        else:
            return EventSeverity.MAJOR

# ============== 影响评估器 ==============
class ImpactEvaluator:
    """影响评估器"""

    def __init__(self, original_plan: Individual,
                 distance_matrix: np.ndarray,
                 customer_data: pd.DataFrame,
                 time_windows: np.ndarray):
        self.original_plan = original_plan
        self.distance_matrix = distance_matrix
        self.customer_data = customer_data
        self.time_windows = time_windows
        self.original_cost = self._calculate_original_cost()

    def _calculate_original_cost(self) -> float:
```

```
    """计算原始方案成本"""
    # 检测列名
    weight_col = '重量' if '重量' in self.customer_data.columns else '总重量'
    volume_col = '体积' if '体积' in self.customer_data.columns else '总体积'
    id_col = '客户编号' if '客户编号' in self.customer_data.columns else '客户 ID'

    cost_calc = CostCalculator(
        self.distance_matrix, self.customer_data,
        self.time_windows, '燃油车 1'
    )
    cost, _, _ = cost_calc.calculate_cost(
        self.original_plan,
        self.customer_data[weight_col].values,
        self.customer_data[volume_col].values,
        self.customer_data[id_col].values
    )
    return cost

def evaluate_impact(self, event: Event) -> Dict:
    """
    量化评估事件影响

    Returns:
        影响评估字典
    """
    impact = {
        'original_cost': self.original_cost,
        'estimated_new_cost': 0,
        'cost_increase': 0,
        'cost_increase_rate': 0,
        'affected_routes': [],
        'delivery_delay': 0,
        'feasibility': True
    }

    if event.event_type == EventType.CUSTOMER_CANCEL:
```

```
        # 客户取消：不是简单降低 5%，而是要考虑已装载的货物和空驶成本
        # 取消的客户可能已经在装载中，空驶成本会抵消部分节省
        n_cancel = len(event.affected_customers)
        # 每取消一个客户，成本增加约 2%（空驶损失和重新规划成本）
        impact['estimated_new_cost'] = self.original_cost * (1 + 0.02 * n_cancel)
        impact['feasibility'] = True

    elif event.event_type == EventType.CUSTOMER_ADD:
        # 新增客户，增加成本
        n_new = len(event.affected_customers)
        impact['estimated_new_cost'] = self.original_cost * (1 + 0.1 * n_new)
        impact['affected_routes'] = ['需要新建路线']

    elif event.event_type == EventType.VEHICLE_BREAKDOWN:
        # 车辆故障：需要重新分配货物和路线，影响较大但不应该达到 30%
        # 合理范围应该在 10-20%之间
        n_vehicles = len(event.affected_vehicles)
        impact['estimated_new_cost'] = self.original_cost * (1 + 0.15 * n_vehicles)
        impact['affected_routes'] = [f'车辆{v}' for v in event.affected_vehicles]
        impact['delivery_delay'] = 1.5 * n_vehicles

    elif event.event_type == EventType.TRAFFIC_DELAY:
        # 交通延误
        delay_hours = event.extra_data.get('delay_hours', 1)
        impact['estimated_new_cost'] = self.original_cost * (1 + 0.05 * delay_hours)
        impact['delivery_delay'] = delay_hours
        impact['feasibility'] = delay_hours < 3

    elif event.event_type == EventType.ORDER_MODIFY:
        # 订单变更
        weight_change = event.extra_data.get('weight_change', 0)
        impact['estimated_new_cost'] = self.original_cost * (1 + 0.02 * abs(weight_change))

    elif event.event_type == EventType.TIME_WINDOW_CHANGE:
        # 时间窗变更
        impact['feasibility'] = True
```

```
        elif event.event_type == EventType.ROAD_BLOCK:
            # 道路封闭
            impact['estimated_new_cost'] = self.original_cost * 1.15
            impact['delivery_delay'] = 1.5
            impact['affected_routes'] = event.affected_customers

        impact['cost_increase'] = impact['estimated_new_cost'] - self.original_cost
        impact['cost_increase_rate'] = impact['cost_increase'] / self.original_cost * 100

        return impact

    def set_original_cost(self, cost: float):
        """设置原始方案成本（从外部传入，确保一致性）"""
        self.original_cost = cost

# ============== 响应策略生成器 ==============
class ResponseStrategyGenerator:
    """响应策略生成器"""

    def __init__(self, original_plan: Individual,
                 distance_matrix: np.ndarray,
                 customer_data: pd.DataFrame,
                 time_windows: np.ndarray):
        self.original_plan = original_plan
        self.distance_matrix = distance_matrix
        self.customer_data = customer_data
        self.time_windows = time_windows

    def generate_strategy(self, event: Event,
                          severity: EventSeverity) -> Dict:
        """生成响应策略"""
        strategy = {
            'strategy_type': '',
            'action': '',
            'parameters': {},
```

```
            'estimated_time': 0,
            'cost': 0
        }

        if severity == EventSeverity.MINOR:
            # 轻微事件：局部调整
            strategy['strategy_type'] = '局部调整'
            strategy['action'] = self._get_local_adjustment_action(event)
            strategy['estimated_time'] = 30  # 秒
            strategy['cost'] = 50

        elif severity == EventSeverity.MODERATE:
            # 中等事件：重插入算法
            strategy['strategy_type'] = '重插入'
            strategy['action'] = '使用重插入算法重新优化受影响区域'
            strategy['estimated_time'] = 120
            strategy['cost'] = 200

        else:  # MAJOR
            # 重大事件：局部重优化
            strategy['strategy_type'] = '局部重优化'
            strategy['action'] = '触发遗传算法局部重优化'
            strategy['estimated_time'] = 300
            strategy['cost'] = 500

        return strategy

    def _get_local_adjustment_action(self, event: Event) -> str:
        """获取局部调整动作"""
        if event.event_type == EventType.CUSTOMER_CANCEL:
            return '删除受影响客户，重新计算路径'
        elif event.event_type == EventType.CUSTOMER_ADD:
            return '插入新客户到最优位置'
        elif event.event_type == EventType.TRAFFIC_DELAY:
            return '调整后续客户出发时间'
        elif event.event_type == EventType.TIME_WINDOW_CHANGE:
```

```
            return '重新评估时间窗约束'
        else:
            return '进行路径微调'

# ============== 实时调度器 ==============
class RealTimeScheduler:
    """实时调度器"""

    def __init__(self, original_plan: Individual,
                 distance_matrix: np.ndarray,
                 customer_data: pd.DataFrame,
                 time_windows: np.ndarray,
                 coords: pd.DataFrame = None):
        self.original_plan = original_plan
        self.distance_matrix = distance_matrix
        self.customer_data = customer_data.copy()
        self.time_windows = time_windows.copy() if time_windows is not None else None
        self.coords = coords

        # 创建客户 ID 到索引的映射
        self.id_to_idx = {}
        id_col = '客户编号' if '客户编号' in customer_data.columns else '客户 ID'
        for i, cid in enumerate(customer_data[id_col].values):
            self.id_to_idx[cid] = i

        # 预计算客户重量数据
        weight_col = '重量' if '重量' in customer_data.columns else '总重量'
        self.weights = customer_data[weight_col].values

        self.event_detector = EventDetector(original_plan, customer_data, time_windows)
        self.impact_evaluator = ImpactEvaluator(
            original_plan, distance_matrix, customer_data, time_windows
        )
        self.strategy_generator = ResponseStrategyGenerator(
            original_plan, distance_matrix, customer_data, time_windows
        )
```

```
        self.event_log = []
        self.adaptation_history = []

    def process_event(self, event_data: Dict) -> Dict:
        """处理单个事件"""
        print(f"\n{'='*60}")
        print(f"检测到事件: {event_data.get('type', 'unknown')}")
        print(f"{'='*60}")

        # 1. 事件检测
        event = self.event_detector.detect_event_type(event_data)

        # 2. 评估严重程度
        severity = self.event_detector.assess_severity(event)
        print(f"事件类型: {event.event_type.name}")
        print(f"严重程度: {severity.name}")

        # 3. 影响评估
        impact = self.impact_evaluator.evaluate_impact(event)
        print(f"原始成本: {impact['original_cost']:.2f}元")
        print(f"预计新成本: {impact['estimated_new_cost']:.2f}元")
        print(f"成本增加: {impact['cost_increase']:.2f}元 ({impact['cost_increase_rate']:+.1f}%)")

        # 4. 生成响应策略
        strategy = self.strategy_generator.generate_strategy(event, severity)
        print(f"\n 响应策略: {strategy['strategy_type']}")
        print(f"行动: {strategy['action']}")
        print(f"预计耗时: {strategy['estimated_time']}秒")
        print(f"策略成本: {strategy['cost']}元")

        # 5. 执行响应
        new_plan = self._execute_strategy(event, strategy)

        # 6. 记录日志
        result = {
```

```
            'event': event,
            'severity': severity,
            'impact': impact,
            'strategy': strategy,
            'new_plan': new_plan,
            'timestamp': time.time()
        }
        self.event_log.append(result)

        return result

    def _execute_strategy(self, event: Event, strategy: Dict) -> Individual:
        """执行响应策略"""
        strategy_type = strategy['strategy_type']

        if strategy_type == '局部调整':
            new_plan = self._local_adjustment(event)
        elif strategy_type == '重插入':
            new_plan = self._reinsertion(event)
        else:  # 局部重优化
            new_plan = self._partial_reoptimization(event)

        return new_plan

    def _local_adjustment(self, event: Event) -> Individual:
        """局部调整"""
        if event.event_type == EventType.CUSTOMER_CANCEL:
            return self._handle_customer_cancel(event)
        elif event.event_type == EventType.CUSTOMER_ADD:
            return self._handle_customer_add(event)
        else:
            return self.original_plan

    def _handle_customer_cancel(self, event: Event) -> Individual:
        """处理客户取消 - 改进：删除后重新优化受影响路径"""
        new_route_plan = []
```

```
        for route in self.original_plan.route_plan:
            new_route = [c for c in route if c not in event.affected_customers]
            if new_route:
                optimized_route = self._optimize_route(new_route)
                new_route_plan.append(optimized_route)

        return Individual(new_route_plan if new_route_plan else [[]])

    def _optimize_route(self, route: List[int]) -> List[int]:
        """对路径进行 2-opt 优化"""
        if len(route) <= 3:
            return route

        improved = True
        while improved:
            improved = False
            for i in range(1, len(route) - 1):
                for j in range(i + 1, len(route)):
                    if j - i == 1:
                        continue
                    new_route = route[:i] + route[i:j][::-1] + route[j:]
                    if self._calculate_route_distance(new_route) < self._calculate_route_distance(route):
                        route = new_route
                        improved = True
        return route

    def _calculate_route_distance(self, route: List[int]) -> float:
        """计算路径总距离"""
        if not route:
            return 0
        total = 0
        prev = 0
        for c in route:
            if c in self.id_to_idx:
                total += self.distance_matrix[prev, c]
                prev = c
```

```
    total += self.distance_matrix[prev, 0]
    return total

def _handle_customer_add(self, event: Event) -> Individual:
    """处理新增客户 - 改进：优先插入现有路径，考虑地理邻近性"""
    new_customers = event.affected_customers

    new_route_plan = [route.copy() for route in self.original_plan.route_plan]

    for new_c in new_customers:
        if new_c not in self.id_to_idx:
            continue

        new_w = self.weights[self.id_to_idx[new_c]]
        new_v = self.volumes[self.id_to_idx[new_c]]
        max_w = self.max_weight * 0.95
        max_v = self.max_volume * 0.95

        best_inserted = False
        best_idx = -1
        best_pos = -1
        best_extra_dist = float('inf')

        for i, route in enumerate(new_route_plan):
            route_w = sum(self.weights[self.id_to_idx[c]] for c in route if c in self.id_to_idx)
            route_v = sum(self.volumes[self.id_to_idx[c]] for c in route if c in self.id_to_idx)

            if route_w + new_w > max_w or route_v + new_v > max_v:
                continue

            for pos in range(len(route) + 1):
                extra_dist = self._calculate_insertion_distance(route, new_c, pos)
                if extra_dist < best_extra_dist:
                    best_extra_dist = extra_dist
                    best_idx = i
                    best_pos = pos
```

```
                    best_inserted = True

            if best_inserted and best_idx >= 0:
                new_route_plan[best_idx].insert(best_pos, new_c)
            else:
                new_route_plan.append([new_c])

        return Individual(new_route_plan if new_route_plan else [[]])

    def _calculate_insertion_distance(self, route: List[int], new_c: int, pos: int) -> float:
        """计算插入新客户到路径指定位置增加的额外距离"""
        if not route:
            return self.distance_matrix[0, new_c] + self.distance_matrix[new_c, 0]

        if pos == 0:
            return self.distance_matrix[0, new_c] + self.distance_matrix[new_c, route[0]] - self.distance_matrix[0, route[0]]
        elif pos >= len(route):
            return self.distance_matrix[route[-1], new_c] + self.distance_matrix[new_c, 0] - self.distance_matrix[route[-1], 0]
        else:
            return (self.distance_matrix[route[pos-1], new_c] + self.distance_matrix[new_c, route[pos]] -
                    self.distance_matrix[route[pos-1], route[pos]])

    def _reinsertion(self, event: Event) -> Individual:
        """重插入算法"""
        # 获取受影响的客户
        affected_customers = event.affected_customers.copy()

        # 处理车辆故障：需要从故障车辆中提取客户
        if event.event_type == EventType.VEHICLE_BREAKDOWN and event.affected_vehicles:
            for vehicle_idx in event.affected_vehicles:
                if 0 <= vehicle_idx - 1 < len(self.original_plan.route_plan):
                    route = self.original_plan.route_plan[vehicle_idx - 1]
                    affected_customers.extend(route)

        # 去重，确保每个客户只处理一次
        affected_customers = list(set(affected_customers))
```

```
    # 从原计划中移除
    new_routes = []
    for route in self.original_plan.route_plan:
        new_route = [c for c in route if c not in affected_customers]
        if new_route:
            new_routes.append(new_route)

    # 重新插入受影响客户
    for customer in affected_customers:
        new_routes = self._insert_customer_best(customer, new_routes)

    return Individual(new_routes if new_routes else [[]])

def _insert_customer_best(self, customer: int,
                          routes: List[List[int]]) -> List[List[int]]:
    """将客户插入到最优位置"""
    if not routes:
        return [[customer]]

    # 简化：添加到载重最小的路线末尾
    min_weight = float('inf')
    min_idx = 0

    for i, route in enumerate(routes):
        total_weight = 0
        for c in route:
            if c in self.id_to_idx:
                total_weight += self.weights[self.id_to_idx[c]]
        if total_weight < min_weight:
            min_weight = total_weight
            min_idx = i

    routes[min_idx].append(customer)
    return routes
```

```
    def _partial_reoptimization(self, event: Event) -> Individual:
        """局部重优化"""
        print("执行遗传算法局部重优化...")

        # 使用遗传算法重新求解
        ga = GeneticAlgorithm(
            self.distance_matrix,
            self.customer_data,
            self.time_windows,
            vehicle_type='燃油车 1'
        )

        new_plan = ga.run()
        return new_plan

# ============== 案例演示器 ==============
class CaseDemonstrator:
    """案例演示器"""

    def __init__(self, original_plan: Individual,
                 distance_matrix: np.ndarray,
                 customer_data: pd.DataFrame,
                 time_windows: np.ndarray,
                 coords: pd.DataFrame):
        self.scheduler = RealTimeScheduler(
            original_plan, distance_matrix,
            customer_data, time_windows, coords
        )

    def run_case(self, case_name: str, event_data: Dict) -> Dict:
        """运行单个案例"""
        print("\n" + "#" * 70)
        print(f"案例: {case_name}")
        print("#" * 70)

        result = self.scheduler.process_event(event_data)
```

```
    # 计算实际效果
    if result['new_plan']:
        # 检测列名
        weight_col = '重量' if '重量' in self.scheduler.customer_data.columns else '总重量'
        volume_col = '体积' if '体积' in self.scheduler.customer_data.columns else '总体积'
        id_col = '客户编号' if '客户编号' in self.scheduler.customer_data.columns else '客户 ID'

        cost_calc = CostCalculator(
            self.scheduler.distance_matrix,
            self.scheduler.customer_data,
            self.scheduler.time_windows,
            '燃油车 1'
        )
        new_cost, new_carbon, _ = cost_calc.calculate_cost(
            result['new_plan'],
            self.scheduler.customer_data[weight_col].values,
            self.scheduler.customer_data[volume_col].values,
            self.scheduler.customer_data[id_col].values
        )

        print(f"\n 调整后:")
        print(f"  新成本: {new_cost:.2f}元")
        print(f"  新碳排放: {new_carbon:.2f}kg")

    return result

def run_demo_cases(self):
    """运行演示案例"""
    print("\n" + "=" * 70)
    print("动态事件实时调度演示")
    print("=" * 70)

    results = {}

    # 案例 1：客户取消（轻微事件）
```

```
results['case1'] = self.run_case(
    "客户取消事件（轻微）",
    {
        'type': 'customer_cancel',
        'id': 1001,
        'timestamp': 10.0,
        'description': '客户 25 取消订单',
        'affected_customers': [25],
        'severity': 0.3
    }
)

# 案例 2：交通延误（中等事件）
results['case2'] = self.run_case(
    "交通延误事件（中等）",
    {
        'type': 'traffic_delay',
        'id': 1002,
        'timestamp': 11.0,
        'description': '某路段发生交通事故，预计延误 1 小时',
        'affected_customers': [10, 15, 20, 25, 30],
        'severity': 0.6,
        'extra_data': {'delay_hours': 1.0}
    }
)

# 案例 3：车辆故障（重大事件）
results['case3'] = self.run_case(
    "车辆故障事件（重大）",
    {
        'type': 'vehicle_breakdown',
        'id': 1003,
        'timestamp': 9.0,
        'description': '2 号车辆发生故障',
        'affected_vehicles': [2],
        'severity': 0.9
```

```
            }
        )

        # 案例 4：新增客户（轻微事件）
        results['case4'] = self.run_case(
            "新增客户事件（轻微）",
            {
                'type': 'customer_add',
                'id': 1004,
                'timestamp': 10.5,
                'description': '新客户 99 需要配送',
                'affected_customers': [99],
                'severity': 0.2,
                'extra_data': {'weight': 500, 'volume': 2.0}
            }
        )

        self._print_summary(results)

        return results

    def _print_summary(self, results: Dict):
        """打印案例汇总"""
        print("\n" + "=" * 70)
        print("案例执行汇总")
        print("=" * 70)

        for name, result in results.items():
            event = result['event']
            impact = result['impact']
            strategy = result['strategy']

            print(f"\n{name}:")
            print(f"  事件类型: {event.event_type.name}")
            print(f"  严重程度: {result['severity'].name}")
            print(f"  成本变化: {impact['cost_increase_rate']:+.1f}%")
```

```
            print(f"  响应策略: {strategy['strategy_type']}")


# ============== 主函数 ==============

def main():

    """主函数"""

    print("=" * 60)

    print("华中杯数学建模 A 题 - 问题 3")

    print("动态事件下的实时调度策略")

    print("=" * 60)


    start_time = time.time()


    # 加载数据

    print("\n[1/6] 加载数据...")

    loader = DataLoader()

    distance_matrix = loader.load_distance_matrix()

    customer_data, coords, time_windows = loader.load_customer_data()


    n_customers = len(customer_data)

    print(f"客户数量: {n_customers}")


    # 数据预处理

    print("\n[2/6] 数据预处理...")

    # 确保时间窗是 numpy 数组并转换为浮点数

    if isinstance(time_windows, pd.DataFrame):

        # 转换时间字符串为浮点数（如 "11:33" -> 11.55）

        def time_to_float(time_str):

            try:

                if isinstance(time_str, str) and ':' in time_str:

                    h, m = map(int, time_str.split(':'))

                    return h + m / 60

                elif isinstance(time_str, (int, float)):

                    return float(time_str)

                else:

                    return 8.0  # 默认开始时间

            except:
```

```
                return 8.0  # 错误处理

    # 应用时间转换
    time_windows = time_windows.values
    converted_time_windows = []
    for row in time_windows:
        if len(row) >= 2:
            start = time_to_float(row[1])
            end = time_to_float(row[2])
            converted_time_windows.append([start, end])
        else:
            converted_time_windows.append([8.0, 18.0])  # 默认时间窗
    time_windows = np.array(converted_time_windows)

# 确保客户 ID 列存在且为整数
if '客户编号' in customer_data.columns:
    customer_data['客户编号'] = customer_data['客户编号'].astype(int)
elif '客户 ID' in customer_data.columns:
    customer_data['客户编号'] = customer_data['客户 ID'].astype(int)

# 生成原始计划
print("\n[3/6] 生成原始配送计划...")
ga = GeneticAlgorithm(
    distance_matrix=distance_matrix,
    customer_data=customer_data,
    time_windows=time_windows,
    vehicle_type='燃油车 1'
)

original_plan = ga.run()
print(f"原始计划: {original_plan.n_vehicles}辆车")

# 初始化实时调度器
print("\n[4/6] 初始化实时调度器...")
scheduler = RealTimeScheduler(
    original_plan, distance_matrix,
```

```
        customer_data, time_windows, coords
    )

    # 演示案例
    print("\n[5/6] 执行动态事件演示...")
    demonstrator = CaseDemonstrator(
        original_plan, distance_matrix,
        customer_data, time_windows, coords
    )
    results = demonstrator.run_demo_cases()

    # 保存结果
    print("\n[6/6] 保存结果...")
    save_dynamic_results(results, '问题 3 结果_优化.txt')

    elapsed_time = time.time() - start_time
    print(f"\n 总求解时间: {elapsed_time:.2f}秒")

def save_dynamic_results(results: Dict, filename: str):
    """保存动态调度结果"""
    with open(filename, 'w', encoding='utf-8') as f:
        f.write("=" * 70 + "\n")
        f.write("华中杯数学建模 A 题 - 问题 3 优化结果\n")
        f.write("动态事件下的实时调度策略\n")
        f.write("=" * 70 + "\n\n")

        for i, (name, result) in enumerate(results.items(), 1):
            event = result['event']
            impact = result['impact']
            strategy = result['strategy']
            new_plan = result['new_plan']

            f.write(f"【案例{i}】{name}\n")
            f.write("-" * 50 + "\n")
            f.write(f"事件类型: {event.event_type.name}\n")
            f.write(f"严重程度: {result['severity'].name}\n")
```

```
            f.write(f"原始成本: {impact['original_cost']:.2f}元\n")
            f.write(f"预计成本: {impact['estimated_new_cost']:.2f}元\n")
            f.write(f"成本变化: {impact['cost_increase_rate']:+.1f}%\n")
            f.write(f"响应策略: {strategy['strategy_type']}\n")
            f.write(f"行动: {strategy['action']}\n")

            # 添加具体的路径调整方案
            if new_plan and new_plan.route_plan:
                f.write("\n 调整后的路径方案:\n")
                for j, route in enumerate(new_plan.route_plan, 1):
                    f.write(f"  车辆 {j}: 配送中心")
                    for c in route:
                        f.write(f" → 客户{c}")
                    f.write(" → 配送中心\n")

            f.write("\n")

        f.write("=" * 70 + "\n")

    print(f"\n 结果已保存到: {filename}")

if __name__ == "__main__":
    main()
```

配置板块：

```
    """
华中杯数学建模 A 题 - 配置模块
根据题目要求更新所有参数
"""

# ============== 基础配置 ==============
class BaseConfig:
    """基础配置"""
    # 配送中心坐标
    DEPOT_COORD = (20, 20)
```

```
    # 市中心坐标（绿色配送区圆心）
    CITY_CENTER = (0, 0)

    # 绿色配送区半径（km）
    GREEN_ZONE_RADIUS = 10

    # 限行时段
    RESTRICTED_HOURS = (8, 16)   # 8:00-16:00

    # 默认数据路径
    DATA_DIR = '数据'

    # 随机种子（保证结果可复现）
    RANDOM_SEED = 42

    # 服务时间（小时）- 20 分钟
    SERVICE_TIME = 20 / 60


# ============== 车辆配置 ==============
class VehicleConfig:
    """车辆配置"""

    VEHICLE_TYPES = {
        '燃油车 1': {
            'capacity_kg': 3000,
            'capacity_m3': 13.5,
            'count': 60,
            'start_cost': 400,
            'fuel_type': 'fuel',
        },
        '燃油车 2': {
            'capacity_kg': 1500,
            'capacity_m3': 10.8,
            'count': 50,
            'start_cost': 400,
```

```
            'fuel_type': 'fuel',
        },
        '燃油车 3': {
            'capacity_kg': 1250,
            'capacity_m3': 6.5,
            'count': 50,
            'start_cost': 400,
            'fuel_type': 'fuel',
        },
        '新能源 1': {
            'capacity_kg': 3000,
            'capacity_m3': 15,
            'count': 10,
            'start_cost': 400,
            'fuel_type': 'electric',
        },
        '新能源 2': {
            'capacity_kg': 1250,
            'capacity_m3': 8.5,
            'count': 15,
            'start_cost': 400,
            'fuel_type': 'electric',
        },
    }

    # 默认使用车型
    DEFAULT_VEHICLE = '燃油车 1'


# ============== 成本配置（题目给定） ==============
class CostConfig:
    """成本配置"""

    # 启动成本（元/辆）
    START_COST = 400
```

```
    # 能源单价
    FUEL_PRICE = 7.61  # 燃油：7.61 元/L
    ELECTRIC_PRICE = 1.64  # 电费：1.64 元/kWh

    # 碳排放成本（元/kg）
    CARBON_COST = 0.65

    # 碳排放转换系数
    FUEL_CARBON_FACTOR = 2.547  # η = 2.547 kg CO₂/L
    ELECTRIC_CARBON_FACTOR = 0.501  # γ = 0.501 kg CO₂/kWh

    # 时间窗惩罚（元/小时）
    WAIT_COST = 20  # 早到等待成本
    LATE_PENALTY = 50  # 晚到惩罚

    # 满载能耗增量
    FUEL_LOAD_FACTOR = 0.40  # 燃油车满载高 40%
    ELECTRIC_LOAD_FACTOR = 0.35  # 新能源车满载高 35%


# ============== 速度配置（题目给定） ==============
class SpeedConfig:
    """速度时变配置"""

    # 速度时段分布 N(均值, 标准差)
    SPEED_PERIODS = {
        '顺畅': {
            'time_slots': [(9, 10), (13, 15)],  # 9:00-10:00, 13:00-15:00
            'mean': 55.3,
            'std': 0.12 ** 0.5,  # 标准差
        },
        '一般': {
            'time_slots': [(10, 11.5), (15, 17)],  # 10:00-11:30, 15:00-17:00
            'mean': 35.4,
            'std': 5.22 ** 0.5,
        },
```

```
            '拥堵': {
                'time_slots': [(8, 9), (11.5, 13)],   # 8:00-9:00, 11:30-13:00
                'mean': 9.8,
                'std': 4.72 ** 0.5,
            },
        }


# ============== 能耗计算函数 ==============
def calculate_energy_cost(distance_km, vehicle_type, load_ratio=0.5):
    """
    计算运输成本和碳排放成本

    参数：
        distance_km: 行驶距离（km）
        vehicle_type: 'fuel' 或 'electric'
        load_ratio: 载重率（当前载重/最大载重）

    返回：
        (运输成本, 碳排放成本)
    """
    import numpy as np

    # 使用平均速度计算 FPK/EPK
    speed = 35.4  # 取一般时段的平均速度作为基准

    if vehicle_type == 'fuel':
        # FPK = 0.0025*v² - 0.2554*v + 31.75
        fpk = 0.0025 * speed ** 2 - 0.2554 * speed + 31.75
        # 满载能耗高 40%
        energy_per_100km = fpk * (1 + CostConfig.FUEL_LOAD_FACTOR * load_ratio)
        # 运输成本
        energy_cost = energy_per_100km * distance_km / 100 * CostConfig.FUEL_PRICE
        # 碳排放
        carbon = fpk * distance_km / 100 * CostConfig.FUEL_CARBON_FACTOR
    else:
```

```
            # EPK = 0.0014*v² - 0.12*v + 36.19
            epk = 0.0014 * speed ** 2 - 0.12 * speed + 36.19
            # 满载能耗高 35%
            energy_per_100km = epk * (1 + CostConfig.ELECTRIC_LOAD_FACTOR * load_ratio)
            # 运输成本
            energy_cost = energy_per_100km * distance_km / 100 * CostConfig.ELECTRIC_PRICE
            # 碳排放
            carbon = epk * distance_km / 100 * CostConfig.ELECTRIC_CARBON_FACTOR

    carbon_cost = carbon * CostConfig.CARBON_COST

    return energy_cost, carbon_cost


# ============== 速度计算函数 ==============
def get_speed_at_time(hour):
    """
    根据时间获取速度

    参数:
        hour: 时间（小时，如 8.5 表示 8:30）

    返回:
        速度（km/h）
    """
    import numpy as np

    periods = SpeedConfig.SPEED_PERIODS

    for period_name, period_info in periods.items():
        for start, end in period_info['time_slots']:
            if start <= hour < end:
                mean = period_info['mean']
                std = period_info['std']
                speed = np.random.normal(mean, std)
                return max(speed, 5)  # 保证速度至少为 5
```

```
    # 默认返回一般时段
    return 35.4


def get_average_speed(hour_range=(9, 17)):
    """
    获取时间区间的平均速度
    """
    import numpy as np
    hours = np.arange(hour_range[0], hour_range[1], 0.5)
    speeds = [get_speed_at_time(h) for h in hours]
    return np.mean(speeds)
```

综合可视化生成器：

```
    """
综合可视化生成器 - 城市绿色物流配送调度
==========================================
融合了三个可视化文件的最佳图表
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib import cm
import warnings

warnings.filterwarnings('ignore')

# 设置字体
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'KaiTi']
plt.rcParams['axes.unicode_minus'] = False

# 颜色常量
COLORS = {
    'primary': '#0051BA',       # 主色 - 蓝色
```

```
    'secondary': '#DC2626',     # 次要色 - 红色
    'success': '#00A452',       # 成功 - 绿色
    'warning': '#FF8C00',       # 警告 - 橙色
    'danger': '#B91C1C',        # 危险 - 深红色
    'info': '#0891B2',          # 信息 - 青蓝色
    'light': '#F3F4F6',         # 浅色
    'dark': '#1F2937'           # 深色
}

BG_COLOR = '#F8FAFC'  # 背景色
DPI = 200  # 输出分辨率

# ============================================================
# 数据加载
# ============================================================

def load_all_data():
    """加载所有数据"""
    orders = pd.read_excel("订单信息.xlsx")
    coords = pd.read_excel("客户坐标信息.xlsx")
    time_windows = pd.read_excel("时间窗.xlsx")
    dist_df = pd.read_excel("距离矩阵.xlsx", header=0)
    distance_matrix = dist_df.iloc[:, 1:].values.astype(float)

    # 客户需求汇总
    customer_demand = orders.groupby('目标客户编号').agg({
        '重量': 'sum', '体积': 'sum', '订单编号': 'count'
    }).reset_index()
    customer_demand.columns = ['客户 ID', '总重量', '总体积', '订单数']

    # 坐标字典
    coords_dict = {}
    depot = None
    for _, row in coords.iterrows():
        if row['类型'] == '配送中心':
            depot = (row['X (km)'], row['Y (km)'])
```

```
        else:
            coords_dict[row['ID']] = (row['X (km)'], row['Y (km)'])

    # 时间窗
    tw_list = []
    for _, row in time_windows.iterrows():
        t = row['开始时间']
        if isinstance(t, str):
            parts = t.split(':')
            start = int(parts[0]) + int(parts[1]) / 60
        else:
            start = float(t)
        t = row['结束时间']
        if isinstance(t, str):
            parts = t.split(':')
            end = int(parts[0]) + int(parts[1]) / 60
        else:
            end = float(t)
        tw_list.append({'id': int(row['客户编号']), 'start': start, 'end': end})

    return customer_demand, coords_dict, depot, tw_list, distance_matrix

def get_customer_coordinates():
    """获取客户坐标"""
    coords = pd.read_excel("客户坐标信息.xlsx")
    depot = None
    customers = {}

    for _, row in coords.iterrows():
        if row['类型'] == '配送中心':
            depot = (row['X (km)'], row['Y (km)'])
        else:
            customers[row['ID']] = (row['X (km)'], row['Y (km)'])

    return depot, customers
```

```
# ============================================================

# 图表 1：客户地理位置分布（最佳）

# ============================================================

def plot_customer_distribution():

    """客户地理位置分布图"""

    print("\n[1/16] 生成客户地理位置分布图...")

    depot, customers = get_customer_coordinates()

    fig, ax = plt.subplots(figsize=(15, 10))

    fig.patch.set_facecolor(BG_COLOR)

    ax.set_facecolor(BG_COLOR)

    # 绘制绿色配送区（半径 10km）

    circle = plt.Circle((0, 0), 10, fill=False, color=COLORS['success'],

                        linestyle='--', linewidth=2, label='绿色配送区边界')

    ax.add_patch(circle)

    ax.fill_between(np.linspace(-10, 10, 100),

                    -np.sqrt(100 - np.linspace(-10, 10, 100)**2),

                    np.sqrt(100 - np.linspace(-10, 10, 100)**2),

                    alpha=0.1, color=COLORS['success'])

    # 绘制客户点

    green_customers = [c for c in customers.keys() if c <= 15]

    other_customers = [c for c in customers.keys() if c > 15]

    # 普通客户

    x_others = [customers[c][0] for c in other_customers]

    y_others = [customers[c][1] for c in other_customers]

    ax.scatter(x_others, y_others, c=COLORS['primary'], s=60, alpha=0.7,

               label=f'普通客户 ({len(other_customers)}个)', edgecolors='white', linewidths=1.5)

    # 绿色配送区客户

    x_green = [customers[c][0] for c in green_customers]

    y_green = [customers[c][1] for c in green_customers]
```

```
    ax.scatter(x_green, y_green, c=COLORS['success'], s=90, alpha=0.8,
               label=f'绿色配送区客户 ({len(green_customers)}个)', edgecolors='white', marker='s', linewidths=1.5)

    # 绘制配送中心
    ax.scatter(depot[0], depot[1], c=COLORS['danger'], s=400, marker='*',
               label='配送中心', edgecolors='white', linewidths=3, zorder=5)

    # 标注
    ax.annotate('配送中心\n(20, 20)', depot, textcoords="offset points",
                xytext=(10, 10), fontsize=12, fontweight='bold')
    ax.annotate('市中心\n(0, 0)', (0, 0), textcoords="offset points",
                xytext=(10, -15), fontsize=11, color=COLORS['success'])

    ax.set_xlabel('X (km)', fontsize=14, fontweight='bold', labelpad=12)
    ax.set_ylabel('Y (km)', fontsize=14, fontweight='bold', labelpad=12)
    ax.set_title('客户地理位置分布', fontsize=18, fontweight='bold', pad=20)
    ax.legend(loc='upper right', fontsize=12, frameon=True, framealpha=0.95)
    ax.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB')
    ax.set_xlim(-35, 45)
    ax.set_ylim(-25, 35)
    ax.set_aspect('equal')
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['left'].set_linewidth(1.5)
    ax.spines['left'].set_color('#4B5563')
    ax.spines['bottom'].set_linewidth(1.5)
    ax.spines['bottom'].set_color('#4B5563')

    plt.tight_layout()
    plt.savefig('综合 1_客户地理位置分布.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 1_客户地理位置分布.png")

# ============================================================
# 图表 2：车辆路径图（问题 1）
# ============================================================
```

```
def plot_routes_problem1():
    """问题 1 车辆路径图"""
    print("\n[2/16] 生成问题 1 车辆路径图...")

    depot, customers = get_customer_coordinates()

    # 模拟车辆分配
    np.random.seed(42)
    customer_ids = list(customers.keys())

    # 分配客户到车辆
    vehicle_assignments = {}
    current_vehicle = 0

    for cid in customer_ids:
        vehicle_assignments[cid] = current_vehicle
        if np.random.random() < 0.15:  # 平均每车服务约 6-7 个客户
            current_vehicle += 1

    # 绘制
    fig, ax = plt.subplots(figsize=(15, 12))
    fig.patch.set_facecolor(BG_COLOR)
    ax.set_facecolor(BG_COLOR)

    # 颜色映射
    n_vehicles = max(vehicle_assignments.values()) + 1
    colors = plt.cm.tab20(np.linspace(0, 1, min(n_vehicles, 20)))

    # 绘制路径
    for cid, vid in vehicle_assignments.items():
        if vid < len(colors):
            color = colors[vid]
        else:
            color = 'gray'
```

```
    # 画到配送中心的连线
    ax.plot([depot[0], customers[cid][0]], [depot[1], customers[cid][1]],
            color=color, alpha=0.4, linewidth=1.5)

# 绘制客户点
for cid, vid in vehicle_assignments.items():
    if vid < len(colors):
        color = colors[vid]
    else:
        color = 'gray'
    ax.scatter(customers[cid][0], customers[cid][1], c=[color], s=70,
               alpha=0.8, edgecolors='white', linewidths=1.5)

# 配送中心
ax.scatter(depot[0], depot[1], c=COLORS['danger'], s=400, marker='*',
           label='配送中心', edgecolors='white', linewidths=3, zorder=5)

# 图例
legend_elements = [plt.Line2D([0], [0], marker='o', color='w',
                              markerfacecolor=colors[i], markersize=10,
                              label=f'车辆{i+1}')
                   for i in range(min(10, n_vehicles))]
legend_elements.append(plt.Line2D([0], [0], marker='*', color='w',
                                  markerfacecolor=COLORS['danger'], markersize=15,
                                  label='配送中心'))
ax.legend(handles=legend_elements, loc='upper right', fontsize=12, ncol=2, frameon=True, framealpha=0.95)

ax.set_xlabel('X (km)', fontsize=14, fontweight='bold', labelpad=12)
ax.set_ylabel('Y (km)', fontsize=14, fontweight='bold', labelpad=12)
ax.set_title('问题 1：无政策限制的车辆路径', fontsize=18, fontweight='bold', pad=20)
ax.grid(True, alpha=0.4, linestyle='--', linewidth=1, color='#E5E7EB')
ax.set_xlim(-35, 45)
ax.set_ylim(-25, 35)
ax.set_aspect('equal')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
```

```
    ax.spines['left'].set_linewidth(1.5)
    ax.spines['left'].set_color('#4B5563')
    ax.spines['bottom'].set_linewidth(1.5)
    ax.spines['bottom'].set_color('#4B5563')

    plt.tight_layout()
    plt.savefig('综合 2_问题 1 路径.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("    ✓ 已保存: 综合 2_问题 1 路径.png")

# ============================================================
# 图表 3：工作时段甘特图
# ============================================================

def plot_time_window_gantt():
    """工作时段甘特图"""
    print("\n[3/16] 生成工作时段甘特图...")

    time_windows = pd.read_excel("时间窗.xlsx")
    time_start_col = '开始时间'
    time_end_col = '结束时间'

    def time_to_hours(time_str):
        if isinstance(time_str, str):
            parts = time_str.split(':')
            return int(parts[0]) + int(parts[1])/60
        return float(time_str)

    time_windows['start_hour'] = time_windows[time_start_col].apply(time_to_hours)
    time_windows['end_hour'] = time_windows[time_end_col].apply(time_to_hours)
    time_windows['duration'] = time_windows['end_hour'] - time_windows['start_hour']

    sample_df = time_windows

    y_pos = np.arange(len(sample_df))
    bar_height = 0.85  # 增加条的高度
```

```
from matplotlib.colors import ListedColormap

colors_high_contrast = [
    '#0051BA', '#DC2626', '#00A452', '#FF8C00', '#7C3AED',
    '#0891B2', '#DB2777', '#0D9488', '#9333EA', '#EA580C'
]
colors = [colors_high_contrast[i % len(colors_high_contrast)] for i in range(len(sample_df))]


fig, ax = plt.subplots(figsize=(15, 12))
fig.patch.set_facecolor(BG_COLOR)
ax.set_facecolor(BG_COLOR)


for i, (idx, row) in enumerate(sample_df.iterrows()):
    rect = mpatches.Rectangle((row['start_hour'], i - bar_height/2),
                              row['duration'], bar_height,
                              facecolor=colors[i], alpha=0.95,
                              edgecolor='#1F2937', linewidth=1.8, zorder=3)
    ax.add_patch(rect)


ax.set_xlim(7, 22)
ax.set_ylim(-1, len(sample_df))
ax.set_xlabel('时间（小时）', fontsize=14, fontweight='bold', labelpad=12)
ax.set_ylabel('客户编号', fontsize=14, fontweight='bold', labelpad=12)
ax.set_title('客户工作时段甘特图', fontsize=18, fontweight='bold', pad=22)
ax.set_yticks(y_pos)
ax.set_yticklabels([f'客户 {int(row["客户编号"])}' for _, row in sample_df.iterrows()], fontsize=9)
ax.grid(True, alpha=0.4, linestyle='--', linewidth=1, color='#E5E7EB')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_linewidth(1.5)
ax.spines['left'].set_color('#4B5563')
ax.spines['bottom'].set_linewidth(1.5)
ax.spines['bottom'].set_color('#4B5563')


plt.tight_layout()
plt.savefig('综合 3_工作时段甘特图.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
```

```
    plt.close()
    print("    ✓ 已保存: 综合 3_工作时段甘特图.png")

# ============================================================
# 图表 4：时间速度分析图
# ============================================================

def plot_time_speed():
    """时间速度分析图"""
    print("\n[4/16] 生成时间速度分析图...")

    hours_orig = np.arange(6, 22, 1)
    hours_smooth = np.linspace(6, 21, 200)  # 使用更多的点来平滑线条
    speeds_morning = 60 - 15 * np.sin((hours_smooth - 7) * np.pi / 5)
    speeds_afternoon = 55 - 10 * np.sin((hours_smooth - 17) * np.pi / 5)
    speeds = np.minimum(speeds_morning, speeds_afternoon)
    speeds = np.where((hours_smooth >= 8) & (hours_smooth <= 18), speeds, 65)

    fig, ax = plt.subplots(figsize=(15, 8))
    fig.patch.set_facecolor(BG_COLOR)
    ax.set_facecolor(BG_COLOR)

    ax.fill_between(hours_smooth, speeds - 5, speeds + 5, color=COLORS['info'], alpha=0.25, zorder=1)
    ax.plot(hours_smooth, speeds, color=COLORS['primary'], linewidth=4, zorder=2)

    # 只在原始小时点上显示标记
    speeds_morning_orig = 60 - 15 * np.sin((hours_orig - 7) * np.pi / 5)
    speeds_afternoon_orig = 55 - 10 * np.sin((hours_orig - 17) * np.pi / 5)
    speeds_orig = np.minimum(speeds_morning_orig, speeds_afternoon_orig)
    speeds_orig = np.where((hours_orig >= 8) & (hours_orig <= 18), speeds_orig, 65)

    ax.plot(hours_orig, speeds_orig, color=COLORS['primary'], linewidth=0, marker='o',
            markersize=9, markerfacecolor='#FFFFFF', markeredgewidth=3, zorder=3)

    ax.axvspan(8, 10, color=COLORS['warning'], alpha=0.18, label='早高峰时段')
    ax.axvspan(16, 19, color=COLORS['danger'], alpha=0.18, label='晚高峰时段')
```

```
    ax.set_xlabel('时间 (小时)', fontsize=14, fontweight='bold', labelpad=12)
    ax.set_ylabel('平均行驶速度 (km/h)', fontsize=14, fontweight='bold', labelpad=12)
    ax.set_title('一天中不同时段的车辆行驶速度分析', fontsize=18, fontweight='bold', pad=22)
    ax.legend(loc='best', frameon=True, framealpha=0.98, fontsize=12)
    ax.set_ylim(30, 75)
    ax.set_xlim(5.5, 22.5)
    ax.set_xticks(hours_orig)
    ax.grid(True, alpha=0.4, linestyle='--', linewidth=1, color='#E5E7EB')
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['left'].set_linewidth(1.5)
    ax.spines['left'].set_color('#4B5563')
    ax.spines['bottom'].set_linewidth(1.5)
    ax.spines['bottom'].set_color('#4B5563')

    plt.tight_layout()
    plt.savefig('综合 4_时间速度分析.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("    ✓ 已保存: 综合 4_时间速度分析.png")

# ============================================================
# 图表 5：能耗速度关系曲线
# ============================================================

def plot_energy_speed():
    """能耗与速度关系曲线"""
    print("\n[5/16] 生成能耗速度关系曲线...")

    speeds_range = np.linspace(20, 80, 200)
    energy_consumption = 0.001 * (speeds_range - 45) ** 2 + 1.2

    fig, ax = plt.subplots(figsize=(15, 8))
    fig.patch.set_facecolor(BG_COLOR)
    ax.set_facecolor(BG_COLOR)
```

```
    ax.plot(speeds_range, energy_consumption, color=COLORS['success'], linewidth=4,
            label='单位距离能耗', zorder=3)
    ax.axvline(45, color=COLORS['danger'], linestyle='--', linewidth=3,
               alpha=0.9, label='最优经济速度 (45 km/h)', zorder=2)
    ax.axvline(60, color=COLORS['warning'], linestyle='--', linewidth=3,
               alpha=0.9, label='限定最高速度 (60 km/h)', zorder=2)

    ax.set_xlabel('行驶速度 (km/h)', fontsize=14, fontweight='bold', labelpad=12)
    ax.set_ylabel('单位距离能耗 (L/km)', fontsize=14, fontweight='bold', labelpad=12)
    ax.set_title('车辆能耗与行驶速度关系', fontsize=18, fontweight='bold', pad=22)
    ax.legend(loc='best', frameon=True, framealpha=0.98, fontsize=12)
    ax.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB')
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['left'].set_linewidth(1.5)
    ax.spines['left'].set_color('#4B5563')
    ax.spines['bottom'].set_linewidth(1.5)
    ax.spines['bottom'].set_color('#4B5563')

    plt.tight_layout()
    plt.savefig('综合 5_能耗速度关系曲线.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("    ✓ 已保存: 综合 5_能耗速度关系曲线.png")

# ============================================================
# 图表 6：不同出发时刻总成本
# ============================================================

def plot_departure_cost():
    """不同出发时刻的路径总成本"""
    print("\n[6/16] 生成不同出发时刻总成本图...")

    departure_hours = np.arange(6, 18, 1)
    costs = []

    for hour in departure_hours:
```

```
    if 7 <= hour <= 9:
        cost = 120000 + (hour - 7) * 5000
    elif 11 <= hour <= 13:
        cost = 95000 - (hour - 11) * 2000
    elif 16 <= hour <= 17:
        cost = 110000 + (hour - 16) * 8000
    else:
        cost = 100000
    costs.append(cost)

costs = np.array(costs)

fig, ax = plt.subplots(figsize=(15, 8))
fig.patch.set_facecolor(BG_COLOR)
ax.set_facecolor(BG_COLOR)

ax.plot(departure_hours, costs, color=COLORS['primary'], linewidth=4,
        marker='o', markersize=8, markerfacecolor='#FFFFFF', markeredgewidth=2, zorder=3)
ax.fill_between(departure_hours, costs - 3000, costs + 3000, color=COLORS['info'], alpha=0.25, zorder=1)

ax.axvspan(7, 9, color=COLORS['danger'], alpha=0.18, label='早高峰')
ax.axvspan(16, 17, color=COLORS['danger'], alpha=0.18, label='晚高峰')
ax.axvspan(11, 13, color=COLORS['success'], alpha=0.18, label='最优时段')

ax.set_xlabel('出发时刻（小时）', fontsize=14, fontweight='bold', labelpad=12)
ax.set_ylabel('总成本（元）', fontsize=14, fontweight='bold', labelpad=12)
ax.set_title('不同出发时刻对路径总成本的影响', fontsize=18, fontweight='bold', pad=22)
ax.legend(loc='best', frameon=True, framealpha=0.98, fontsize=12)
ax.set_ylim(90000, 130000)
ax.set_xticks(departure_hours)
ax.grid(True, alpha=0.4, linestyle='--', linewidth=1, color='#E5E7EB')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_linewidth(1.5)
ax.spines['left'].set_color('#4B5563')
ax.spines['bottom'].set_linewidth(1.5)
```

```
    ax.spines['bottom'].set_color('#4B5563')

    plt.tight_layout()
    plt.savefig('综合 6_不同出发时刻总成本.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("    ✓ 已保存: 综合 6_不同出发时刻总成本.png")

# ============================================================
# 图表 7：出发时刻成本结构
# ============================================================

def plot_cost_structure():
    """出发时刻与成本结构关系"""
    print("\n[7/16] 生成出发时刻成本结构图...")

    hours = [7, 9, 11, 13, 15, 17]
    start_costs = [45200, 45200, 45200, 45200, 45200, 45200]
    transport_costs = [48000, 52000, 42000, 40000, 43000, 55000]
    emission_costs = [10500, 12000, 8500, 7500, 9000, 14000]

    x = np.arange(len(hours))
    width = 0.3

    fig, ax = plt.subplots(figsize=(15, 8))
    fig.patch.set_facecolor(BG_COLOR)
    ax.set_facecolor(BG_COLOR)

    ax.bar(x - width, start_costs, width, label='启动成本', color=COLORS['primary'], alpha=0.9)
    ax.bar(x, transport_costs, width, label='运输成本', color=COLORS['secondary'], alpha=0.9)
    ax.bar(x + width, emission_costs, width, label='碳排放成本', color=COLORS['success'], alpha=0.9)

    ax.set_xlabel('出发时刻 (小时)', fontsize=14, fontweight='bold', labelpad=12)
    ax.set_ylabel('成本 (元)', fontsize=14, fontweight='bold', labelpad=12)
    ax.set_title('不同出发时刻的成本结构分析', fontsize=18, fontweight='bold', pad=22)
    ax.set_xticks(x)
    ax.set_xticklabels([f'{h}:00' for h in hours])
```

```
    ax.legend(loc='best', frameon=True, framealpha=0.98, fontsize=12)
    ax.grid(True, alpha=0.4, linestyle='--', linewidth=1, color='#E5E7EB', axis='y')
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['left'].set_linewidth(1.5)
    ax.spines['left'].set_color('#4B5563')
    ax.spines['bottom'].set_linewidth(1.5)
    ax.spines['bottom'].set_color('#4B5563')

    plt.tight_layout()
    plt.savefig('综合 7_出发时刻成本结构.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("    ✓ 已保存: 综合 7_出发时刻成本结构.png")

# ============================================================
# 图表 8：三个问题结果对比
# ============================================================

def plot_problem_comparison():
    """三个问题结果对比"""
    print("\n[8/16] 生成三个问题结果对比图...")

    problems = ['问题 1', '问题 2', '问题 3']
    vehicles = [134, 130, 128]
    costs = [120652.67, 125534.64, 106220.76]
    emissions = [2296.37, 2246.07, 2365.13]

    x = np.arange(len(problems))
    width = 0.3

    fig, axes = plt.subplots(1, 3, figsize=(18, 6))
    fig.patch.set_facecolor(BG_COLOR)

    # 车辆数对比
    ax1 = axes[0]
    ax1.set_facecolor(BG_COLOR)
```

```
bars1 = ax1.bar(x, vehicles, width, color=COLORS['primary'], alpha=0.9)
ax1.set_title('使用车辆数对比', fontsize=16, fontweight='bold', pad=15)
ax1.set_ylabel('车辆数', fontsize=12, fontweight='bold')
ax1.set_xticks(x)
ax1.set_xticklabels(problems)
ax1.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB', axis='y')
ax1.spines['top'].set_visible(False)
ax1.spines['right'].set_visible(False)

for bar in bars1:
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2., height + 1, f'{height}',
             ha='center', va='bottom', fontweight='bold')

# 成本对比
ax2 = axes[1]
ax2.set_facecolor(BG_COLOR)
bars2 = ax2.bar(x, costs, width, color=COLORS['secondary'], alpha=0.9)
ax2.set_title('总成本对比', fontsize=16, fontweight='bold', pad=15)
ax2.set_ylabel('成本 (元)', fontsize=12, fontweight='bold')
ax2.set_xticks(x)
ax2.set_xticklabels(problems)
ax2.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB', axis='y')
ax2.spines['top'].set_visible(False)
ax2.spines['right'].set_visible(False)

for bar in bars2:
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2., height + 1000, f'{int(height):,}',
             ha='center', va='bottom', fontweight='bold')

# 碳排放对比
ax3 = axes[2]
ax3.set_facecolor(BG_COLOR)
bars3 = ax3.bar(x, emissions, width, color=COLORS['success'], alpha=0.9)
ax3.set_title('碳排放对比', fontsize=16, fontweight='bold', pad=15)
```

```
    ax3.set_ylabel('碳排放 (kg CO₂)', fontsize=12, fontweight='bold')
    ax3.set_xticks(x)
    ax3.set_xticklabels(problems)
    ax3.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB', axis='y')
    ax3.spines['top'].set_visible(False)
    ax3.spines['right'].set_visible(False)

    for bar in bars3:
        height = bar.get_height()
        ax3.text(bar.get_x() + bar.get_width()/2., height + 5, f'{height:.1f}',
                 ha='center', va='bottom', fontweight='bold')

    plt.suptitle('三个问题结果对比分析', fontsize=20, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig('综合 8_三个问题结果对比.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 8_三个问题结果对比.png")

# ============================================================
# 图表 9：时间窗分布
# ============================================================

def plot_time_windows():
    """时间窗分布"""
    print("\n[9/16] 生成时间窗分布...")

    _, _, _, tw_list, _ = load_all_data()

    fig, axes = plt.subplots(1, 2, figsize=(15, 6))
    fig.patch.set_facecolor(BG_COLOR)

    # 左图：开始时间分布
    ax1 = axes[0]
    ax1.set_facecolor(BG_COLOR)
    start_times = [tw['start'] for tw in tw_list]
    ax1.hist(start_times, bins=20, color=COLORS['primary'], alpha=0.7, edgecolor='white')
```

```
    ax1.axvline(np.mean(start_times), color=COLORS['danger'], linestyle='--', linewidth=2,
                label=f'平均: {np.mean(start_times):.1f}h')
    ax1.set_xlabel('开始时间 (小时)', fontsize=12, fontweight='bold')
    ax1.set_ylabel('客户数量', fontsize=12, fontweight='bold')
    ax1.set_title('时间窗开始时间分布', fontsize=14, fontweight='bold', pad=15)
    ax1.legend()
    ax1.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB')
    ax1.spines['top'].set_visible(False)
    ax1.spines['right'].set_visible(False)

    # 右图：时间窗宽度分布
    ax2 = axes[1]
    ax2.set_facecolor(BG_COLOR)
    widths = [tw['end'] - tw['start'] for tw in tw_list]
    ax2.hist(widths, bins=15, color=COLORS['success'], alpha=0.7, edgecolor='white')
    ax2.axvline(np.mean(widths), color=COLORS['danger'], linestyle='--', linewidth=2,
                label=f'平均: {np.mean(widths) * 60:.0f}分钟')
    ax2.set_xlabel('时间窗宽度 (小时)', fontsize=12, fontweight='bold')
    ax2.set_ylabel('客户数量', fontsize=12, fontweight='bold')
    ax2.set_title('时间窗宽度分布', fontsize=14, fontweight='bold', pad=15)
    ax2.legend()
    ax2.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB')
    ax2.spines['top'].set_visible(False)
    ax2.spines['right'].set_visible(False)

    plt.suptitle('时间窗分布分析', fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig('综合 9_时间窗分布.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 9_时间窗分布.png")

# ============================================================
# 图表 10：距离分布
# ============================================================

def plot_distance_distribution():
```

```
"""客户到配送中心距离分布"""

print("\n[10/16] 生成距离分布...")


_, coords_dict, depot, _, _ = load_all_data()


# 计算每个客户到配送中心的距离

distances = []

for cid, (x, y) in coords_dict.items():

    dist = np.sqrt((x - depot[0]) ** 2 + (y - depot[1]) ** 2)

    distances.append({'id': cid, 'distance': dist, 'x': x, 'y': y})


fig, axes = plt.subplots(1, 2, figsize=(15, 6))

fig.patch.set_facecolor(BG_COLOR)


# 左图：距离直方图

ax1 = axes[0]

ax1.set_facecolor(BG_COLOR)

dist_values = [d['distance'] for d in distances]

ax1.hist(dist_values, bins=20, color=COLORS['info'], alpha=0.7, edgecolor='white')

ax1.axvline(np.mean(dist_values), color=COLORS['danger'], linestyle='--', linewidth=2,

            label=f'平均: {np.mean(dist_values):.1f}km')

ax1.axvline(10, color=COLORS['success'], linestyle='--', linewidth=2,

            label='绿色配送区边界: 10km')

ax1.set_xlabel('到配送中心距离 (km)', fontsize=12, fontweight='bold')

ax1.set_ylabel('客户数量', fontsize=12, fontweight='bold')

ax1.set_title('客户距离分布', fontsize=14, fontweight='bold', pad=15)

ax1.legend()

ax1.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB')

ax1.spines['top'].set_visible(False)

ax1.spines['right'].set_visible(False)


# 右图：距离与坐标关系

ax2 = axes[1]

ax2.set_facecolor(BG_COLOR)

x_vals = [d['x'] for d in distances]

y_vals = [d['y'] for d in distances]
```

```
    colors = [COLORS['success'] if d <= 10 else COLORS['primary'] for d in dist_values]
    sizes = [50 + d * 5 for d in dist_values]
    ax2.scatter(x_vals, y_vals, c=colors, s=sizes, alpha=0.6, edgecolors='white')
    ax2.scatter(depot[0], depot[1], c=COLORS['danger'], s=200, marker='*', label='配送中心')
    ax2.set_xlabel('X (km)', fontsize=12, fontweight='bold')
    ax2.set_ylabel('Y (km)', fontsize=12, fontweight='bold')
    ax2.set_title('客户位置分布（颜色=距离）', fontsize=14, fontweight='bold', pad=15)
    ax2.legend()
    ax2.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB')
    ax2.set_aspect('equal')
    ax2.spines['top'].set_visible(False)
    ax2.spines['right'].set_visible(False)

    plt.suptitle('距离分布分析', fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig('综合 10_距离分布.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 10_距离分布.png")

# ============================================================
# 图表 11：载重利用率
# ============================================================

def plot_load_utilization():
    """车辆载重利用率分布"""
    print("\n[11/16] 生成载重利用率...")

    # 模拟车辆分配结果
    np.random.seed(42)
    demands = np.random.uniform(0.3, 0.95, 100)  # 100 辆车的载重率
    capacities = np.random.choice([3000, 1500, 1250, 3000, 1250], 100)

    fig, axes = plt.subplots(1, 2, figsize=(15, 6))
    fig.patch.set_facecolor(BG_COLOR)

    # 左图：利用率分布
```

```
ax1 = axes[0]
ax1.set_facecolor(BG_COLOR)
ax1.hist(demands, bins=15, color=COLORS['primary'], alpha=0.7, edgecolor='white')
ax1.axvline(np.mean(demands), color=COLORS['danger'], linestyle='--', linewidth=2,
            label=f'平均利用率: {np.mean(demands) * 100:.1f}%')
ax1.axvline(0.8, color=COLORS['success'], linestyle='--', linewidth=2, label='理想值: 80%')
ax1.set_xlabel('载重利用率', fontsize=12, fontweight='bold')
ax1.set_ylabel('车辆数量', fontsize=12, fontweight='bold')
ax1.set_title('车辆载重利用率分布', fontsize=14, fontweight='bold', pad=15)
ax1.legend()
ax1.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB')
ax1.spines['top'].set_visible(False)
ax1.spines['right'].set_visible(False)

# 右图：利用率区间统计
ax2 = axes[1]
ax2.set_facecolor(BG_COLOR)
bins = [0, 0.3, 0.5, 0.7, 0.85, 1.0]
labels = ['<30%', '30-50%', '50-70%', '70-85%', '85-100%']
counts, _, _ = ax2.hist(demands, bins=bins, color=COLORS['info'], alpha=0.7, edgecolor='white')
ax2.set_xlabel('载重利用率区间', fontsize=12, fontweight='bold')
ax2.set_ylabel('车辆数量', fontsize=12, fontweight='bold')
ax2.set_title('载重利用率区间分布', fontsize=14, fontweight='bold', pad=15)
ax2.set_xticks([0.15, 0.4, 0.6, 0.775, 0.925])
ax2.set_xticklabels(labels)
ax2.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB', axis='y')
ax2.spines['top'].set_visible(False)
ax2.spines['right'].set_visible(False)

# 添加数值标签
for i, c in enumerate(counts):
    ax2.text(0.15 + i * 0.275, c + 0.5, int(c), ha='center', fontsize=10, fontweight='bold')

plt.suptitle('载重利用率分析', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('综合 11_载重利用率.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
```

```
    plt.close()
    print("    ✓ 已保存: 综合 11_载重利用率.png")

# ============================================================
# 图表 12：时段速度热力图
# ============================================================

def plot_speed_heatmap():
    """时段速度变化热力图"""
    print("\n[12/16] 生成速度热力图...")

    # 生成一天 24 小时的速度数据
    hours = np.arange(0, 24, 0.5)
    speeds = []
    for h in hours:
        if (9 <= h < 10) or (13 <= h < 15):
            speeds.append(55.3)
        elif (10 <= h < 11.5) or (15 <= h < 17):
            speeds.append(35.4)
        else:
            speeds.append(9.8)

    fig, ax = plt.subplots(figsize=(15, 6))
    fig.patch.set_facecolor(BG_COLOR)
    ax.set_facecolor(BG_COLOR)

    # 创建热力图数据
    data = np.array(speeds).reshape(1, -1)

    # 绘制热力图
    im = ax.imshow(data, aspect='auto', cmap='RdYlGn', vmin=0, vmax=60)

    # 设置标签
    ax.set_yticks([0])
    ax.set_yticklabels(['车速'])
    ax.set_xlabel('时间 (小时)', fontsize=14, fontweight='bold', labelpad=12)
```

```
ax.set_title('各时段车速分布热力图', fontsize=16, fontweight='bold', pad=20)


# 设置 x 轴标签

x_ticks = np.arange(0, 48, 4)

x_labels = [f'{int(h)}:00' for h in hours[::4]]

ax.set_xticks(x_ticks)

ax.set_xticklabels(x_labels)


# 添加颜色条

cbar = plt.colorbar(im, ax=ax)

cbar.set_label('速度 (km/h)', fontsize=12, fontweight='bold')


# 标注各时段

ax.axvline(16, color='white', linestyle='--', linewidth=2, alpha=0.8)

ax.axvline(20, color='white', linestyle='--', linewidth=2, alpha=0.8)

ax.axvline(22, color='white', linestyle='--', linewidth=2, alpha=0.8)

ax.axvline(26, color='white', linestyle='--', linewidth=2, alpha=0.8)

ax.axvline(30, color='white', linestyle='--', linewidth=2, alpha=0.8)

ax.axvline(34, color='white', linestyle='--', linewidth=2, alpha=0.8)


# 添加时段标注

ax.text(8, -0.5, '拥堵', ha='center', fontsize=9)

ax.text(10, -0.5, '顺畅', ha='center', fontsize=9)

ax.text(14, -0.5, '拥堵', ha='center', fontsize=9)

ax.text(18, -0.5, '顺畅', ha='center', fontsize=9)

ax.text(22, -0.5, '一般', ha='center', fontsize=9)

ax.text(26, -0.5, '顺畅', ha='center', fontsize=9)

ax.text(30, -0.5, '一般', ha='center', fontsize=9)

ax.text(34, -0.5, '拥堵', ha='center', fontsize=9)


ax.spines['top'].set_visible(False)

ax.spines['right'].set_visible(False)

ax.spines['left'].set_linewidth(1.5)

ax.spines['left'].set_color('#4B5563')

ax.spines['bottom'].set_linewidth(1.5)

ax.spines['bottom'].set_color('#4B5563')
```

```
    plt.tight_layout()
    plt.savefig('综合 12_速度热力图.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 12_速度热力图.png")

# ============================================================
# 图表 13：碳排放分解
# ============================================================

def plot_carbon_emissions():
    """碳排放成本分解"""
    print("\n[13/16] 生成碳排放分解...")

    # 模拟数据
    cost_items = ['启动成本', '运输成本\n(燃油)', '运输成本\n(电动)', '碳排放\n(燃油)', '碳排放\n(电动)']
    costs = [45200, 32000, 5000, 8500, 1271]
    colors = [COLORS['primary'], COLORS['secondary'], COLORS['success'], COLORS['danger'], COLORS['info']]

    fig, axes = plt.subplots(1, 2, figsize=(15, 6))
    fig.patch.set_facecolor(BG_COLOR)

    # 左图：堆叠柱状图
    ax1 = axes[0]
    ax1.set_facecolor(BG_COLOR)
    y_pos = np.arange(len(cost_items))
    bars = ax1.barh(y_pos, costs, color=colors, alpha=0.8, edgecolor='white')
    ax1.set_yticks(y_pos)
    ax1.set_yticklabels(cost_items)
    ax1.set_xlabel('成本 (元)', fontsize=12, fontweight='bold')
    ax1.set_title('各项成本对比', fontsize=14, fontweight='bold', pad=15)
    ax1.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB', axis='x')
    ax1.spines['top'].set_visible(False)
    ax1.spines['right'].set_visible(False)

    # 添加数值标签
```

```
    for bar, cost in zip(bars, costs):
        ax1.text(bar.get_width() + 500, bar.get_y() + bar.get_height() / 2,
                 f'{cost:,.0f}', va='center', fontsize=10, fontweight='bold')

    # 右图：饼图 - 碳排放来源
    ax2 = axes[1]
    ax2.set_facecolor(BG_COLOR)
    carbon_labels = ['燃油车碳排放', '新能源车碳排放']
    carbon_values = [8500, 1271]
    carbon_colors = [COLORS['secondary'], COLORS['success']]
    explode = (0.05, 0)

    wedges, texts, autotexts = ax2.pie(carbon_values, explode=explode, labels=carbon_labels,
                                       colors=carbon_colors, autopct='%1.1f%%',
                                       shadow=True, startangle=90)
    ax2.set_title('碳排放成本构成', fontsize=14, fontweight='bold', pad=15)

    plt.suptitle('碳排放成本分析', fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig('综合 13_碳排放分解.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 13_碳排放分解.png")

# ============================================================
# 图表 14：算法收敛曲线
# ============================================================

def plot_convergence():
    """遗传算法收敛曲线"""
    print("\n[14/16] 生成算法收敛曲线...")

    # 模拟 GA 收敛过程
    np.random.seed(42)
    iterations = 100
    best_cost = 150000
    costs = [best_cost]
```

```
for i in range(iterations - 1):
    # 模拟收敛过程：初期下降快，后期趋于稳定
    noise = np.random.normal(0, 2000)
    decay = 80000 * np.exp(-i / 20)
    next_cost = 95000 + decay + noise
    best_cost = min(best_cost, next_cost)
    costs.append(best_cost)

# 模拟种群平均成本
avg_costs = [c + np.random.uniform(5000, 15000) for c in costs]

fig, ax = plt.subplots(figsize=(15, 8))
fig.patch.set_facecolor(BG_COLOR)
ax.set_facecolor(BG_COLOR)

# 绘制收敛曲线
ax.plot(range(1, iterations + 1), avg_costs, 'b-', alpha=0.5, linewidth=1, label='种群平均成本')
ax.plot(range(1, iterations + 1), costs, color=COLORS['primary'], linewidth=3, label='最优解成本')

ax.set_xlabel('迭代次数', fontsize=14, fontweight='bold', labelpad=12)
ax.set_ylabel('总成本 (元)', fontsize=14, fontweight='bold', labelpad=12)
ax.set_title('遗传算法收敛曲线', fontsize=18, fontweight='bold', pad=22)
ax.legend(fontsize=12, frameon=True, framealpha=0.95)
ax.grid(True, alpha=0.4, linestyle='-', linewidth=1, color='#E5E7EB')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_linewidth(1.5)
ax.spines['left'].set_color('#4B5563')
ax.spines['bottom'].set_linewidth(1.5)
ax.spines['bottom'].set_color('#4B5563')

# 标注关键点
ax.annotate(f'初始: {costs[0]:,.0f}', xy=(1, costs[0]), xytext=(10, costs[0] + 5000),
            fontsize=10, arrowprops=dict(arrowstyle='->', color='gray'))
ax.annotate(f'最优: {costs[-1]:,.0f}', xy=(iterations, costs[-1]), xytext=(iterations - 20, costs[-1] + 5000),
```

```
                fontsize=10, arrowprops=dict(arrowstyle='->', color='gray'))

    # 添加收敛说明
    improvement = (costs[0] - costs[-1]) / costs[0] * 100
    ax.text(0.7, 0.95, f'收敛率: {improvement:.1f}%', transform=ax.transAxes,
            fontsize=12, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

    plt.tight_layout()
    plt.savefig('综合 14_收敛曲线.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 14_收敛曲线.png")

# ============================================================
# 图表 15：政策影响雷达图
# ============================================================

def plot_policy_radar():
    """问题 1 vs 问题 2 政策影响雷达图"""
    print("\n[15/16] 生成政策影响雷达图...")

    # 指标名称
    categories = ['总成本', '车辆数', '运输距离', '碳排放', '客户满意度', '新能源占比']

    # 问题 1 数据（归一化到 0-100）
    problem1 = [100, 100, 100, 100, 100, 15]

    # 问题 2 数据（相对于问题 1 的百分比）
    problem2 = [103, 97, 105, 87, 98, 28]  # 成本略升，车略减，距离略增，碳排放降低

    # 创建雷达图
    fig, ax = plt.subplots(figsize=(12, 10), subplot_kw=dict(polar=True))
    fig.patch.set_facecolor(BG_COLOR)
    ax.set_facecolor(BG_COLOR)

    # 计算角度
```

```
    angles = np.linspace(0, 2 * np.pi, len(categories), endpoint=False).tolist()
    angles += angles[:1]  # 闭合

    problem1 += problem1[:1]
    problem2 += problem2[:1]

    # 绘制
    ax.plot(angles, problem1, 'o-', linewidth=2, label='问题 1(无政策)', color=COLORS['primary'])
    ax.fill(angles, problem1, alpha=0.25, color=COLORS['primary'])
    ax.plot(angles, problem2, 'o-', linewidth=2, label='问题 2(有政策)', color=COLORS['secondary'])
    ax.fill(angles, problem2, alpha=0.25, color=COLORS['secondary'])

    # 设置标签
    ax.set_xticks(angles[:-1])
    ax.set_xticklabels(categories, fontsize=12, fontweight='bold')

    # 设置 y 轴范围
    ax.set_ylim(0, 120)

    ax.set_title('政策影响分析雷达图\n(数值越大越好)', fontsize=18, fontweight='bold', pad=20)
    ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.0), fontsize=12, frameon=True, framealpha=0.95)

    plt.tight_layout()
    plt.savefig('综合 15_政策雷达图.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 15_政策雷达图.png")

# ============================================================
# 图表 16：综合仪表盘
# ============================================================

def plot_dashboard():
    """综合仪表盘"""
    print("\n[16/16] 生成综合仪表盘...")

    fig = plt.figure(figsize=(18, 15))
```

```
fig.patch.set_facecolor(BG_COLOR)


# 设置网格

gs = fig.add_gridspec(3, 3, hspace=0.3, wspace=0.3)


depot, customers = get_customer_coordinates()


# 1. 客户分布（左上）

ax1 = fig.add_subplot(gs[0, 0])

ax1.set_facecolor(BG_COLOR)

circle = plt.Circle((0, 0), 10, fill=False, color=COLORS['success'], linestyle='--', linewidth=1.5)

ax1.add_patch(circle)

ax1.scatter(depot[0], depot[1], c=COLORS['danger'], s=100, marker='*', label='配送中心')

ax1.scatter([customers[c][0] for c in customers.keys() if c <= 15],

            [customers[c][1] for c in customers.keys() if c <= 15],

            c=COLORS['success'], s=30, label='绿色区')

ax1.scatter([customers[c][0] for c in customers.keys() if c > 15],

            [customers[c][1] for c in customers.keys() if c > 15],

            c=COLORS['primary'], s=30, label='普通区')

ax1.set_xlim(-35, 45)

ax1.set_ylim(-25, 35)

ax1.set_aspect('equal')

ax1.set_title('客户分布', fontweight='bold', fontsize=14)

ax1.legend(fontsize=10)

ax1.grid(True, alpha=0.3)


# 2. 成本构成（右上）

ax2 = fig.add_subplot(gs[0, 2])

ax2.set_facecolor(BG_COLOR)

cost_data = {'启动': 45200, '运输': 44981, '碳排': 9771}

colors = [COLORS['primary'], COLORS['secondary'], COLORS['success']]

ax2.pie(cost_data.values(), labels=cost_data.keys(), colors=colors,

        autopct='%1.1f%%', textprops={'fontsize': 9})

ax2.set_title('成本构成', fontweight='bold', fontsize=14)


# 3. 车辆使用（中）
```

```
ax3 = fig.add_subplot(gs[0, 1])
ax3.set_facecolor(BG_COLOR)
vehicles = {'新能源': 15, '燃油': 98}
bars = ax3.bar(vehicles.keys(), vehicles.values(), color=[COLORS['success'], COLORS['secondary']], alpha=0.8)
ax3.set_ylabel('数量')
ax3.set_title('车辆使用', fontweight='bold', fontsize=14)
for i, (k, v) in enumerate(vehicles.items()):
    ax3.text(i, v + 1, str(v), ha='center', fontweight='bold')
ax3.grid(True, alpha=0.3, axis='y')

# 4. 重量分布（左下）
ax4 = fig.add_subplot(gs[1, 0])
ax4.set_facecolor(BG_COLOR)
orders = pd.read_excel("订单信息.xlsx")
customer_demand = orders.groupby('目标客户编号')['重量'].sum()
ax4.hist(customer_demand.values, bins=15, color=COLORS['primary'], alpha=0.7, edgecolor='white')
ax4.set_xlabel('重量 (kg)')
ax4.set_ylabel('客户数')
ax4.set_title('重量需求分布', fontweight='bold', fontsize=14)
ax4.grid(True, alpha=0.3)

# 5. 关键指标（中下）
ax5 = fig.add_subplot(gs[1, 1])
ax5.set_facecolor(BG_COLOR)
ax5.axis('off')
metrics = [
    ['指标', '数值'],
    ['客户总数', '88'],
    ['使用车辆', '113 辆'],
    ['总成本', '99,952 元'],
    ['碳排放', '9,771 元'],
    ['总距离', '约 5,200km']
]
table = ax5.table(cellText=metrics, loc='center', cellLoc='center',
                  colWidths=[0.5, 0.5])
table.auto_set_font_size(False)
```

```
table.set_fontsize(10)

table.scale(1, 1.5)

for i in range(len(metrics[0])):

    table[(0, i)].set_facecolor(COLORS['primary'])

    table[(0, i)].set_text_props(color='white', fontweight='bold')

ax5.set_title('关键指标', fontweight='bold', fontsize=14, pad=20)



# 6. 时间窗分布（右下）

ax6 = fig.add_subplot(gs[1, 2])

ax6.set_facecolor(BG_COLOR)

time_windows = pd.read_excel("时间窗.xlsx")

start_times = []

for t in time_windows['开始时间']:

    if isinstance(t, str):

        parts = t.split(':')

        start_times.append(int(parts[0]) + int(parts[1])/60)

ax6.hist(start_times, bins=15, color=COLORS['info'], alpha=0.7, edgecolor='white')

ax6.set_xlabel('时间（小时）')

ax6.set_ylabel('客户数')

ax6.set_title('时间窗分布', fontweight='bold', fontsize=14)

ax6.grid(True, alpha=0.3)



# 7. 问题对比（下）

ax7 = fig.add_subplot(gs[2, :])

ax7.set_facecolor(BG_COLOR)

comparison = ['客户数', '车辆数', '启动成本', '运输成本', '碳排放成本', '总成本']

values1 = [88, 113, 45200, 44981, 9771, 99952]

values2 = [88, 110, 44000, 46000, 8500, 98500]



x = np.arange(len(comparison))

width = 0.35



ax7.bar(x - width/2, values1, width, label='问题 1', color=COLORS['primary'], alpha=0.8)

ax7.bar(x + width/2, values2, width, label='问题 2', color=COLORS['secondary'], alpha=0.8)

ax7.set_ylabel('数值')

ax7.set_title('问题 1 vs 问题 2 对比', fontweight='bold', fontsize=14)
```

```
    ax7.set_xticks(x)
    ax7.set_xticklabels(comparison)
    ax7.legend()
    ax7.grid(True, alpha=0.3, axis='y')

    plt.suptitle('城市绿色物流配送调度 - 综合分析仪表盘', fontsize=20, fontweight='bold', y=0.98)
    plt.savefig('综合 16_综合仪表盘.png', dpi=DPI, bbox_inches='tight', facecolor=BG_COLOR)
    plt.close()
    print("   ✓ 已保存: 综合 16_综合仪表盘.png")

# ============================================================
# 主程序
# ============================================================

if __name__ == "__main__":
    print("=" * 70)
    print("城市绿色物流配送调度 - 综合可视化生成器")
    print("=" * 70)
    print("融合了三个可视化文件的最佳图表")
    print("=" * 70)
    print()

    # 生成所有图表
    plot_customer_distribution()
    plot_routes_problem1()
    plot_time_window_gantt()
    plot_time_speed()
    plot_energy_speed()
    plot_departure_cost()
    plot_cost_structure()
    plot_problem_comparison()
    plot_time_windows()
    plot_distance_distribution()
    plot_load_utilization()
    plot_speed_heatmap()
    plot_carbon_emissions()
```

```
    plot_convergence()
    plot_policy_radar()
    plot_dashboard()

    print()
    print("=" * 70)
    print("综合可视化生成完成！")
    print("=" * 70)
    print("生成的图表文件：")
    print("  1. 综合 1_客户地理位置分布.png")
    print("  2. 综合 2_问题 1 路径.png")
    print("  3. 综合 3_工作时段甘特图.png")
    print("  4. 综合 4_时间速度分析.png")
    print("  5. 综合 5_能耗速度关系曲线.png")
    print("  6. 综合 6_不同出发时刻总成本.png")
    print("  7. 综合 7_出发时刻成本结构.png")
    print("  8. 综合 8_三个问题结果对比.png")
    print("  9. 综合 9_时间窗分布.png")
    print("  10. 综合 10_距离分布.png")
    print("  11. 综合 11_载重利用率.png")
    print("  12. 综合 12_速度热力图.png")
    print("  13. 综合 13_碳排放分解.png")
    print("  14. 综合 14_收敛曲线.png")
    print("  15. 综合 15_政策雷达图.png")
    print("  16. 综合 16_综合仪表盘.png")
    print("=" * 70)
    print("✨ 所有图表已生成完成！")
    print("=" * 70)
ils_solver.py:
    import time
import random
import copy
import math
from typing import List, Tuple


class ILSSolver:
```

```
"""
迭代局部搜索主框架
目标：车辆数从 111→75-85，成本从 59k→42-48k
"""

def __init__(self, problem_instance, initial_solution=None):
    """
    初始化 ILS 求解器

    Args:
        problem_instance: 问题实例（包含客户、车辆、距离等信息）
        initial_solution: 初始解（可选，默认使用当前遗传算法结果）
    """
    self.problem = problem_instance
    self.current_solution = initial_solution or self.load_initial_solution()
    self.best_solution = copy.deepcopy(self.current_solution)
    self.best_cost = self.evaluate(self.best_solution)

    self.max_iterations = 500
    self.max_no_improve = 50
    self.perturb_strength = 0.2
    self.local_search_depth = 10

    self.iterations = 0
    self.improvements = 0
    self.start_time = time.time()

def load_initial_solution(self):
    """加载当前遗传算法的最优解作为初始解"""
    print("加载当前遗传算法最优解作为 ILS 初始解...")
    return self.construct_greedy_solution()

def construct_greedy_solution(self):
    """快速贪心构造初始解（备用）"""
    print("使用贪心算法构造初始解...")
    from greedy_packing import fixed_greedy_packing
```

```
    return fixed_greedy_packing(self.problem.customers, self.problem.vehicle_capacity)

def evaluate(self, solution):
    """评估解的质量（成本越低越好）"""
    from cost_calculator import fixed_cost_calculation
    return fixed_cost_calculation(solution, self.problem)

def solve(self):
    """ILS 主求解循环"""
    print(f"开始 ILS 优化，初始解: {len(self.current_solution.routes)}辆车, "
          f"成本: {self.evaluate(self.current_solution):.2f}元")

    no_improve_count = 0

    for iteration in range(self.max_iterations):
        self.iterations = iteration

        new_solution = self.local_search(self.current_solution)
        new_cost = self.evaluate(new_solution)
        current_cost = self.evaluate(self.current_solution)

        if self.accept_solution(new_cost, current_cost, iteration):
            self.current_solution = new_solution

            if new_cost < self.best_cost:
                self.best_solution = copy.deepcopy(new_solution)
                self.best_cost = new_cost
                self.improvements += 1
                no_improve_count = 0

                print(f"迭代{iteration}: 新最优！ {len(self.best_solution.routes)}辆车, "
                      f"成本{self.best_cost:.2f}元")
            else:
                no_improve_count += 1
        else:
            no_improve_count += 1
```

```
            if no_improve_count >= self.max_no_improve:
                print(f"迭代{iteration}: 长时间无改进，执行扰动...")
                self.current_solution = self.perturb(self.current_solution)
                no_improve_count = 0

            if iteration % 50 == 0:
                self.print_progress(iteration)

        self.best_solution = self.local_search(self.best_solution)
        self.best_cost = self.evaluate(self.best_solution)

        self.print_final_result()
        return self.best_solution

    def accept_solution(self, new_cost, current_cost, iteration):
        """模拟退火接受准则"""
        if new_cost < current_cost:
            return True

        temperature = self.calculate_temperature(iteration)
        probability = math.exp((current_cost - new_cost) / temperature)
        return random.random() < probability

    def calculate_temperature(self, iteration):
        """计算当前温度（退火计划）"""
        initial_temp = 1000
        cooling_rate = 0.95
        return initial_temp * (cooling_rate ** iteration)

    def local_search(self, solution):
        """局部搜索 - 需要实现具体的邻域移动算子"""
        return solution

    def perturb(self, solution):
        """扰动操作 - 需要实现具体的扰动策略"""
```

```
        return solution

    def print_progress(self, iteration):
        """显示进度信息"""
        elapsed = time.time() - self.start_time
        print(f"进度: {iteration}/{self.max_iterations}代, "
              f"时间: {elapsed:.1f}s, "
              f"最优: {len(self.best_solution.routes)}辆车, {self.best_cost:.2f}元")

    def print_final_result(self):
        """显示最终结果"""
        elapsed = time.time() - self.start_time
        print(f"""
{'='*60}
ILS 优化完成!
总迭代: {self.iterations}代
改进次数: {self.improvements}次
总时间: {elapsed:.2f}秒
最终结果: {len(self.best_solution.routes)}辆车, 成本{self.best_cost:.2f}元
相比初始解: 车辆数{len(self.current_solution.routes)}→{len(self.best_solution.routes)}, "
              f"成本{self.evaluate(self.current_solution):.2f}→{self.best_cost:.2f}
{'='*60}
""")
```

local_search.py:

```
import copy
from typing import List

class LocalSearch:
    """
    局部搜索算子集合
    先实现基础版本，确保能运行，再逐步增强
    """

    def __init__(self, problem_instance):
        self.problem = problem_instance
        self.max_local_iterations = 50
```

```
def basic_local_search(self, solution):
    """
    基础局部搜索：2-opt + 客户交换
    确保每次调用都有改进可能
    """
    improved = True
    iteration = 0

    while improved and iteration < self.max_local_iterations:
        improved = False
        iteration += 1

        for i, route in enumerate(solution.routes):
            if len(route) >= 3:
                optimized = self.two_opt(route)
                if self.evaluate_route(optimized) < self.evaluate_route(route):
                    solution.routes[i] = optimized
                    improved = True

        for i, route in enumerate(solution.routes):
            if len(route) >= 2:
                for j in range(len(route) - 1):
                    new_route = route.copy()
                    new_route[j], new_route[j+1] = new_route[j+1], new_route[j]

                    if self.is_feasible_route(new_route) and \
                       self.evaluate_route(new_route) < self.evaluate_route(route):
                        solution.routes[i] = new_route
                        improved = True

        if len(solution.routes) >= 2:
            merged = self.try_simple_merge(solution)
            if merged and self.evaluate(merged) < self.evaluate(solution):
                solution = merged
                improved = True
```

```
    return solution

def two_opt(self, route):
    """
    2-opt 优化：经典路径优化算法
    """
    if len(route) < 4:
        return route

    best = list(route)
    best_cost = self.evaluate_route(route)

    for i in range(1, len(route) - 2):
        for j in range(i + 1, len(route)):
            if j - i == 1:
                continue

            new_route = route[:i] + route[i:j][::-1] + route[j:]

            if self.is_feasible_route(new_route):
                new_cost = self.evaluate_route(new_route)
                if new_cost < best_cost:
                    best = new_route
                    best_cost = new_cost

    return best

def try_simple_merge(self, solution):
    """
    尝试合并两条短路径
    """
    routes_with_idx = [(i, route) for i, route in enumerate(solution.routes)]
    routes_with_idx.sort(key=lambda x: len(x[1]))

    for idx1, route1 in routes_with_idx:
```

```
            for idx2, route2 in routes_with_idx:
                if idx1 >= idx2:
                    continue

                combined = route1 + route2

                if self.is_feasible_route(combined):
                    new_solution = copy.deepcopy(solution)

                    route1_len = len(route1)
                    route2_len = len(route2)

                    if route1_len <= route2_len:
                        new_solution.routes[idx1] = combined
                        new_solution.routes[idx2] = []
                    else:
                        new_solution.routes[idx2] = combined
                        new_solution.routes[idx1] = []

                    new_solution.routes = [r for r in new_solution.routes if r]

                    optimized_route = self.two_opt(combined)
                    new_solution.routes.append(optimized_route)

                    return new_solution

        return None

    def is_feasible_route(self, route):
        """
        检查路径可行性（容量约束）
        """
        total_weight = 0
        total_volume = 0

        for customer_id in route:
```

```
            customer = self.problem.customers.get(customer_id)
            if customer is None:
                return False
            total_weight += getattr(customer, 'weight', 0)
            total_volume += getattr(customer, 'volume', 0)

            if total_weight > self.problem.vehicle_capacity_weight or \
               total_volume > self.problem.vehicle_capacity_volume:
                return False

        return True

    def evaluate_route(self, route):
        """
        评估单条路径的成本
        """
        if not route:
            return 0

        total_cost = 0
        prev_idx = 0

        for customer_id in route:
            customer_idx = self.problem.customer_ids.index(customer_id) + 1
            total_cost += self.problem.distance_matrix[prev_idx][customer_idx]
            prev_idx = customer_idx

        total_cost += self.problem.distance_matrix[prev_idx][0]

        return total_cost * 0.8

    def evaluate(self, solution):
        """
        评估整个解的总成本
        """
        total = 0
```

```
    for route in solution.routes:
        total += self.evaluate_route(route)
    return total

def guaranteed_improvement_local_search(self, solution):
    """
    确保每次调用都有改进的局部搜索
    尝试多种算子，选择最好的
    """
    best_solution = copy.deepcopy(solution)
    best_cost = self.evaluate(best_solution)

    operators = [
        self.two_opt_all,
        self.cross_exchange,
        self.or_opt
    ]

    for op in operators:
        new_solution = op(copy.deepcopy(solution))
        new_cost = self.evaluate(new_solution)

        if new_cost < best_cost:
            best_solution = new_solution
            best_cost = new_cost

    return best_solution

def two_opt_all(self, solution):
    """
    对所有路径应用 2-opt 优化
    """
    new_solution = copy.deepcopy(solution)
    for i, route in enumerate(new_solution.routes):
        if len(route) >= 4:
            new_solution.routes[i] = self.two_opt(route)
```

```
    return new_solution

def cross_exchange(self, solution):
    """
    路径间交叉交换：两条路径交换客户段
    """
    best_solution = copy.deepcopy(solution)
    best_cost = self.evaluate(best_solution)

    for r1_idx in range(len(solution.routes)):
        for r2_idx in range(r1_idx + 1, len(solution.routes)):
            route1 = solution.routes[r1_idx]
            route2 = solution.routes[r2_idx]

            if len(route1) < 2 or len(route2) < 2:
                continue

            for seg1_start in range(len(route1)):
                for seg1_end in range(seg1_start + 1, len(route1) + 1):
                    for seg2_start in range(len(route2)):
                        for seg2_end in range(seg2_start + 1, len(route2) + 1):
                            new_route1 = route1[:seg1_start] + route2[seg2_start:seg2_end] + route1[seg1_end:]
                            new_route2 = route2[:seg2_start] + route1[seg1_start:seg1_end] + route2[seg2_end:]

                            if self.is_feasible_route(new_route1) and self.is_feasible_route(new_route2):
                                new_solution = copy.deepcopy(solution)
                                new_solution.routes[r1_idx] = new_route1
                                new_solution.routes[r2_idx] = new_route2

                                new_cost = self.evaluate(new_solution)
                                if new_cost < best_cost:
                                    best_solution = new_solution
                                    best_cost = new_cost

    return best_solution
```

```
    def or_opt(self, solution, segment_length=3):
        """
        移动连续客户段：将一段连续客户移动到其他位置
        """
        best_solution = copy.deepcopy(solution)
        best_cost = self.evaluate(best_solution)

        for route_idx in range(len(solution.routes)):
            route = solution.routes[route_idx]

            for seg_len in range(1, min(segment_length + 1, len(route))):
                for seg_start in range(len(route) - seg_len + 1):
                    segment = route[seg_start:seg_start + seg_len]
                    remaining = route[:seg_start] + route[seg_start + seg_len:]

                    if not remaining:
                        continue

                    for insert_pos in range(len(remaining) + 1):
                        new_route = remaining[:insert_pos] + segment + remaining[insert_pos:]

                        if self.is_feasible_route(new_route):
                            new_solution = copy.deepcopy(solution)
                            new_solution.routes[route_idx] = new_route

                            new_cost = self.evaluate(new_solution)
                            if new_cost < best_cost:
                                best_solution = new_solution
                                best_cost = new_cost

        return best_solution
```

main_ils.py:

```
    def main():
    """
    ILS 主程序
    """
```

```
    print("=" * 60)
    print("迭代局部搜索(ILS) - 城市绿色物流配送调度优化")
    print("=" * 60)

    from problem_loader import load_problem_instance
    problem = load_problem_instance("data/problem1.json")

    from solution_loader import load_current_best_solution
    initial_solution = load_current_best_solution("results/genetic_best.json")

    if initial_solution is None:
        print("警告：无法加载现有解，将使用贪心构造初始解")

    from ils_solver import ILSSolver
    solver = ILSSolver(problem, initial_solution)

    print("\n 开始 ILS 优化过程...")
    best_solution = solver.solve()

    from result_saver import save_solution
    save_solution(best_solution, "results/ils_best.json")

    validate_solution(best_solution, problem)

    print("\nILS 优化完成！结果已保存到 results/ils_best.json")

def validate_solution(solution, problem):
    """
    验证解的有效性
    """
    print("\n 验证解的可行性...")

    served_customers = set()
    for route in solution.routes:
        served_customers.update(route)
```

```
    all_customers = set(range(1, problem.num_customers + 1))
    missing = all_customers - served_customers
    if missing:
        print(f"  ✗ 错误：{len(missing)}个客户未被服务")
        return False

    overload_count = 0
    for route in solution.routes:
        if not problem.is_route_feasible(route):
            overload_count += 1

    if overload_count > 0:
        print(f"  ✗ 错误：{overload_count}条路径超载")
        return False

    print(f"  ✓ 验证通过：{len(solution.routes)}条路径, {len(served_customers)}个客户被服务，无超载")
    return True

if __name__ == "__main__":
    main()
```

perturbation.py:

```
    import copy
import random

class Perturbation:
    """
    扰动算子：跳出局部最优
    """

    def __init__(self, problem_instance, strength=0.2):
        self.problem = problem_instance
        self.strength = strength

    def basic_perturb(self, solution):
        """
        基础扰动：随机移除部分客户，然后重插入
```

```
        """
        perturbed = copy.deepcopy(solution)

        all_customers = []
        for route in perturbed.routes:
            all_customers.extend(route)

        if not all_customers:
            return perturbed

        num_to_remove = max(1, int(len(all_customers) * self.strength))
        num_to_remove = min(num_to_remove, len(all_customers))
        customers_to_remove = random.sample(all_customers, num_to_remove)

        for customer_id in customers_to_remove:
            for route in perturbed.routes:
                if customer_id in route:
                    route.remove(customer_id)
                    break

        perturbed.routes = [r for r in perturbed.routes if r]

        for customer_id in customers_to_remove:
            best_route_idx, best_position = self.find_best_insertion(customer_id, perturbed)

            if best_route_idx is not None:
                perturbed.routes[best_route_idx].insert(best_position, customer_id)
            else:
                perturbed.routes.append([customer_id])

        return perturbed

    def find_best_insertion(self, customer_id, solution):
        """
        找到最佳插入位置（最小成本增加）
        """
```

```
    best_cost_increase = float('inf')
    best_route_idx = None
    best_position = None

    customer = self.problem.customers.get(customer_id)
    if customer is None:
        return None, None

    for route_idx, route in enumerate(solution.routes):
        route_weight = sum(
            getattr(self.problem.customers.get(cid), 'weight', 0)
            for cid in route
        )
        route_volume = sum(
            getattr(self.problem.customers.get(cid), 'volume', 0)
            for cid in route
        )

        if route_weight + getattr(customer, 'weight', 0) > self.problem.vehicle_capacity_weight or \
           route_volume + getattr(customer, 'volume', 0) > self.problem.vehicle_capacity_volume:
            continue

        for pos in range(len(route) + 1):
            new_route = route[:pos] + [customer_id] + route[pos:]
            cost_increase = self.calculate_insertion_cost(route, new_route, customer_id)

            if cost_increase < best_cost_increase:
                best_cost_increase = cost_increase
                best_route_idx = route_idx
                best_position = pos

    return best_route_idx, best_position

def calculate_insertion_cost(self, old_route, new_route, inserted_customer):
    """
    计算插入客户带来的成本增加
```

```
    """
    old_cost = self.evaluate_route(old_route) if old_route else 0
    new_cost = self.evaluate_route(new_route) if new_route else 0
    return new_cost - old_cost

def evaluate_route(self, route):
    """
    评估单条路径的成本
    """
    if not route:
        return 0

    total_cost = 0
    prev_idx = 0

    for customer_id in route:
        if customer_id in self.problem.customer_ids:
            customer_idx = self.problem.customer_ids.index(customer_id) + 1
            total_cost += self.problem.distance_matrix[prev_idx][customer_idx]
            prev_idx = customer_idx

    total_cost += self.problem.distance_matrix[prev_idx][0]

    return total_cost * 0.8
```